

FROM CLEO SOL TO NETTSPEND

Creates a playlist which can transition from one artist to another



Table Of Contents

Analysis	7
Introduction to the problem	7
Introduction to the stakeholder	7
Identifying stakeholders	7
Research of existing tools	9
Stakeholder Research	10
Stakeholder research methods	10
Stakeholder survey responses	10
Second Client interview	11
Key takeaways	11
Choosing key technologies	12
Primary coding language	12
Finding similar artists	12
Making a network	13
Spotify Authorization workflow	14
Creating the UI	15
Design Research	15
Critical Path	16
Problem Breakdown	17
Database	17
DatabaseManager	17
ArtistNetwork	17
WebServer	18
PlaylistCreator	18
Spotify Playlist Creation Prototyping	19
Sending POST + GET requests and parsing the responses	19
Authorization with PKCE Extention	19
Creating a playlist	21
Web Server Prototype	24

Front-end code	24
Back-end code	25
Test	26
Network Expansion Prototype	27
Breadth-First Search	27
Getting similar artists	27
Creating a network	27
Test:	29
Deciding on minimum popularity (3 rd Client Interview)	30
Program requirements	31
Documented Design	34
High-level overview	34
Class Diagram	34
Hierarchy + Dependencies	35
Explanation of classes	35
Program flowcharts	39
Expanding the network flowchart:	39
User flow flowchart	40
Making a database	41
Using microsoft sqlite	41
Designing the database	41
Diagram of database classes	42
Database Repository SQL statements	43
Artist Repository SQL statements	44
Connections Repository SQL statements	46
Representing Network at runtime	47
Overview	47
ArtistNode	47
ArtistNetwork	47
Calling External Services	49
Overview	49
LastFmApiExpanderService	49

SpotifyClientCredentialAuthorization	51
SpotifyExpanderApiService	51
SpotifyGetPlaylistSongs	52
SpotifyPkceCodeAuthorizationFlow	55
CreatePlaylistService	55
Expanding the Network	57
Overview	57
Normalising last.fm Data + Adding Spotify data	57
ArtistVerificationService	58
ArtistNetworkExpanderDatabaseService	59
ArtistNetworkExpander	59
Web Server	60
Overview	60
Initialising the server	61
Api Class	61
Website	63
Overview	63
index.html	64
report issue modal	64
view-all-artists.html	65
callback.html	65
Logic (script.js)	65
Technical Solution	69
Technical skills	69
Programming techniques	71
Global Types	72
ArtistNode	72
Database Services	72
Interfaces:	72
ArtistRepository	73
Connection Repository	76
Database Repository	77

External Web Services 78

Interfaces	78
CreatePlaylistService	78
GetPlaylistSongsService	81
LastFmApiService	86
SpotifyClientCredentialAuthorizer	88
SpotifyExpanderService	89
SpotifyPkceCodeAuthorizer	92

NetworkExpander 94

Interfaces	94
Program.cs (program entry point #1)	94
NetworkExpanderDatabaseService	95
ArtistVerificationService	96
ArtistNetworkExpander	97

Web Server 100

Interfaces	100
ArtistNetworkDatabaseService	100
ArtistNetwork	101
ArtistNetworkDatabaseService	102
API	102
Program.cs (entry point 2)	105
WebServerDatabaseService	106

Website 107

index.html	107
view-artists.html	108
callback.html	108
script.js	108
report.js	111
styles.css	112

Testing 117

Overview 117

Black Box Testing 117

White Box Testing	119
Evaluation	130
Objectives Self Evaluation	130
User Feedback	130
Playlist transition feedback	130
Design Survey feedback	132
Interview with main client	132
Improvements + Final Conclusion	133
Conclusion	134
References	135

understanding of musical artists, and is interested in a solution to find a good path between two different artists using similar connections. He is also a Computer Science student, meaning that he will not demand more than is realistic, and will be available for regular communication.

Other shareholders:

NEA Project - Outreach 3a

4. Do you use any streaming services? *

☐ Yes

☐ No

5. What is your favorite? *

6. How old are you? *

☐ Under 16

☐ 16-19

☐ 20-24

☐ 25-34

☐ 35-44

☐ 45-54

☐ 55+

7. If so, what streaming service do you use? *

☐ Spotify

☐ Apple Music

☐ YouTube Music

☐ Hi-Fidelity streaming option (e.g., Tidal)

☐ Other

8. How would you rate your knowledge of music artists? *

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10

Very Little I know a lot

9. Would you be interested in a tool to find the shortest path between two completely different music artists, based on characteristics such as shared featured songs, "fans also like" and shared genres? *

☐ Yes, definitely

☐ Yes, maybe

☐ I might be interested and I'll have to see

☐ No, not at all

☐ I don't know

10. What is your name? *

11. How old are you? *

☐ Under 16

☐ 16-19

☐ 20-24

☐ 25-34

☐ 35-44

☐ 45-54

☐ 55+

12. How often do you listen? *

☐ More than 2 hours a day

☐ Between 1-2 hours a day

☐ Less than 1 hour a day

13. Do you use any streaming services? *

☐ Yes

☐ No

14. If so, what streaming service do you use? *

☐ Spotify

☐ Apple Music

☐ YouTube Music

☐ Hi-Fidelity streaming option (e.g., Tidal)

☐ Other

15. How would you rate your knowledge of music artists? *

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10

Very Little I know a lot

16. Would you be interested in a tool to find the shortest path between two completely different music artists, based on characteristics such as shared featured songs, "fans also like" and shared genres? *

☐ Yes, definitely

☐ Yes, maybe

☐ I might be interested and I'll have to see

☐ No, not at all

☐ I don't know

First Zoom interview with client

Q: What sticks out as something that you really don't like when using Spotify API tools?

A: I hate it when they take very long to answer; my curiosity wanes fast when they take forever to answer a query. Also, when the UI is overcomplicated and slow, or outdated, I don't like that. In fact, I would want the UI to be as simple as possible so it doesn't outdate itself.

Q: Have you used any specifically systems that find connections between Spotify artists?

A: Yes, Boil the Frog, when it still worked

Q: What have been your positive/negative interactions with Boil the Frog

A: Well, in some cases, it does a really good job of answering the question “What links this artist to another”. It does especially well with artists that are popular, like Prince, or Drake, or Lady Gaga. I also really like the idea of a playlist that transitions from one artist to another.

Q: What about your negative interactions with Boil the Frog

A: Firstly, sometimes I want to be able to change how many songs are added to the playlist per artist. Also, most of the time when I am curious, I want to find the link between two obscure artists, and it doesn't answer most of the time in that situation, even with artist that are quite well known and have made many songs with lots of artists, like Ken Carson.

Q: Well, how obscure are we talking?

A: I mean, if someone has more than 500,000 monthly listeners, I would expect to find them on the graph. But also, older artists who haven't done anything recently and are maybe just riding on the coattails of one very big song, maybe they can have a higher floor

A: I guess a website is nice, but what would really be good would be a Spicetify application

Q: Spicetify? What's that?

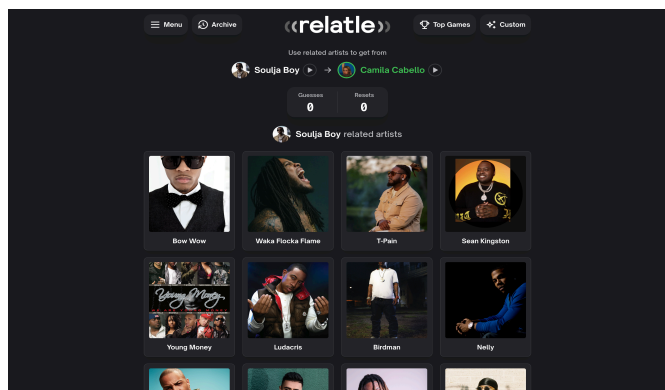
A: Spicetify is a tool that allows you to customize desktop Spotify through themes, extensions, or custom applications.

Q: So what would really be good is something like Boil the Frog, that creates playlists that transition from one artist to another but works within the Spicetify framework, visualizes the connection between artists and works with more obscure artists

A: Yes, exactly. Having something to explore within the Spotify desktop app will be super convenient

Research of existing tools

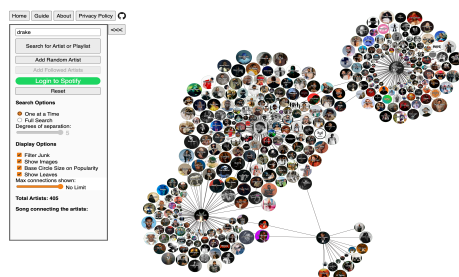
<https://relatle.io/>



Relatle is a daily guessing game where the player needs to find the link between two random artists based on their related artists. It uses the Spotify “Fans also like” data to generate the list of related artists that the user must guess from. It also can find the shortest path between artists, albeit that is not the main functionality of the program.

This is one of the few solutions that actually uses the “fans also like” feature to create links between artists, which makes it unique. It also provides the shortest distance between artists, which is useful. However, it’s limited to a relatively small pool of popular artists, so it doesn’t work well for more niche discoveries. Since it’s mainly designed for the game, finding the shortest path between artists takes quite a few clicks. Plus, it only uses one piece of Spotify data—related artists—without considering other factors that could connect them.

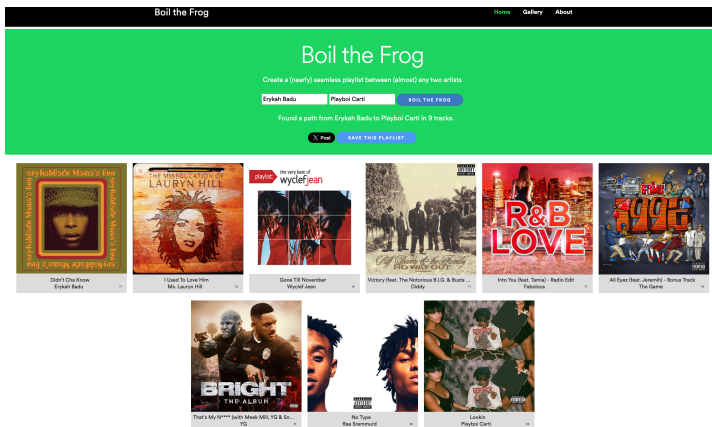
<https://everything-connected.com/>



This is an online source that creates links between artists based on whether they have a song together. You can start with one artist, and then search for another, and if the two artists have a mutual artist that they share songs with then it will reflect that on the graph. The size of the circle reflects the popularity of the artist

As it queries the API at the time of request, it is able to access every artist on Spotify. which is a huge advantage, and it also has some interesting visualizations. However, it doesn’t have a way to find the shortest path between artists, which is a major limitation. The links between artists are only based on shared features, rather than a broader set of factors. It’s also not optimized for mobile, and loading different artists takes a long time, which makes navigation frustrating.

<http://boilthefrog.playlistmachinery.com/>



Perhaps the most like what I want to achieve by the end of my project, Boil the Frog creates a playlist that transitions from one artist to another completely different artist. Both artists must be within top 100,000 popular artists

Instead of just relying on shared songs or “fans also like,” it actually uses an algorithm which takes multiple factors,

which is a nice change. It is the only which can generate a path between two artists and even create a playlist that smoothly transitions from one to the other. But it doesn’t visualize how the artists are connected, which makes it less intuitive. It also struggles with more obscure artists, so it’s not great for deep dives. However, the most important problem is that it relies on a discontinued “relation between artists” value, meaning it no longer works

Stakeholder Research

Stakeholder research methods

To communicate with my stakeholders, I will send out a survey out using Google Forms, with the following question for them to answer

For other decisions that are not easily communicated in a survey, I will be consulting with my main client, Riccardo Sasso



Second Client interview

Q: What are some things that you find appealing about other websites that offer similar Spotify API tools, visually?

A: Yeah, so I like when they use Spotify green and stick to Spotify's design language. It just feels more familiar and cohesive that way. Also, speed is a big one—it has to be fast and responsive. I don't want to wait around for things to load. Simplicity is key too; it should be uncomplicated and easy to use. And it should have links to Spotify so I can easily jump between the tool and my actual Spotify account.

Q: When you say that like when API tools use Spotify's design language, what specifically do you mean

A: I think the key points is using Spotify's colour palate, so black for the background, white for text and that specific green for accents. Oh, and using Spotify's font would be really good. Everything else like the rounded buttons etc is not too important to me.

Q: What else would you like to be able to see on the design?

A: One thing missing in Boiling the Frog is a way to see all the artists that exist within the network. That would be useful. Also, there should be an easy way to report problems—like if something isn't working right, I shouldn't have to dig around to figure out where to send feedback. A place to request artists would be nice too, in case something's missing. And I really liked how Boiling the Frog had its source code available along with an explanation of how it worked. Transparency like that is always a plus.

Q: What should the name of the playlist be?

A: I like the idea of calling it "From ___ to ___." It just makes sense since it's mapping a journey between two artists.

Q: What would be your the maximum time from opening the website to having the playlist in your library?

A: No more than two minutes. Anything longer than that and it starts feeling slow.

Q: What are some other general priorities that you have?

A: Keeping my login data safe is a big one

Key takeaways

The survey showed that Spotify usage is **evenly** split between desktop and other platforms, but most users don't know how to install CLI tools and are more likely to visit a website than use an app. Functionality is the main priority over design, and most users think the cutoff for similar artists should be around 5-8. When it comes to data storage, the majority prefer saving to their personal library rather than a shared one. The interview reinforced some key client needs—they want the tool to match Spotify's design language, display the full artist network, and include features for reporting issues and requesting artists. They also expect the playlist name to follow a "From ___ to ___" format based on the start and end artists, and the whole process shouldn't

Primary coding language

While I have experience with Python and JavaScript, C# stands out as the language I've spent the most time with. One of the most compelling reasons to choose C# is the sheer number of libraries and frameworks it offers. Specifically, ASP.NET Core is a huge advantage for seamlessly linking the logic to the client-side, which will be crucial for my project. Additionally, C# tends to be faster than Python in many cases, especially in multi-threaded or high-performance scenarios. This speed is important, particularly because my clients prioritize fast execution, and I want to ensure the solution performs well under load.

Finding similar artists

To find a path between any two artists, I will construct a network with nodes as artists and edge weight as the strength of connection between two artists. I have two main API services which I can gather data to determine the similarity between artists: Last.FM or the Spotify API. Both work the same fundamentally; deciding on how similar two artists are based on shared listening habits of their respective users.

Last.fm is a website which hosts a music recommender system called “Audioscrobbler” and provides a simple to use API. The key endpoint is “artist.getSimilar”, which when given a starting artists, can return similar artists and a corresponding “weight” value based on how similar the two artists are.

Spotify also hosts an API which returns a list of similar artists given a starting artist. The main difference between Spotify and Last.fm is that it does not provide a weight of connection between artists, although this can be overcome by assigning the order in which artists are ranked a weight. Using the Spotify API also give the opportunity to include other data into the weight by using different calls, for example using the v1 search endpoint to search for shared features and including that in the weight of connection.

Based on shareholder research, my users would rather have lots of artists rather than high quality connections, so using Last.fm is the most reasonable choice

IMPORTANT UPDATE: AS OF NOVEMBER 27 2024, SPOTIFY HAS DEPRECATED THEIR GET ARTIST’S RELATED ARTISTS ENDPOINT DUE TO CONCERNS THAT THE DATA WAS BEING USED TO TRAIN AI MODELS

CHOICE: LAST.FM

Making a network

2D Array	Dictionary
+ Fast lookups: $O(1)$ time complexity to check if an edge exists. + Simple implementation with fixed-size memory allocation. + Efficient for dense graphs where most nodes are connected. - High memory usage: $O(n^2)$ space complexity - Inefficient for sparse graphs - Adding/removing nodes is costly since resizing requires rebuilding the matrix.	+ Efficient for sparse graphs: $O(n + m)$ space complexity (only stores existing edges). + Adding/removing nodes and edges is more flexible. + Uses less memory compared to a 2D array in most cases. - Slower lookups: Checking if an edge exists takes $O(\text{degree})$, worse than $O(1)$

To find a path between the two artists, I will need to make a network, with the nodes representing the artists and the edges representing how similar the two artists are. There are two ways store a network computationally; using a 2d array or using a dictionary

A 2D array is a straightforward way to represent a network, where each row and column correspond to an artist, and the values at the intersections indicate the similarity between them. This method is particularly useful if the network is dense, meaning most artists have

some level of similarity with many others. To implement this, I would first create an array where the indices map directly to specific artists. If there are n artists, the array would be $(N \times N)$ with a value of zero for artists that have no similarity and a positive value for those that do. Accessing the similarity between two artists is efficient because it requires only an index lookup. However, this approach has drawbacks, if the network is sparse, meaning most artists are not directly connected, a 2D array wastes a lot of space storing zeros.

A dictionary-based implementation, on the other hand, is more efficient for sparse networks. Here, each artist is a key in the dictionary, and the value is another dictionary storing only the artists they have a similarity with and the corresponding similarity score. This avoids storing unnecessary data and makes it easier to dynamically add new connections without needing to resize an entire array. For example, if an artist has only a few direct similarities, their dictionary entry would contain only those relationships instead of reserving space for every possible connection. While this method saves memory, it can be slightly slower in some cases since dictionary lookups, while fast, are not as direct as array indexing. In my case, as the relationships between artists are relatively sparse, the dictionary-based approach is the better choice.

CHOICE: Dictionary

Spotify Authorization workflow

Authorization flow	Authorization flow w/PKCE extension
+ More widely supported and well-documented. - Requires a client secret, which can be exposed if mishandled. - Susceptible to authorization code interception if the client secret is leaked.	+ More secure as it doesn't require a client secret. - Slightly more complex due to PKCE challenge-verifier mechanism.

To create and add a playlist to a user's Spotify library, I must use either the authorization code flow or the authorization with PKCE code flow. Both methods require the user to grant permission for private scopes, like `playlist-modify-private`, but they handle authentication differently.

The authorization code flow is the standard OAuth 2.0 method that requires a client secret. First, I direct the user to Spotify's authorization URL with the required scopes. If they approve, Spotify redirects them back with an authorization code. I then exchange this code for an access token by making a request to Spotify's token endpoint, which requires my client ID, client secret, and the same redirect URI. The access token lets me modify their playlists, and I can refresh it using the refresh token provided in the response.

The authorization with PKCE (Proof Key for Code Exchange) flow is designed for security in public-facing apps, like web or mobile apps, where storing a client secret isn't safe. Instead of using a client secret, I generate a random code verifier and create a hashed version called the code challenge. When I send the user to Spotify's authorization page, I include the code challenge. Once they approve and I get the authorization code, I exchange it for an access token

by sending the code verifier instead of a client secret. This prevents intercepted authorization codes from being used by attackers.

As my primary client has stated that one of the general priorities is safety, I will use the PKCE flow as it is generally the more secure method for applications which cannot keep the user token safe

CHOICE: PKCE

Creating the UI

Spicetify	Website
+ Fully integrated in the Spotify desktop client, do not have to go to a different website - As it is an unofficial technology there are some compatibility and performance issues - Does not work on mobile	+ Works on all platforms + Best performance + Easiest development - Less integrated with Spotify

Spicetify is an open-source tool for customizing Spotify's desktop app, letting users modify the UI and add new features. It supports installing tools through its built-in "Marketplace" app or directly via the CLI using GitHub repos. My main client suggested it as an option, but it only makes sense if: 1) most shareholders use Spotify's desktop client, 2) they're comfortable installing CLI tools, and 3) design matters more than speed. None of these are true, so a website is the better choice, with the bonus that it works cross-platform.

CHOICE: Website

Design Research

Colour Palette

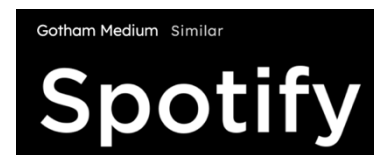
One key takeaway from my client interview is that they want a design which uses Spotify's palette and fonts.

Font

Spotify uses a proprietary font called "Spotify Mix". From sources online, the most similar open source font is Gotham Medium

Spotify brand hex, RGB, CMYK and Pantone® (PMS) color codes

Green				
Hex Code	#1db95b			
RGB values	(31, 185, 92)			
CMYK values	(83%, 0%, 46%, 0%)			
Pantone®	Pantone 349 C			
White				
Hex Code	#ffffff			
RGB values	(255, 255, 255)			
CMYK values	(0%, 0%, 0%, 0%)			
Pantone®	Pantone N			
Black				
Hex Code	#000000			
RGB values	(0, 0, 0)			
CMYK values	(100%, 100%, 100%, 100%)			
Pantone®	Pantone N			



Critical Path

My deadline for my project is the 14th of March, but I want to be able to finish it by the end of february. Here are the key milestones I know I will need to complete and the rough time I expect them to take

Phase 1: Prototype all key technologies

Phase 2: Create a network and a way to persistently save the network in a databse

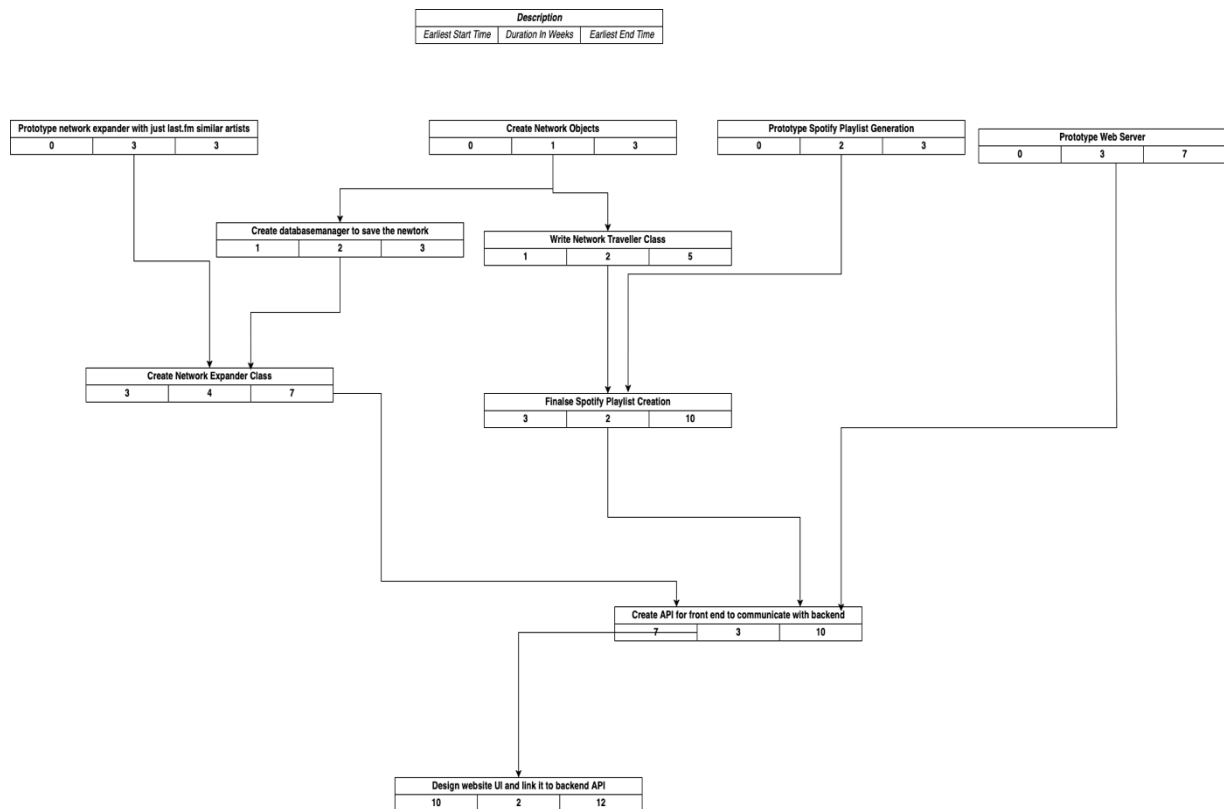
Phase 3: Create the networkexpander and networktraveller logic, and can generate a network of artists from a given starting artist and find the shortest path between them

Phase 3: Program is able to create a playlist from the path given and add it to the user library

Phase 4: Create an Asp.Net core web server which has POST and GET api calls which my website can use

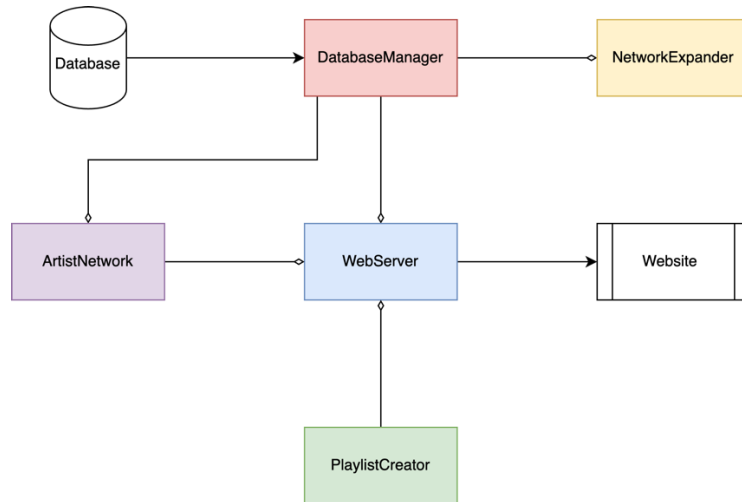
Phase 5: Website is designed and add finishing touches

Here is a prescedence diagram which breaks down what I need to do:



Problem Breakdown

Here is a diagram of the key components of my problem:



Database

Stores the key information about the artists and connections required

Key components:

Artist Table – stores information about the artists

Connection table – stores information about the connection between artists

DatabaseManager

Contains the relevant classes to interact with the database

Key methods

GetNetwork – returns the network for the artist network to use

AddArtist – gets all the fields for information about the artist

AddConnection – adds an connection between two artists to the database

GetArtist – returns key information for an artist

GetConnection – returns key information about the connection between artists

ArtistNetwork

Represents the network at runtime

Key Fields

AdjacencyMatrix – A 2D Dictionary of key information about an artist

Key Methods

FindPathWithDijkstras – Uses dijkstras algorithm to find the shortest path between artists

WebServer

Represents the web server for the website to interact with

Key Methods

FindPath – finds the shortest path between two artists (using artistnetwork)

Authenticate – handles user login to spotify

Create playlist – handles user creating playlist

PlaylistCreator

Handles playlist creation

Key Methods

GetSongs – Gets the songs given the path of artists

CreatePlaylist – Creates the playlist in the user library

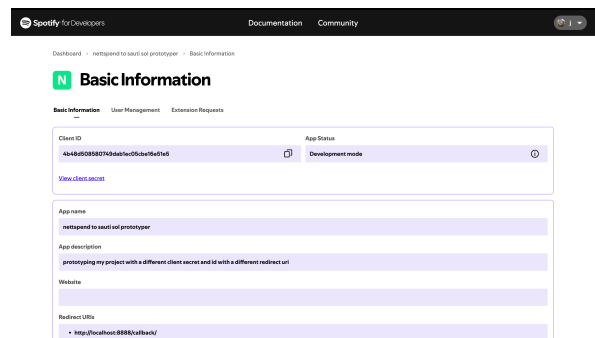
Spotify Playlist Creation Prototyping

Sending POST + GET requests and parsing the responses

To interact with the relevant API'S, I will need to get familiar with the format required to send GET and POST requests and parsing the relevant responses. GET requests are to get data from the server (for example, an artists unique ID), and POST requests are usually to send data to the server (for example, adding a playlist to a user playlist) The key libraries to do this in C# is [System.Text.Json](#) and [System.Net.Http](#)

Authorization with PKCE Extention

One form of authentication which I will use is the Authorization with PKCE extention. PKCE (Proof Key for Code Exchange) is an OAuth 2.0 extension that makes authorization more secure for public clients like mobile and web apps. To prototype playlist creation, which includes access to the private scope "playlist-modify-public", I will need to make a basic authorization class



Firstly, I will need to register an app with Spotify's API developer program

This gives me access to my own public clientId and a private client secret.

Then, I wrote a program which follows the PKCE flow. It works by generating a random string in [GenerateCodeVerifier](#) which is then hashed in SH256 using [GenerateCodeChallenge](#) before the authorization request. The program opens the browser and user logs in to spotify, after which the authorization server can issue an authorization code.

With this authorization code, the server can send back the original code verifier (original randomly generated string) in [ExchangeAuthorizationCodeForAccessToken](#). Here is a prototype of an authorization flow I wrote to practice creating the playlist. The main difference between this and my final authorization for the final project is that I will include randomly generated states so that the PKCE token is never sent to the user, and also so that it can handle multiple servers

```

using System.Net;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using System.Web;

namespace nettspendtosautisolprototyping;

public class SpotifyPkce
{
    private readonly string _clientId = "4b48d508580749dab1ec05cbe16e51e5"; // this is my application
    id, it does not need to be kept secret
    private readonly string _redirectUri = "http://localhost:8888/callback/"; // this is my callback
    uri which i specified in my application
    private string? _accessToken; // final access token for the user

    public string GetAuthoizationPKCEAccessToken() // returns pkce access token
    {
        if (!string.IsNullOrEmpty(_accessToken))
            return _accessToken;
        string codeVerifier = GenerateCodeVerifier();
        string codeChallenge = GenerateCodeChallenge(codeVerifier);
        using HttpListener listener = new(); // listens on redirect uri for callback from user logging
in
        listener.Prefixes.Add(_redirectUri);
        listener.Start();
        OpenBrowserForAuthorization(codeChallenge); // opens browser so user can login to spotify

        var context = listener.GetContext();
        var query = context.Request.Url?.Query;
        listener.Stop();
        if (string.IsNullOrEmpty(query) || !query.Contains("code=")) // if user rejects to login
            throw new Exception("Authorization failed. No code received.");
        string authorizationCode = HttpUtility.ParseQueryString(query).Get("code")!;
        return ExchangeAuthorizationCodeForAccessToken(authorizationCode, codeVerifier);
    }

    private void OpenBrowserForAuthorization(string codeChallenge)
    {
        // sends the code challenge which is just a hased random number to spotify
        string authUrl =
        $"https://accounts.spotify.com/authorize?response_type=code&client_id={_clientId}&redirect_uri={Uri.EscapeDataString(_redirectUri)}&code_challenge_method=S256&code_challenge={codeChallenge}&scope=playlist-modify-private";

        Console.WriteLine("Opening browser for authorization...");
        System.Diagnostics.Process.Start(new System.Diagnostics.ProcessStartInfo
        {
            FileName = authUrl,
            UseShellExecute = true,
        });
    }

    private string ExchangeAuthorizationCodeForAccessToken(string authorizationCode, string
codeVerifier)
    {
        using HttpClient client = new();
        var response = client.PostAsync("https://accounts.spotify.com/api/token",
        new FormUrlEncodedContent(new[]
        {
            new KeyValuePair<string, string>("grant_type", "authorization_code"),
            new KeyValuePair<string, string>("code", authorizationCode),
            new KeyValuePair<string, string>("redirect_uri", _redirectUri),
            new KeyValuePair<string, string>("client_id", _clientId),
            new KeyValuePair<string, string>("code_verifier", codeVerifier)
        })).Result; // sends the authorization code recieved from spotify, the client id, the
redirect url and the
// original code verifier to spotify, in exchange for the access token
        if (!response.IsSuccessStatusCode)
            throw new Exception($"Failed to exchange authorization code. Status:
{response.StatusCode}");
        string jsonResponse = response.Content.ReadAsStringAsync().Result;
        var document = JsonDocument.Parse(jsonResponse);
    }
}

```

```

_accessToken = document.RootElement.GetProperty("access_token").GetString();
return _accessToken!;
}
private static string GenerateCodeVerifier() // generates a random string
{
    byte[] bytes = new byte[32];
    using RandomNumberGenerator rng = RandomNumberGenerator.Create();
    rng.GetBytes(bytes);
    return Convert.ToBase64String(bytes)
        .Replace("+", "-")
        .Replace("/", "_")
        .Replace("=", "");
}
private static string GenerateCodeChallenge(string codeVerifier) // hashes the code verifier
{
    using SHA256 sha256 = SHA256.Create();
    byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(codeVerifier));
    return Convert.ToBase64String(bytes)
        .Replace("+", "-")
        .Replace("/", "_")
        .Replace("=", "");
}
}

```

Creating a playlist

Now that I have an authorization code, I can test the API endpoints I need for the playlist creation. I wrote a program in which I log in, enter the name of the playlist, the length of the playlist and the names of the songs I want to add to the playlist, and spotify adds the playlist to my library with the songs I specified and with the name I specified. This will allow me to practice all the key endpoints which I will need to use in my final code's playlist creation, as well as sending POST and GET requests to HTTP servers and parsing the results into JSON

https://api.spotify.com/v1/search	https://api.spotify.com/v1/me	https://api.spotify.com/v1/users/{user_id}/playlists	https://api.spotify.com/v1/playlists/{playlist_id}/tracks
GET /search Search for items using the Spotify Search API. The search results are returned in a JSON object. The search results are returned in a JSON object. The search results are returned in a JSON object. Parameters: q (string): The search query. type (string): The type of item to search for. Possible values: album, artist, playlist, track, show, episode. limit (integer): The number of items to return. Default: 20. offset (integer): The offset of the items to return. Default: 0. market (string): The market to search in. Default: 'all'. Response: A JSON object containing the search results. The structure is as follows: <pre> { "albums": { "items": [{ "album_type": "album", "artists": [{ "external_urls": { "spotify": "https://open.spotify.com/artist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "name": "Drake", "type": "artist", "uri": "spotify:artist:1dfeix48Wt8e0A581264" }], "external_urls": { "spotify": "https://open.spotify.com/album/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "total_tracks": 15, "type": "album", "uri": "spotify:album:1dfeix48Wt8e0A581264" }], "limit": 20, "offset": 0, "total": 1 }, "artists": { "items": [{ "external_urls": { "spotify": "https://open.spotify.com/artist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "name": "Drake", "type": "artist", "uri": "spotify:artist:1dfeix48Wt8e0A581264" }], "limit": 20, "offset": 0, "total": 1 }, "playlists": { "items": [{ "collaborative": false, "description": null, "external_urls": { "spotify": "https://open.spotify.com/playlist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": null, "name": "Drake", "owner": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "private": false, "public": true, "snapshot_id": null, "tracks": { "items": [{ "added_at": "2016-10-29T12:00:00Z", "added_by": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "track": { "album": { "album_type": "album", "artists": [{ "external_urls": { "spotify": "https://open.spotify.com/artist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "name": "Drake", "type": "artist", "uri": "spotify:artist:1dfeix48Wt8e0A581264" }], "external_urls": { "spotify": "https://open.spotify.com/album/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "total_tracks": 15, "type": "album", "uri": "spotify:album:1dfeix48Wt8e0A581264" }, "duration_ms": 215000, "explicit": false, "external_urls": { "spotify": "https://open.spotify.com/track/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "is_local": false, "name": "Drake", "popularity": 100, "preview_url": "https://p.scdn.co/mp3-preview/1dfeix48Wt8e0A581264", "track_type": "track", "uri": "spotify:track:1dfeix48Wt8e0A581264" } }], "limit": 20, "offset": 0, "total": 1 }, "type": "playlist", "uri": "spotify:playlist:1dfeix48Wt8e0A581264" }], "limit": 20, "offset": 0, "total": 1 }, "shows": { "items": [{ "external_urls": { "spotify": "https://open.spotify.com/show/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "publisher": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "total_episodes": 15, "type": "show", "uri": "spotify:show:1dfeix48Wt8e0A581264" }], "limit": 20, "offset": 0, "total": 1 }, "episodes": { "items": [{ "external_urls": { "spotify": "https://open.spotify.com/episode/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "publisher": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "duration_ms": 215000, "explicit": false, "external_urls": { "spotify": "https://open.spotify.com/episode/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "is_local": false, "name": "Drake", "popularity": 100, "preview_url": "https://p.scdn.co/mp3-preview/1dfeix48Wt8e0A581264", "track_type": "episode", "uri": "spotify:episode:1dfeix48Wt8e0A581264" }], "limit": 20, "offset": 0, "total": 1 } } </pre>	Request GET /me	Web API: Endpoints (Playlists) Create Playlist Create Playlist [Docs] Create a playlist for a Spotify user. The playlist will be empty until you add songs. Each user is generally limited to a maximum of 1000 playlists. Authentication scopes - playlist-modify-public - playlist-modify-private Request POST /users/{user_id}/playlists Parameters name (string): The name of the playlist. The name must be unique and cannot be longer than 50 characters. description (string): The description of the playlist. The description must be unique and cannot be longer than 300 characters. collaborative (boolean): Whether the playlist is collaborative. Default: false. public (boolean): Whether the playlist is public. Default: false. Response A JSON object containing the playlist information. The structure is as follows: <pre> { "collaborative": false, "description": null, "external_urls": { "spotify": "https://open.spotify.com/playlist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": null, "name": "Drake", "owner": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "private": false, "public": true, "snapshot_id": null, "tracks": { "items": [{ "added_at": "2016-10-29T12:00:00Z", "added_by": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "track": { "album": { "album_type": "album", "artists": [{ "external_urls": { "spotify": "https://open.spotify.com/artist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "name": "Drake", "type": "artist", "uri": "spotify:artist:1dfeix48Wt8e0A581264" }], "external_urls": { "spotify": "https://open.spotify.com/album/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "total_tracks": 15, "type": "album", "uri": "spotify:album:1dfeix48Wt8e0A581264" }, "duration_ms": 215000, "explicit": false, "external_urls": { "spotify": "https://open.spotify.com/track/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "is_local": false, "name": "Drake", "popularity": 100, "preview_url": "https://p.scdn.co/mp3-preview/1dfeix48Wt8e0A581264", "track_type": "track", "uri": "spotify:track:1dfeix48Wt8e0A581264" } }], "limit": 20, "offset": 0, "total": 1 }, "type": "playlist", "uri": "spotify:playlist:1dfeix48Wt8e0A581264" } </pre>	Web API: Endpoints (Playlists) Add Items to Playlist Add Items to Playlist [Docs] Add one or more items to a Spotify playlist. Authentication scopes - playlist-modify-public - playlist-modify-private Request POST /playlists/{playlist_id}/tracks Parameters uris (array of strings): The URIs of the items to add. The URIs must be in the format: spotify:album:1dfeix48Wt8e0A581264, spotify:track:1dfeix48Wt8e0A581264, or spotify:episode:1dfeix48Wt8e0A581264. timestamps (array of strings): The timestamps of the items to add. The timestamps must be in the format: 1234567890123. Response A JSON object containing the playlist information. The structure is as follows: <pre> { "collaborative": false, "description": null, "external_urls": { "spotify": "https://open.spotify.com/playlist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": null, "name": "Drake", "owner": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "private": false, "public": true, "snapshot_id": null, "tracks": { "items": [{ "added_at": "2016-10-29T12:00:00Z", "added_by": { "external_urls": { "spotify": "https://open.spotify.com/user/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "type": "user", "uri": "spotify:user:1dfeix48Wt8e0A581264" }, "track": { "album": { "album_type": "album", "artists": [{ "external_urls": { "spotify": "https://open.spotify.com/artist/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "name": "Drake", "type": "artist", "uri": "spotify:artist:1dfeix48Wt8e0A581264" }], "external_urls": { "spotify": "https://open.spotify.com/album/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "images": [{ "height": 640, "url": "https://i.scdn.co/image/ab67616d0000b2731dfeix48Wt8e0A581264", "width": 640 }], "name": "Drake", "release_date": "2016-10-29", "release_date_precision": "day", "total_tracks": 15, "type": "album", "uri": "spotify:album:1dfeix48Wt8e0A581264" }, "duration_ms": 215000, "explicit": false, "external_urls": { "spotify": "https://open.spotify.com/track/1dfeix48Wt8e0A581264" }, "id": "1dfeix48Wt8e0A581264", "is_local": false, "name": "Drake", "popularity": 100, "preview_url": "https://p.scdn.co/mp3-preview/1dfeix48Wt8e0A581264", "track_type": "track", "uri": "spotify:track:1dfeix48Wt8e0A581264" } }], "limit": 20, "offset": 0, "total": 1 }, "type": "playlist", "uri": "spotify:playlist:1dfeix48Wt8e0A581264" } </pre>

Response 200 401 403 429 Search response ▶ tracks object ▶ artists object ▶ albums object ▶ playlists object ▶ shows object ▶ episodes object	Response 200 401 403 429 A user country string The country of the user, as set in the user's account profile. See ISO 3166-2 alpha-2 country codes display_name string The name displayed on the user's profile, null if not available. email string The user's email address, as entered by the user when creating their account. Important! This field is only available when the current user has granted the <code>email</code> scope. ▶ explicit_content object The user's explicit content settings. This field is only available when the current user has granted the <code>explicit-content</code> scope. ▶ external_urls object Known external URLs for this user. ▶ followers object Information about the followers of the user. href string A link to the Web API endpoint for this user. id string The Spotify user ID for the user. ▶ images array of ImageObject The user's profile image. product string The user's Spotify subscription level: "premium", "free", etc. (The subscription level "open" is not available.) type string The object type: "user" uri string The Spotify URI for the user.	Response 201 401 403 429 A playlist ▶ collaborative boolean true if the owner allows other users to modify the playlist. description string The playlist description. Only returned for modified, verified playlists, otherwise null. ▶ external_urls object Known external URLs for this playlist. ▶ followers object Information about the followers of the playlist. href string A link to the Web API endpoint providing full details of the playlist. id string The Spotify ID for the playlist. ▶ images array of ImageObject Images for the playlist. The array may be empty or contain up to three images. The images are returned in order of decreasing importance. name string The name of the playlist. ▶ owner object The user who owns the playlist public boolean The playlist's public/private status (if it is added to the user's profile). true if the playlist is public. snapshot_id string The version identifier for the current playlist. Can be supplied in other requests to target a specific version of the playlist. ▶ tracks object The tracks of the playlist. type string The object type: "playlist" uri string The Spotify URI for the playlist.	Response 201 401 403 429 A snapshot ID for the playlist snapshot_id string Example: "abc"
This is how I am able to search for songs by name and get the spotify ID of the song, which is required to add to songs to the playlist.	This is where I am able to get the spotify Id of the current artist based on their authorization earlier	This is the url which I will send the POST request to create a playlist in the user library.	This is the url which I will send the POST request to add tracks to the playlist which I created in the step before

```

SpotifyPkce authorizer = new SpotifyPkce();
var pkceToken = authorizer.GetAuthoizationPKCEAccessToken(); //gets the pkce token. THIS IS NOT A SAFE
WAY TO DO THIS. WILL CHANGE FOR FINAL PROJECT
Console.WriteLine("Please enter the name of the playlist: ");
string playlistName = Console.ReadLine();
Console.WriteLine("Please enter the length of the playlist: ");
int playlistLength = int.Parse(Console.ReadLine());
string[] songs = new string[playlistLength];
for (int i = 0; i < playlistLength; i++)
{
    Console.WriteLine($"Enter the name of song {i + 1}: ");
    string song = Console.ReadLine();
    songs[i] = song;
}
string[] songIds = new string[songs.Length];
client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", pkceToken); //this allows me to access both the scoped and
non-scoped data from spotify

HttpResponseMessage userIdResponse = client.GetAsync("https://api.spotify.com/v1/me").Result; //sends
get request for users id, next few lines is parsing

string stringUserIdResponse = userIdResponse.Content.ReadAsStringAsync().Result;
JsonDocument userIdDocument = JsonDocument.Parse(stringUserIdResponse);
string userId = userIdDocument.RootElement.GetProperty("id").GetString(); //this gets the users id
which i can use to add songs to teh playlist
for (int i = 0; i < songs.Length; i++)
{
    string song = songs[i];
    string url =
    $"https://api.spotify.com/v1/search?q={Uri.EscapeDataString(song)}&type=track&limit=1"; //use the

```

```

spotify v1 search endpoint to find the ids. limits to one response

HttpResponseMessage songResponse = client.GetAsync(url).Result;
string songResponseContent = songResponse.Content.ReadAsStringAsync().Result;
JsonDocument songDocument = JsonDocument.Parse(songResponseContent);

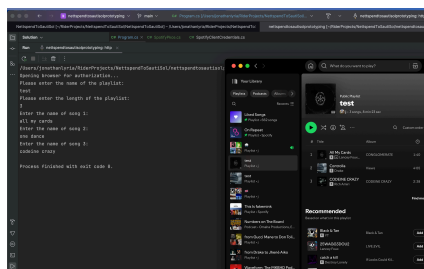
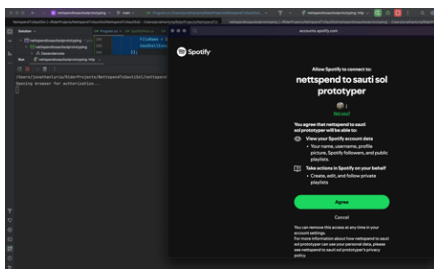
var tracks =
songDocument.RootElement.GetProperty("tracks").GetProperty("items").EnumerateArray().FirstOrDefault();
//it always returns an array so it forces to get only the first item
songIds[i] = tracks.GetProperty("id").GetString();
}
// prepare the POST body
var payload = new
{
    name = $"{playlistName}",
    @public = false
};
string jsonPayload = JsonSerializer.Serialize(payload);
StringContent createPlaylist = new StringContent(jsonPayload, System.Text.Encoding.UTF8,
"application/json");
HttpResponseMessage playlistResponseMessage =
client.PostAsync($"https://api.spotify.com/v1/users/{userId}/playlists", createPlaylist).Result;
//POST request creates the playlist in the user library

string jsonResponse = playlistResponseMessage.Content.ReadAsStringAsync().Result;
JsonDocument document = JsonDocument.Parse(jsonResponse);
string playlistId = document.RootElement.GetProperty("id").GetString();

var requestBody = new
{
    uris = songIds.Select(trackId => $"spotify:track:{trackId}") //spotify requires you to specify the
type of track it is
}; //prepares to batch the songids found in the search to add to the playlist
string jsonBody = JsonSerializer.Serialize(requestBody);
StringContent addSongsToPlaylist = new StringContent(jsonBody, Encoding.UTF8, "application/json");
HttpResponseMessage addSongsResponseMessage =
client.PostAsync($"https://api.spotify.com/v1/playlists/{playlistId}/tracks",
addSongsToPlaylist).Result; // posts the songs to the playlist
System.Diagnostics.Process.Start(new System.Diagnostics.ProcessStartInfo
{
    FileName = $"https://api.spotify.com/v1/playlists/{playlistId}/tracks",
    UseShellExecute = true,
}); //opens the playlist in window

```

Here is my test making a short 3 song playlist, which also verifies that the authorization works:



As can be seen by this test, there is one issue; the spotify v1 search point is not accurate when searching from a name. Therefore, I will make sure that when creating the playlist I only search by using the unique spotify Id. Luckily, this will be easy as I will be searching by artist on the v1 search endpoint, not by song name

Web Server Prototype

My final project will have a web server which can handle the majority of the heavy lifting in terms of logic so that 1) I can keep the PKCE token safe 2) The user does not have access to the database 3) The heavy lifting in terms of computation is done by the server.

Using the knowledge gained from using Spotify's API, I know that I will need to create a server which can handle both GET and POST requests. To accomplish this I will create a HTTP server using the Asp.Net Core and create communication using ApiController. I modified my test playlist code to separate it into a frontend which handles user input and a backend which handles the spotify API logic. I will create one GET and one POST request to familiarise myself..

Front-end code

```
private static readonly HttpClient Client = new HttpClient();

public static async Task Main(string[] args)
{
    SpotifyPkce authorizer = new SpotifyPkce();
    string pkceToken = authorizer.GetAuthoizationPKCEAccessToken(); //get pkce token

    Console.WriteLine("Please enter the name of the playlist: ");
    string playlistName = Console.ReadLine();

    Console.WriteLine("Please enter the number of songs in the playlist: ");
    int playlistLength = int.Parse(Console.ReadLine());

    string[] songs = new string[playlistLength];
    for (int i = 0; i < playlistLength; i++)
    {
        Console.WriteLine($"Enter the name of song {i + 1}: ");
        songs[i] = Console.ReadLine();
    }

    // Send song names to get song IDs
    string requestUrl = "http://localhost:5055/api/get-song-ids?" +
        string.Join("&", songs.Select(song => $"songs={Uri.EscapeDataString(song)}"));
    //adds the songs to the url

    HttpResponseMessage getSongsResponse = await Client.GetAsync(requestUrl);
    if (!getSongsResponse.IsSuccessStatusCode)
    {
        Console.WriteLine("Error retrieving song IDs.");
        return;
    }

    string responseContent = await getSongsResponse.Content.ReadAsStringAsync();
    JsonDocument document = JsonDocument.Parse(responseContent);

    List<string> songIds = document.RootElement.GetProperty("songIds").EnumerateArray()
        .Select(e => e.GetString()).ToList(); //convert the response to a list, turning it into a list

    Console.WriteLine($"Retrieved Song IDs: {string.Join(", ", songIds)}");

    // Send song IDs to create a playlist
    string playlistUrl = "http://localhost:5055/api/post-test-playlist";
    var playlistPayload = new { Name = playlistName, Uris = songIds, Token = pkceToken }; // create
    POST body
    string playlistJson = JsonSerializer.Serialize(playlistPayload); // serialise payload
    StringContent playlistContent = new StringContent(playlistJson, Encoding.UTF8,
    "application/json");

    HttpResponseMessage playlistResponse = await Client.PostAsync(playlistUrl, playlistContent);
    if (!playlistResponse.IsSuccessStatusCode)
    {
        Console.WriteLine("Error creating playlist.");
    }
}
```

```

        return;
    }

    string playlistResult = await playlistResponse.Content.ReadAsStringAsync();
    Console.WriteLine($"Playlist link: {playlistResult}");// get a link to the playlist

```

Back-end code

```

public class PlaylistInfoTransferObject() // data transfer object as post requests can only receive
one variable from the body
{
    public string Name { get; set; }
    public List<string> Uris { get; set; }
    public string Token { get; set; }
}

[ApiController]
[Route("api")]
public class Api : ControllerBase //inherits from ControllerBase which is part of
Microsoft.AspNetCore.Mvc
{
    private static readonly HttpClient Client = new HttpClient();

    [HttpGet("get-song-ids")]
    public IActionResult GetSongIds([FromQuery] List<string> songs)
    {
        SpotifyClientCredentials credentials = new SpotifyClientCredentials(); // uses client
credentials flow as it is unscoped data; will be explained shortly
        var token = credentials.GetAccessTokenAsync().Result.AccessToken;
        Client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

        List<string> songIds = new List<string>();
        foreach (string song in songs)
        {
            string url =
$"https://api.spotify.com/v1/search?q={Uri.EscapeDataString(song)}&type=track&limit=1";
            HttpResponseMessage songResponse = Client.GetAsync(url).Result;
            string songResponseContent = songResponse.Content.ReadAsStringAsync().Result;
            JsonDocument songDocument = JsonDocument.Parse(songResponseContent);

            var track =
songDocument.RootElement.GetProperty("tracks").GetProperty("items").EnumerateArray().FirstOrDefault();
            songIds.Add("spotify:track:" + track.GetProperty("id").GetString());
        }

        return Ok(new { songIds }); // returns the status code 200 that confirms everything went well
and the list of songIds
    }

    [HttpPost("post-test-playlist")]
    public IActionResult PostTestPlaylist([FromBody] PlaylistInfoTransferObject playlistInfo) // uses
the data transfer object
    {
        Client.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", playlistInfo.Token); //uses token as
authentication from user THIS IS NOT SAFE, WILL CHANGE FOR FINAL CODE
        HttpResponseMessage userIdResponse = Client.GetAsync("https://api.spotify.com/v1/me").Result;
        string stringUserIdResponse = userIdResponse.Content.ReadAsStringAsync().Result;
        JsonDocument userIdDocument = JsonDocument.Parse(stringUserIdResponse);
        string userId = userIdDocument.RootElement.GetProperty("id").GetString();

        var payload = new
        {
            name = playlistInfo.Name, // from data transfer object
            @public = false
        };
        string jsonPayload = JsonSerializer.Serialize(payload);
        StringContent createPlaylist = new StringContent(jsonPayload, System.Text.Encoding.UTF8,
"application/json");
    }
}

```

```

        HttpResponseMessage playlistResponseMessage =
Client.PostAsync($"https://api.spotify.com/v1/users/{userId}/playlists", createPlaylist).Result;
        JsonDocument document =
JsonDocument.Parse(playlistResponseMessage.Content.ReadAsStringAsync().Result);
        string? playlistId = document.RootElement.GetProperty("id").GetString();

        var requestBody = new
        {
            uris = playlistInfo.Uris,
        };
        string jsonBody = JsonSerializer.Serialize(requestBody);
        StringContent addSongsToPlaylist = new StringContent(jsonBody, Encoding.UTF8,
"application/json");
        HttpResponseMessage addSongsResponseMessage =
Client.PostAsync($"https://api.spotify.com/v1/playlists/{playlistId}/tracks",
addSongsToPlaylist).Result;
        Console.WriteLine(addSongsResponseMessage.Content.ReadAsStringAsync().Result);
        return Ok(new
        {
            playlistLink = $"https://open.spotify.com/playlist/{playlistId}", // returns the
playlistId
        });
    }
}

```

Test

Front-end input

```

opening browser for authorization...
Please enter the name of the playlist:
Test Server
Please enter the number of songs in the playlist:
3
Enter the name of song 1:
lettering
Enter the name of song 2:
mufasa
Enter the name of song 3:
easter pink
Retrieved Song IDs: spotify:track:4YfVot7p3LusP58JqM6w, spotify:track:3oNFJSqIQ8xfDYWB87W, spotify:track:4mwuLW8HATu9S8abilixV
Playlist link: {"playlistLink": "https://open.spotify.com/playlist/3vdsqje2xbXvAnoaWcSL"}
Process finished with exit code 0.

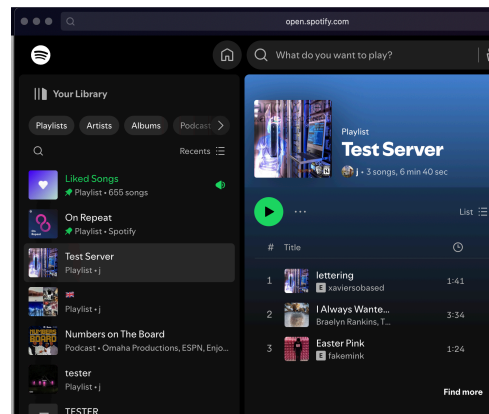
```

Backend Output

```

/Users/jonathanlyria/iderProjects/nettspendofautiso/nettspendofautisolprototyping/nettspendofautisolprototyping/bin/Debug/net8.0/nettspendofautisolprototyping
Info: Microsoft.Hosting.Lifetime[14]
      New listening on: http://localhost:5005
Info: Microsoft.Hosting.Lifetime[8]
      Application started. Press Ctrl+C to shut down.
Info: Microsoft.Hosting.Lifetime[8]
      Hosting environment: Development
Info: Microsoft.Hosting.Lifetime[8]
      Content root path: /Users/jonathanlyria/iderProjects/nettspendofautiso/nettspendofautisolprototyping/nettspendofautisolprototyping
Info: Microsoft.AspNetCore.Hosting.Diagnostics[1]
      Request starting HTTP/1.1 GET http://localhost:5005/api/get-song-ids/songs-lettering6songs-mufasa6songs-easter720pink - -
Info: Microsoft.AspNetCore.Routing.EndpointMiddleware[8]
      Executing endpoint 'nettspendofautisolprototyping.Api.GetSongIds (nettspendofautisolprototyping)'
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[100]
      Route matched with {action = "GetSongIds", controller = "Api"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult GetSongIds(System.Collections.Generic.IEnumerable<string>)
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[101]
      Executing action method nettspendofautisolprototyping.Api.GetSongIds (nettspendofautisolprototyping) - Validation state: Valid
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[102]
      Executed action method nettspendofautisolprototyping.Api.GetSongIds (nettspendofautisolprototyping), returned result Microsoft.AspNetCore.Mvc.ObjectResult in 1125.3ms
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor[1]
      Executing ObjectResult, writing value of type 'cor__AnonymousType0'[[{"System.Collections.Generic.List`1[System.String, System.Private.CoreLib, Version=8.0.0.0, Culture=neutral, PublicKeyToken=7cedd57796148a3d]"}]]
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[106]
      Executed action method nettspendofautisolprototyping.Api.GetSongIds (nettspendofautisolprototyping) in 1127.4222ms
Info: Microsoft.AspNetCore.Routing.EndpointMiddleware[1]
      Executed endpoint 'nettspendofautisolprototyping.Api.GetSongIds (nettspendofautisolprototyping)'
Info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
      Request finished HTTP/1.1 GET http://localhost:5005/api/get-song-ids/songs-lettering6songs-mufasa6songs-easter720pink - 200 - application/json; charset=utf-8 1183.272ms
Info: Microsoft.AspNetCore.Hosting.Diagnostics[1]
      Request starting HTTP/1.1 POST http://localhost:5005/api/post-test-playlist - application/json; charset=utf-8 421
Info: Microsoft.AspNetCore.Routing.EndpointMiddleware[8]
      Executing endpoint 'nettspendofautisolprototyping.Api.PostTestPlaylist (nettspendofautisolprototyping)'
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[100]
      Route matched with {action = "PostTestPlaylist", controller = "Api"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult PostTestPlaylist()
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[101]
      Executing action method nettspendofautisolprototyping.Api.PostTestPlaylist (nettspendofautisolprototyping) - Validation state: Valid
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[102]
      Executed action method nettspendofautisolprototyping.Api.PostTestPlaylist (nettspendofautisolprototyping), returned result Microsoft.AspNetCore.Mvc.ObjectResult in 1125.3ms
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor[1]
      Executing ObjectResult, writing value of type 'cor__AnonymousType0'[[{"System.String, System.Private.CoreLib, Version=8.0.0.0, Culture=neutral, PublicKeyToken=7cedd57796148a3d"}]]
Info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[106]
      Executed action method nettspendofautisolprototyping.Api.PostTestPlaylist (nettspendofautisolprototyping) in 697.5482ms

```



Network Expansion Prototype

Breadth-First Search

To expand my network, I will use a modified version of a breadth-first search. Breadth-first search is an algorithm for traversing or searching tree or graph structures by exploring all nodes at the present depth before moving on to the next level. It starts at a given root node, and visits all its neighbours before visiting their neighbours, and so on, until every reachable node has been explored. It uses a queue to keep track of nodes that need to be visited next.

To choose my starting artist, I consult with my client, who suggested Drake on the basis that he is very popular and will have similar artists in both rap and RnB, which will mean that the network will have a wide variety of artists to start with.

Getting similar artists

As decided before, I will use the last.fm API to find the list of artists which are similar. The key API call is “artist.getSimilar”. To make valid calls to last.fm I need to register for a last.fm app and use the apikey received from them in my requests.

Creating a network

For my final code, I will create a network with bespoke Artist and Connection objects, but for now I will use a 2D Dictionary in the form Dictionary<string, Dictionary<string, double>> to represent the network.

```
public class Expander
{
    private static readonly string ApiKey = "00751a650c0182344603b9252c66d416"; //last.fm api call
    public Queue<string> ArtistQueue = new Queue<string>(); // queue of artists for breadth-first
search

    private Dictionary<string, Dictionary<string, double>> Network =
        new Dictionary<string, Dictionary<string, double>>(); // 2D dictionary which represents the
network

    public void Expand(string startingArtist, int numIterations) //breadth-first search
    {
        Network.Add(startingArtist, new Dictionary<string, double>());
        ArtistQueue.Enqueue(startingArtist);
        for (int iteration = 0; iteration < numIterations && ArtistQueue.Count > 0; iteration++) //
finds the artists within numIterations degrees of seperataion of startingArtist
        {
            int queueLength = ArtistQueue.Count;
            for (int i = 0; i < queueLength; i++)
            {
                string currentArtist = ArtistQueue.Dequeue();
                var connections = GetSimilarArtists(currentArtist); //Returns a list of strings and
doubles

                foreach (var (name, match) in connections)
                {
                    ArtistQueue.Enqueue(name); //enqueues the connections
                }
            }
        }
    }
}
```

```

private List<(string, double)> GetSimilarArtists(string artist)
{
    List<(string, double)> similarArtistsList = new();
    using HttpClient client = new();
    string url = "http://ws.audioscrobbler.com/2.0/"; //lastfm base url
    var parameters = new Dictionary<string, string>
    {
        { "method", "artist.getSimilar" },
        { "artist", artist },
        { "api_key", ApiKey },
        { "format", "json" },
        { "limit", "5" }
    };

    HttpResponseMessage response = client.GetAsync(BuildUrlWithParams(url, parameters)).Result;
    if (!response.IsSuccessStatusCode)
    {
        Console.WriteLine($"Error: {response.StatusCode}");
        return similarArtistsList;
    }

    string jsonResponse = response.Content.ReadAsStringAsync().Result;
    JsonDocument document = JsonDocument.Parse(jsonResponse);

    if (!document.RootElement.TryGetProperty("similarartists", out JsonElement
similarArtistsElement) ||
        !similarArtistsElement.TryGetProperty("artist", out JsonElement artistArray)) // if the
resposne does not have // the
properties similarartists, or if the property similar artists does not have artists
    {
        return similarArtistsList;
    }

    foreach (var artistData in artistArray.EnumerateArray())
    {
        string name = artistData.GetProperty("name").GetString();
        double match = Convert.ToDouble(artistData.GetProperty("match").GetString());

        if (string.IsNullOrEmpty(name)) continue;

        bool isDuplicate = false;
        if (Network.TryGetValue(artist, out var connections))
        {
            if (connections.ContainsKey(name))
            {
                Console.WriteLine($"Skipping {name}, already exists in network.");
                isDuplicate = true;
            }
        }

        if (isDuplicate) continue;

        Console.WriteLine($"Adding {name}");

        similarArtistsList.Add((name, match));

        if (!Network.ContainsKey(name))
            Network[name] = new Dictionary<string, double>();

        Network[artist][name] = match;
        Network[name][artist] = match;
    }

    return similarArtistsList;
}

public void PrintNetwork()
{
    Console.WriteLine("Artist Network:");
    foreach (var artist in Network)
    {
        Console.WriteLine($"{artist.Key}:");
        foreach (var connection in artist.Value)

```

```

        {
            Console.WriteLine($" = {connection.Key} (Match: {connection.Value:F2})");
        }
    }
}

private static string BuildUrlWithParams(string url, Dictionary<string, string> parameters)
{
    return $"{url}?{string.Join("&", parameters.Select(p =>
    $"{p.Key}={Uri.EscapeDataString(p.Value)}"))}";
}
}

```

Test:

After running with Drake and 3 iterations, here is what was returned:

Drake: = DJ Khaled (Match: 1.00) = PARTYNEXTDOOR (Match: 0.97) = Drake & Future (Match: 0.88) = Bryson Tiller (Match: 0.77) = Tory Lanez (Match: 0.71) = Drake & 21 Savage (Match: 1.00) DJ Khaled: = Drake (Match: 1.00) = Meek Mill (Match: 0.99) = Young Money (Match: 0.80) = Rick Ross (Match: 0.72) = French Montana (Match: 0.69) PARTYNEXTDOOR: = Drake (Match: 0.97) = Bryson Tiller (Match: 1.00) = Sonder (Match: 0.59) = Jhené Aiko (Match: 0.55) = Brent Faiyaz (Match: 0.49) = Tory Lanez (Match: 0.84) Drake & Future: = Drake (Match: 0.88) = Drake & 21 Savage (Match: 0.74) = Future & Young Thug (Match: 0.68) = Future (Match: 0.59) = 21 Savage, Offset & Metro Boomin (Match: 0.58) Bryson Tiller: = Drake (Match: 0.77) = PARTYNEXTDOOR (Match: 1.00) = Brent Faiyaz (Match: 0.67) = Tory Lanez (Match: 0.58) = Summer Walker (Match: 0.52) Tory Lanez: = Drake (Match: 0.71) = Bryson Tiller (Match: 0.58) = PARTYNEXTDOOR (Match: 0.84) = Don Toliver (Match: 0.79) = A BOOGIE WIT DA HOODIE (Match: 0.76) Meek Mill: = DJ Khaled (Match: 0.99) = Rick Ross (Match: 1.00) = French Montana (Match: 0.60) = Dave East (Match: 0.60) = Lil Baby (Match: 0.59) Young Money: = DJ Khaled (Match: 0.80) = Nicki Minaj (Match: 1.00) = Yo Gotti (Match: 0.67) = London On Da Track (Match: 0.59) = Lil' Wayne (Match: 0.51) Rick Ross: = DJ Khaled (Match: 0.72) = Meek Mill (Match: 1.00) = Young Jeezy (Match: 1.00) = T.I. (Match: 0.66) = 2 Chainz (Match: 0.61) = Wale (Match: 0.58) = French Montana (Match: 0.94) French Montana: = DJ Khaled (Match: 0.69) = Meek Mill (Match: 0.60) = Rick Ross (Match: 0.94) = 2 Chainz (Match: 0.91) = Tyga (Match: 0.82) Sonder:	Jhené Aiko: = PARTYNEXTDOOR (Match: 0.55) = Summer Walker (Match: 1.00) = Mariah the Scientist (Match: 0.87) = TWENTY88 (Match: 0.63) = Tink (Match: 0.59) Brent Faiyaz: = PARTYNEXTDOOR (Match: 0.49) = Bryson Tiller (Match: 0.67) = Sonder (Match: 1.00) = AveNoir (Match: 0.39) = Paperboy Fabe (Match: 0.39) = Daniel Caesar (Match: 0.37) Drake & 21 Savage: = Drake & Future (Match: 0.74) = Drake (Match: 1.00) = 21 Savage & Metro Boomin (Match: 0.64) = 21 Savage, Offset & Metro Boomin (Match: 0.44) = Lil Baby & Gunna (Match: 0.41) = Brent Faiyaz (Match: 0.39) Chase Shakur: = Sonder (Match: 0.48) Paperboy Fabe: = Sonder (Match: 0.36) = Brent Faiyaz (Match: 0.39) Mariah the Scientist: = Jhené Aiko (Match: 0.87) = Summer Walker (Match: 0.98) TWENTY88: = Jhené Aiko (Match: 0.63) Tink: = Jhené Aiko (Match: 0.59) = Summer Walker (Match: 0.64) Daniel Caesar: = Brent Faiyaz (Match: 0.37) 21 Savage & Metro Boomin: = Drake & 21 Savage (Match: 0.64) = 21 Savage, Offset & Metro Boomin (Match: 0.69) Lil Baby & Gunna: = Drake & 21 Savage (Match: 0.41) Young Thug: = Future & Young Thug (Match: 1.00) = Future (Match: 1.00) Young Stoner Life, Young Thug & Gunna: = Future & Young Thug (Match: 0.62) Lil Keed: = Future & Young Thug (Match: 0.58) Future & Lil Uzi Vert: = Future & Young Thug (Match: 0.55) Gunna: = Future (Match: 0.67) = Don Toliver (Match: 0.60) Dj Esco: = Future (Match: 0.64) 21 Savage: = Future (Match: 0.52) Offset & Metro Boomin: = Future & Young Thug (Match: 0.68) = Young Thug (Match: 1.00) = Future (Match: 0.70) = Young Stoner Life, Young Thug & Gunna (Match: 0.62) Lil Keed (Match: 0.58)	Future: = Drake & Future (Match: 0.59) = Future & Young Thug (Match: 0.70) = Young Thug (Match: 1.00) = Gunna (Match: 0.67) = Dj Esco (Match: 0.64) = Lil Baby (Match: 0.59) = 21 Savage (Match: 0.52) 21 Savage, Offset & Metro Boomin: = Drake & Future (Match: 0.58) = Drake & 21 Savage (Match: 0.44) = Offset & Metro Boomin (Match: 1.00) = 21 Savage & Metro Boomin (Match: 0.69) = Future & Young Thug (Match: 0.31) Summer Walker: = Bryson Tiller (Match: 0.52) = Jhené Aiko (Match: 1.00) = Mariah the Scientist (Match: 0.98) = Kehlani (Match: 0.72) = Ari Lennox (Match: 0.66) = Tink (Match: 0.64) Don Toliver: = Tory Lanez (Match: 0.79) = Travi\$ Scott (Match: 1.00) = Gunna (Match: 0.60) = Offset (Match: 0.48) = Lil Tecca (Match: 0.43) A BOOGIE WIT DA HOODIE: = Tory Lanez (Match: 0.76) Dave East: = Meek Mill (Match: 0.60) Lil Baby: = Meek Mill (Match: 0.59) = Future (Match: 0.59) Nicki Minaj: = Young Money (Match: 1.00) Yo Gotti: = Young Money (Match: 0.67) London On Da Track: = Young Money (Match: 0.59) Lil' Wayne: = Young Money (Match: 0.51) Young Jeezy: = Rick Ross (Match: 1.00) T.I.: = Rick Ross (Match: 0.66) 2 Chainz: = Rick Ross (Match: 0.61) = French Montana (Match: 0.91) Wale: = Rick Ross (Match: 0.58) Tyga: = French Montana (Match: 0.82) AveNoir: = Sonder (Match: 0.64) = 21 Savage, Offset & Metro Boomin (Match: 1.00) Kehlani: = Summer Walker (Match: 0.72) Ari Lennox: = Summer Walker (Match: 0.66) Travi\$ Scott: = Don Toliver (Match: 1.00) Offset: = Don Toliver (Match: 0.48) Lil Tecca:
--	---	---

= PARTYNEXTDOOR (Match: 0.59) = Brent Faiyaz (Match: 1.00) = AveNoir (Match: 0.64) = Chase Shakur (Match: 0.48) = Paperboy Fabe (Match: 0.36)	= Future & Lil Uzi Vert (Match: 0.55) = 21 Savage, Offset & Metro Boomin (Match: 0.31)	= Don Toliver (Match: 0.43)
---	---	-----------------------------

Deciding on minimum popularity (3rd Client Interview)

The Spotify API does not have an option to return the monthly listeners, rather giving back an arbitrary “Popularity” value on a scale of 1-100. I wrote this program to give to my client, so that they can decide on the values of popularity which will be the cutoff for the popularity. It returns key information about the artist, including popularity and the date of the last popular track

```
namespace nettspendtosautisolprototyping;
public class Program
{
    public static void Main(string[] args)
    {
        SpotifyClientCredentials authorizer = new SpotifyClientCredentials();
        var accessToken = authorizer.GetAccessTokenAsync().Result.AccessToken;
        while (true)
        {
            string artistName = Console.ReadLine();
            GetSpotifyDetails testArtist = new GetSpotifyDetails(artistName, accessToken);
            Console.WriteLine($"id: {testArtist.SpotifyId} popularity: {testArtist.Popularity}, last
popular track: {testArtist.LatestTopTrackReleaseDate}");
        }
    }
}

public class GetSpotifyDetails
{
    public string? SpotifyId { get; private set; }
    public int? Popularity { get; private set; }
    private static readonly HttpClient HttpClient = new HttpClient();
    public string? LatestTopTrackReleaseDate { get; private set; }
    private readonly string _accessToken;
    public GetSpotifyDetails(string artistName, string accessToken)
    {
        _accessToken = accessToken;
        GetArtistDetails(artistName);
        GetLatestTopTrackReleaseDate();
    }
    private void GetArtistDetails(string artistName)
    {
        HttpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
        _accessToken);
        string url = $"https://api.spotify.com/v1/search?q=artist:\"{artistName}\"
&type=artist&limit=1";
        HttpResponseMessage response = HttpClient.GetAsync(url).Result;
        if (response.IsSuccessStatusCode)
        {
            JsonDocument document = JsonDocument.Parse(response.Content.ReadAsStringAsync().Result);
            JsonElement artistData =
            document.RootElement.GetProperty("artists").GetProperty("items").EnumerateArray().FirstOrDefault();
            if (!artistData.Equals(default(JsonElement)))
            {
                SpotifyId = artistData.GetProperty("id").GetString();
                Popularity = artistData.GetProperty("popularity").GetInt32();
            }
        }
    }
}
```

```
private void GetLatestTopTrackReleaseDate()
{
    if (SpotifyId == null) return;
    HttpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
_accessToken);
    string url = $"https://api.spotify.com/v1/artists/{SpotifyId}/top-tracks?market=US";
    HttpResponseMessage response = HttpClient.GetAsync(url).Result;
    if (response.IsSuccessStatusCode)
    {
        JsonDocument document = JsonDocument.Parse(response.Content.ReadAsStringAsync().Result);
        var tracks = document.RootElement.GetProperty("tracks").EnumerateArray().Take(5);

        LatestTopTrackReleaseDate = tracks
            .Select(track => track.GetProperty("album").GetProperty("release_date").GetString())
            .OrderByDescending(date => date)
            .FirstOrDefault();
    }
}
}
```

My clients response was this:

- Artists that have there last top track being less than 1 year old, the minimum cutoff should have a popularity of 50
- Artists that have there last top track with an age 1 and 5 years old, the minimum cutoff should be a popularity 55
- Artists that have there latest top track with age between 5 and 10 years old, the minimum cutoff should be a popularity of 60
- Artists that have there last latest top track with an age between 5 and 10 years old, the minimum cutoff should be a popularity of 65

Program requirements

1. The program must have a way to expand the artists from a starting artist and store it in a persistent manner
 - 1.1. The program must have a way to use external data to get a list of similar artists for an artist
 - 1.1.1. Program must use last.fm data to get a list of verified similar artists and their similarity score to the given artist
 - 1.1.2. Program must use last.fm data to get the top tracks of a given artist
 - 1.1.3. Program must use spotify data to get the spotifyid of a given artist
 - 1.1.4. Program must use spotify data to get the top tracks of a given artist
 - 1.1.5. Program must have a way to compare the top tracks from last.fm artist and spotify artist and reject any artist who doesn't have a single match between the two
 - 1.2. Program must reject any artist with a popularity score too low for their age, as defined:
 - 1.2.1. Program must reject any artist whose last popular track was less than a year ago + has a popularity score of <50
 - 1.2.2. Program must reject any artist whose last popular track was between a year and 5 years ago and has a popularity score of <55
 - 1.2.3. Program must reject any artist whose last popular track was between 5 years and 10 years ago and has a popularity score of <60
 - 1.2.4. Program must reject any artist whose last popular track was more than 10 years ago and has a popualrity score of <65

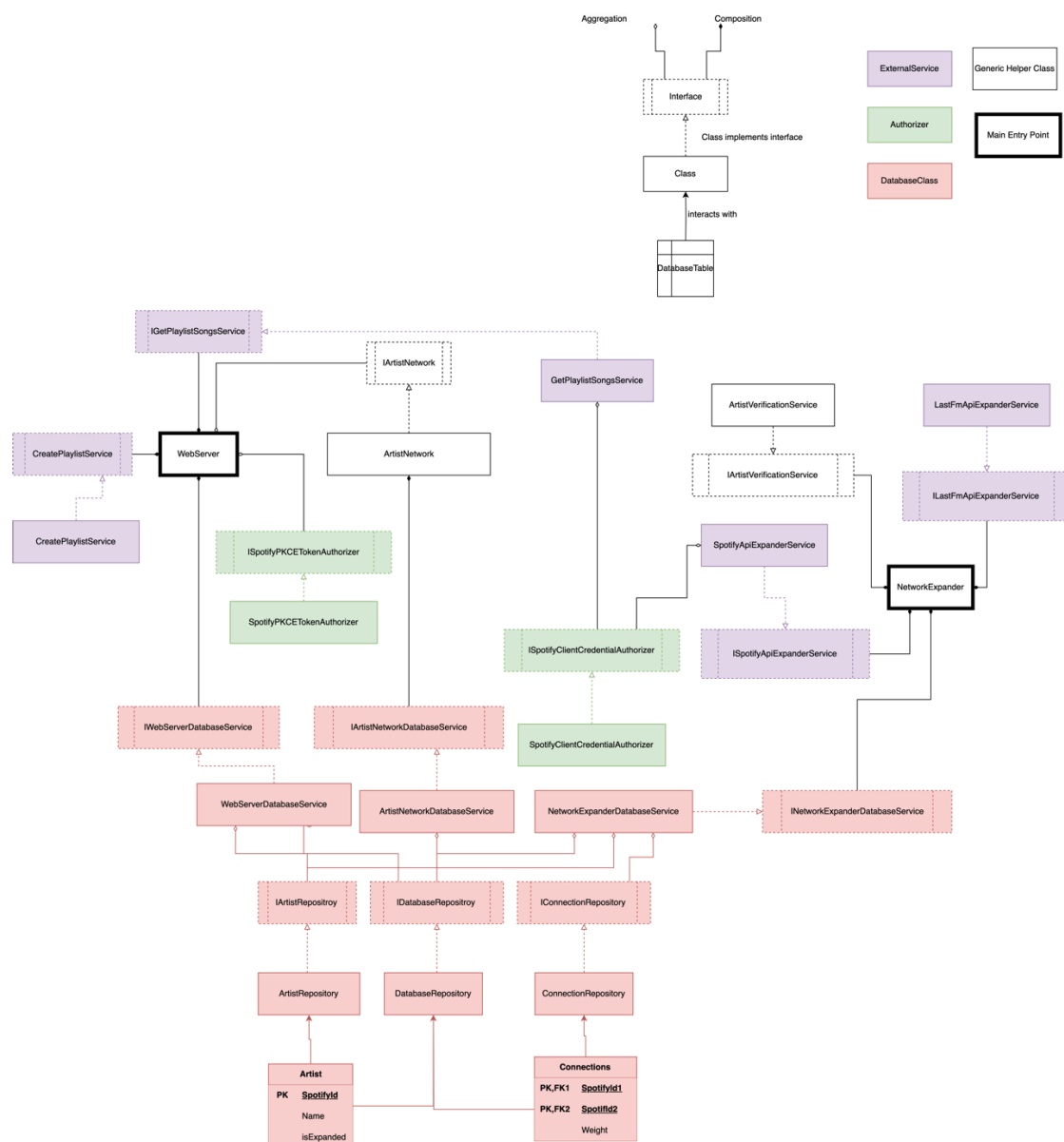
- 1.3. Program must have a database to store found artists
 - 1.3.1. Database must be able to store the spotifyId, name and whether an artist has had their similar artists found
 - 1.3.2. Database must be able to store the two spotifyIds and strength of connection between two artists
- 1.4. Program must use a breadth-first search to expand the network
 - 1.4.1. Program must check whether there are any artists in the database who have not been searched for, and create a queue if there are
 - 1.4.2. If no artists exist in database, at programmers discretion, choose an artist to enqueue to the searchQueue
 - 1.4.3. For each artist in the queue program must find the similar verified (as per requirements earlier) artists and the corresponding strength of connection to that artist
 - 1.4.4. Program must add these artists to the queue and add the connections between the starting artist and the found artist to the database, if it does not already exist
2. Program must have away to find the shortest path between any two artists
 - 2.1. Program must have a way to represent the artists and connections in the database
 - 2.1.1. Program must load artists and connections from database
 - 2.1.2. Program must represent the network as a 2D Dictionary of structure Dictionary(Artist, Dictionary(Artist, Weight))
 - 2.1.3. Artist object must have has which allows $O(1)$ look up time
 - 2.2. Program must have a method which finds the shortest path between any two artists
 - 2.2.1. Program must take two artists as input
 - 2.2.2. Program must use Dijkstras algorithm to find the shortest path between the two artists
 - 2.2.3. Program must output the list of artists on the path between the two artists
3. Program must have a way to find songs for each artist in the path of artists a playlist in the user library
 - 3.1. Program must try to find features for each artist who are next to each other
 - 3.1.1. Program must use the spotify data to search for if two artists have a song
 - 3.1.2. Program must reject any song which has more than the two artists
 - 3.2. Program must find 3 songs for each artist
 - 3.2.1. Program must use the spotify data to find the top 10 popular songs for an artist
 - 3.2.2. Songs must not have any other artists on it
 - 3.2.3. For each feature that has been found for an adjacent artist, take one away from the total number of songs to be found
 - 3.2.4. Program must choose at random the songs, dependent on the number of songs required
4. Program must have a way to create a playlist in the user library
 - 4.1. Program must handle user authentication to handle scoped authentication
 - 4.1.1. Program must use user authentication with PKCE
 - 4.1.2. Program must never save the PKCE Token for longer than necessary
 - 4.2. Program must create a playlist in the user library
 - 4.2.1. Playlist must have name "from ___ to ___" where the gaps are the first and last artists the user entered respectively
 - 4.2.2. Playlist must be added automatically to user library with the songs found for each artist added in order

5. Program must take the user through the process from authentication to playlist creation
 - 5.1. Program must prompt the user to enter the first artist
 - 5.1.1. The website prompts the user to enter the first artist.
 - 5.1.2. It sends a query to the server to verify that the entered artist exists in the database.
 - 5.1.3. User is prompted repeatedly until a valid artist is entered.
 - 5.1.4. The process is then repeated for the second artist.
 - 5.2. Program must display the path between two artist
 - 5.2.1. Program finds shortest path between two artists
 - 5.2.2. Program displays artists on the shortest path between two artists with a ->
 - 5.3. Program must handle user input for authentication
 - 5.3.1. The website must display a prompt to press enter to login to Spotify.
 - 5.3.2. A new window is opened for the user to login to Spotify.
 - 5.3.3. Once the user logs in, the window closes.
 - 5.4. Program must handle finding the playlist songs and creating the playlist
 - 5.4.1. The website prompts the user to press enter to create the playlist.
 - 5.4.2. Once the playlist is created, a new window opens to display the playlist.
 - 5.4.3. The playlist link is also displayed for the user to copy.
6. Website design must follow client and shareholder requirements
 - 6.1. Design must follow Spotify Colour Scheme
 - 6.1.1. Design must have main text colour white
 - 6.1.2. Design must have accent colour "Spotify Green"
 - 6.2. Design must have main spotify font
 - 6.2.1. Main font must be Gotham medium
 - 6.2.2. Title font must be Gotham bold
 - 6.3. Website must have navigation links
 - 6.3.1. Report issue link must open report issue modal
 - 6.3.2. View all artists must open view all artists page
 - 6.3.3. Information button must open link to .md file with information about project
 - 6.3.4. Github logo must open link to github repository
 - 6.4. Website must have a Dynamic Title
 - 6.4.1. Website title starts with "from"
 - 6.4.2. Website title changes to from (first artist) to once a (first artist) has been successfully entered
 - 6.4.3. Website title change to from (first artist) to (second artist) once second artist has successfully been entered
 - 6.5. Website must have a modal which the user can interact with to report issues
 - 6.5.1. Modal must have a text box which prompts user to enter their issue
 - 6.5.2. Issue must be stored in a text file for the programmer to see
 - 6.6. Design a page in the website where users can view all artists and connections between artists in the network
 - 6.6.1. Data is displayed in a table format with the columns: Artist Name, Similar Artists, and View Artist On Spotify.
 - 6.6.2. All artists in the database are included.

Documented Design

High-level overview

Class Diagram



Hierarchy + Dependencies

Here is how the project is organised. The arrow denote dependencies between csharp projects



Explanation of classes

Here is an explanation of my classes. Throughout my documented design I will explain each in more detail as well as fully explaining how each class actually interacts with each other.

Class Name (Interface)	Key Function	Interactions
ArtistRepository (IArtistRespostory)	Contains all of the functions which interact exclusively with the Artist table in the database. This includes searching for artists in the database, checking whether an artist is in the database or adding an artist to the database.	At run time it is injected using constructor injection into the WebServer and NetworkExpander DatabaseServices

DatabaseRepository (IDatabaseRepository)	Contains the functions which require both the Artist table and the Connections table to execute: Creating the tables in the database and using an SQL Join statement	Gets injected using constructor injection into the 3 of the DatabaseServices
ConnectionRepository (IConnectionRepository)	Contains the functions which interact exclusively with the Connection table: Checking whether two artists have a connection, and adding a connection to the database	Gets injected into NetworkExpanderDatabase Service and is injected using constructor injection at runtime
WebServerDatabaseService (IWebServerDatabaseService)	Contains the database operations which the webserver needs for operation, does not actually deal with any SQL but uses the repositories to pass the relevant functions for the webserver to use	Aggregates interfaces for Artist and DatabaseRepository and through its interface gets injected via dependency injection into the WebServer
NetworkExpanderDatabaseService (INetworkExpanderDatabaseService)	Contains the database operations which the NetworkExpander needs for operation, does not actually deal with any SQL but uses the repositories to pass the relevant functions for the webserver to use.	Aggregates interfaces for all 3 repositories and gets injected via constructor injection into the NetworkExpander
ArtistNetworkDatabaseService (IArtistNetworkDatabaseService)	Contains the operations which the network needs to load, does not actually deal with any SQL but uses the repositories to pass the relevant functions for the webserver to use,	Aggregates interface for DatabaseRepository and gets injected into ArtistNetwork through constructor injection

SpotifyClientCredentialAuthorizer (ISpotifyClientCredentialAuthorizer)	Handles getting the accessToken for the client-credential flow: when nonscoped data (such as artist information, track information etc) from spotify is required	At run time it is injected into SpotifyApiExpanderService and GetPlaylistSongsService
SpotifyPKCETokenAuthorizer (ISpotifyPKCETokenAuthorizer)	Handles getting the access Token for the authorization via pkce extention flow; where scoped data needs to be changed/accessed (when editing user playlists).	At runtime it is injected into the WebServer
SpotifyApiExpanderService (ISpotifyApiExpanderService)	Handles retrieving and parsing data from spotify which is required for the network expander. This includes getting the spotifyId, name and popularity for an artists from a given name, and also getting the top 10 tracks for an artist given their spotifyId	Aggregates interface for SpotifyClientCredential Authorizer and at run time it gets injected into the NetworkExpander via constructor injection
LastFmApiExpanderService (ILastFmApiExpanderService)	Handles retrieving and parsing data from last.fm which is required for the networkexpander. This includes getting the top 5 most similar artists to an artist given an artist name, as well as getting the top 10 songs for an artist	Gets injected into the NetworkExpanded at runtime via constructor injection
GetPlaylistSongsService (IGetPlaylistSongsService)	Handles retrieving the songIds for the playlist which is to be created. Has a function which given a list of artistIds, finds 3 songs per artist to be added to the playlist, also looking for	Aggregates interface for SpotifyClientCredential Authorizer and at runtime it gets injected into the webserver via dependency injection

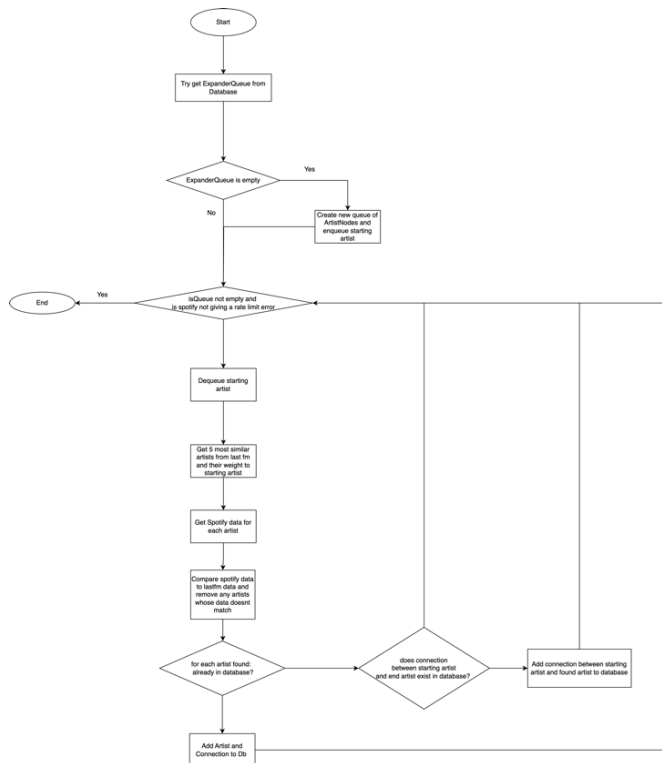
	features between artists.	
CreatePlaylistService (ICreatePlaylistService)	Handles creating and adding the playlist to the user library. Has a function which, given a list of songIds, adds the songs to the user playlist	It gets injected into the webserver via dependency injection at runtime
ArtistVerificationService (IArtistVerificationService)	Handles crosschecking spotify and last.fm data to make sure that they are referring to the same artist. Compares top tracks of artists to see if there is overlap in the lists, and also checks the popularity of the artist compared to the date of their most recent popular song to see whether they meet the minimum popularity requirement	Gets injected into the ArtistNetwork via constructor injection at runtime
ArtistNetwork (IArtistNetwork)	Represents a network of artists at the runtime for the webserver as a 2D Dictionary of ArtistNodes and Dictionary of ArtistNodes and	Aggregates ArtistNetworkDatabase Service and injected into the web server via dependency injection at runtime
ArtistNode	A global type, which represents a verified artist. Has fields SpotifyId and Name and overrides the Hash and equals function so it can work in the dictionary	Gets used as a data type by all of the classes to simplify data transfer of artists with a spotifyId and a name
NetworkExpander	Handles the logic of expanding the network. Checks whether the database is empty, and if not it loads a queue of unexpanded artists and performs a	Aggregates interfaces of NetworkExpanderDatabase Service, SpotifyApiExpanderService, LastFmApiExpanderService and ArtistVerificationService.

	breadth first search, using various services to get a dictionary of artists who have not been expanded yet.	
Api / WebServer	Creates an api which an website can interact with. Has endpoints for verifying whether a user entered artist is in the database, returning a path of artists in the database, creating a playlist from a path of artists and viewing all the artists in the database	Aggregates interfaces of ArtistNetwork, WebServer DatabaseService, SpotifyPKCETokenAuthorizer, GetPlaylistSongsService and CreatePlaylistService

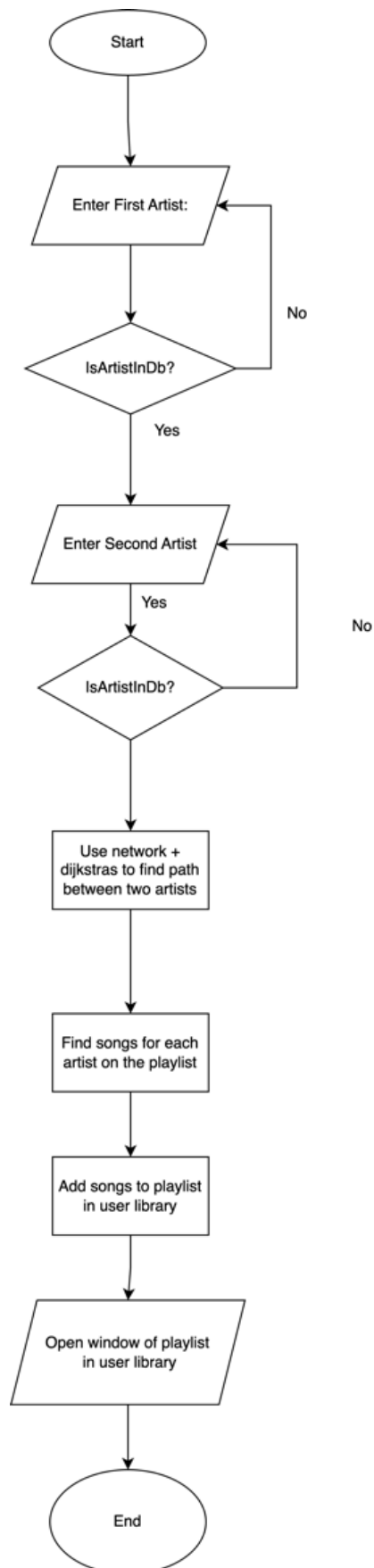
Program flowcharts

To tie everything together, here are two flowcharts of the two main processes. I will go into more detail later but just for a general idea

Expanding the network flowchart:



User flow flowchart



Making a database

Using microsoft sqlite

To create my database, I will use Microsoft SQLite to connect my database to my code. This lightweight database engine is well-suited for local storage and does not require a separate server to run. I can install the necessary packages using NuGet, and once installed, I can add the relevant directives to my code.

To connect my code with the sqlite database, I need to define a `ConnectionString` with the structure: `"Data Source=(projectPath/ArtistNetworkDatabase.db)"`. To execute a command, I use a series of using commands, first starting off with creating a new `SqlConnection` using: `SqlConnection(ConnectionString)` to establish a connection with the database. Then, I can define the query depending on what is required. I can then define an `SqlCommand` with `SqlCommand(query, SqlConnection)`. If the query has a parameter which requires input, such as returning an `spotifyId` given an `artistName`, I can use the `@(variablename)` and replace that with the parameter with `SqlCommand.Parameters.AddWithValue("@(variableName)", variableName)`. This is what happens next depending on the query:

SELECT statement

Define a `SqlDataReader` using `cmd.ExecuteReader`

Returning a boolean (for example does an artist exist in the database)?

- use `if (reader.Read)`, return true else return false

Returning a value (a list of names, a single name)

- use `while (reader.Read)`, based on order of fields specified in the query, use `reader.GetType(0/1/2/3)`

UPDATE/INSERT query

- Execute the defined `SqlCommand` with `cmd.ExecuteNonQuery()`

Designing the database

My database needs to be able to store the following information:

- `SpotifyId`, names and whether an artist has had their similar artists found of the artist
- How strong the weight of connection is between artists

I want my design to be 3NF to eliminate the possibility of data redundancy, which is when data is repeated unnecessarily. The key things I need to make sure is that each column only contains a single values and that there are no partial or transitive dependencies.

Here is an SQL data dictionary of my final database design:

Table Name	Artist
Primary Key	SpotifyId
Foreign Keys	N/A

Data Item	Data Type	Validation	Sample Value
SpotifyId	TEXT	NOT NULL	3TVXtAsR1Inumwj472S9r4
ArtistName	TEXT	NOT NULL	Drake
isExpanded	BIT	NOT NULL	1

Table Name		Connections	
Primary Key		Composite Key: SpotifyId1, SpotifyId2	
Foreign Keys		SpotifyId1 References: Artists:SpotifyId SpotifyId2 References: Artists:SpotifyId	
Data Item	Data Type	Validation	Sample Value
SpotifyId1	TEXT	NOT NULL	3TVXtAsR1Inumwj472S9r4
SpotifyId2	TEXT	NOT NULL	1RyvvyTE3xzB2ZywiAwp0i
Weight	REAL	NOT NULL	0.971263

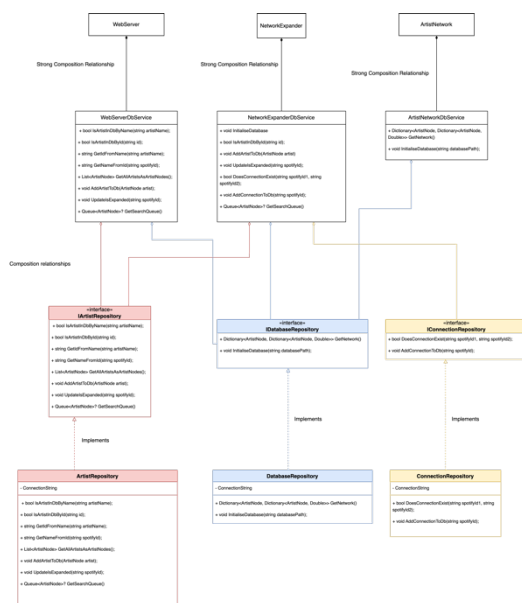
Partial dependencies occur when the primary key is a composite key and a non-key attribute only depends on part of the primary key. The only non-key attribute in the connections table is weight, and this is fully dependent on both the spotifyId1 and spotifyId2. Transitive dependencies occur when a non-key attribute depends on another non-key attribute. As both ArtistName and isExpanded depend directly on SpotifyId, there are no transitive dependencies

Diagram of database classes

Here is a diagram of how it will be organised:

The only parts of the code which interact with the database and have actual SQL is the ArtistRepository, DatabaseRepository and ConnectionRepository.

I need to make sure that my design is modular, which means that networkexpander, webserver and artistnetwork cannot be incharge of creating the databaseservice objects. This is the order that I will follow:



- Create the relevant repository objects with the argument of the databasepath being passed via the configurator arguments
- Inject these relevant repository objects into the database service for the program
- Inject the relevant database service into the relevant object

DatabaseRepository controls when entire database to perform an action. The two main actions which this is required for is when the database is first checked to see whether it exist and has been

initialised, and for when the network needs to be loaded from the database using an SQL Join statement

ArtistRepository controls SQL actions on the Artist table. This includes checking whether an artist is in the database, getting all the artists in the database, getting information about a specific artist and adding an artist to the database

ConnectionRepository controls SQL actions on the Connection table. The two main things is checking whether there already is a connection between artists, and adding a connection to the database

Each service ensures that it only performs necessary actions. For example, the WebServer does not need to be able to add artists to the database, so it is not included in the WebServerDbService

At the initialisation of the all the repositories, the database path where the database is stored is given and a connectionString with structure "Data Source=databasePath" is created

Database Repository SQL statements

Here are all the functions in my database repository.

InitialiseDatabase

This subroutine checks whether a database exists at the given path, if not, it creates a database file and adds the relevant tables to it

```
SUBROUTINE InitialiseDatabase(string databasePath){
    if (database does not exist at databasePath)
        Create database.db file at databasePath

    Create SqliteConnection from connectionString as Connection
    ArtistQuery = "CREATE TABLE IF NOT EXISTS Artist (
        SpotifyId TEXT NOT NULL PRIMARY KEY,
        ArtistName TEXT NOT NULL,
        isExpanded BIT NOT NULL );"
    Create SqliteCommand from Connection and ArtistQuery as artistCommand
    Execute artistCommand
    ConnectionQuery = "CREATE TABLE IF NOT EXISTS Connections (
        SpotifyId1 TEXT NOT NULL REFERENCES Artist(SpotifyId),
        SpotifyId2 TEXT NOT NULL REFERENCES Artist(SpotifyId),
        Weight REAL NOT NULL
    ); "
    Create SqliteCommand from Connection and ConnectionQuery as connectionCommand
    Execute connectionCommand
}
```

GetNetwork

This subroutine uses an SQL JOIN statement to create the network dictionary object from the database

```
FUNCTION Dictionary<ArtistNode, Dictionary<ArtistNode, double>> GetNetwork{
    ArtistNetwork = Initialise new Dictionary<ArtistNode, Dictionary<ArtistNode, double>>
    Create SqliteConnection from ConnectionString as Connection
    query = "SELECT a1.ArtistName, c.SpotifyId1, a2.ArtistName, c.SpotifyId2, c.Weight
    FROM Connections c"
```

```

        JOIN Artist a1 ON c.SpotifyId1 = a1.SpotifyId
        JOIN Artist a2 ON c.SpotifyId2 = a2.SpotifyId"
    Create SqliteCommand from Connection and query as Command
    Create SqliteDataReader from Command as reader
    while (reader reads){
        artistName1 = get first item of reader as string
        spotifyId1 = get second item of reader as string
        artistNode1 = new artistNode(artistName1, spotifyId1)
        artistName2 = get third item of reader as string
        spotifyId2 = get fourth item of reader as string
        artistNode2 = new artistNode(artistName2, spotifyId2)
        weight = get fifth item of reader as string
        if the artistnetwork does not contain key ArtistNode1 {
            Add Key ArtistNode1 and empty Dictionary<AritstNode, double>
        }
        if the artistnetwork does not contain key ArtistNode2 {
            Add Key ArtistNode2 and empty Dictionary<AritstNode, double>}
        ArtistNetwork[ArtistNode1][ArtistNode2] = weight
        ArtistNetwork[ArtistNode2][ArtistNode1] = weight
    }
}

```

Artist Repository SQL statements

Here are all the functions in my artist repository.

isArtistInDbByName

This method returns a boolean true or false depending on whether the given artistName is in the database. It turns it into lower because this is called for user input

```

FUNCTION IsArtistInDbByName(artistName){
    Create SqliteConnection from connectionString as Connection
    Query = "SELECT ArtistName FROM Artist WHERE LOWER(ArtistName) = LOWER(@artistName)"
    Create SqliteComand from Query and Connection as Command
    Replace @artistName with artistName in Command
    Reader = Command.ExecuteReader()
    If reader reads{
        Return true
    }
    Else{
        Return false
    }
}

```

isArtistInDbById

This method returns a boolean true or false depending on whether the given spotifyId is in the database

```

FUNCTION IsArtistInDbByName(artistName){
    Create SqliteConnection from connectionString as Connection
    Query = "SELECT SpotifyId FROM Artist WHERE SpotifyId = @spotifyId"
    Create SqliteComand from Query and Connection as Command
    Replace @artistName with artistName in Command
    Reader = Command.ExecuteReader()
    If reader reads{
        Return true
    }
    Else{
        Return false
    }
}

```

GetNameFromId

This method returns a string which is the ArtistName associated with the given spotifyId

```
FUNCTION string GetNameFromId(spotifyId){
    Create SqliteConnection from connectionString as Connection
    Name = ""
    Query = "SELECT ArtistName FROM Artist WHERE SpotifyId = @spotifyId"
    Create SqliteCommand from Query and Connection as Command
    Replace @spotifyId with spotifyid in Command
    Reader = Command.ExecuteReader()
    while reader reads{
        name = reader.GetString(0);
    }
    return Name
}
```

GetIdFromName

This method returns a string which is the spotifyId associated with a given name

```
FUNCTION string GetIdFromName(artistName){
    Create SqliteConnection from connectionString as Connection
    SpotifyId = ""
    Query = "SELECT SpotifyId FROM Artist WHERE LOWER(ArtistName) = LOWER(@artistName)"
    Create SqliteCommand from Query and Connection as Command
    Replace @artistName with artistName in Command
    Reader = Command.ExecuteReader()
    while reader reads{
        spotifyId = reader.GetString(0);
    }
    returns SpotifyId
}
```

GetSearchQueue

This method returns a queue of ArtistNodes where the expanded = 0

```
FUNCTION Queue<ArtistNode> GetSearchQueue(){
    Create SqliteConnection from connectionString as Connection
    searchQueue = Initialise new queue
    Query = "SELECT ArtistName, SpotifyID FROM Artist WHERE isExpanded = 0"
    Create SqliteCommand from Query and Connection as Command
    Reader = Command.ExecuteReader
    While (reader.read)
    {
        artistName = reader.GetString(0)
        spotifyId = reader.GetString(1)
        artist = new ArtistNode(artistName, spotifyId)
        enqueue artist to searchqueue
    }

    Return searchQueue
}
```

UpdateIsExpanded

This method updates the expanded field of a given spotifyId to 1

```
SUBROUTINE UpdateIsExpanded(string spotifyId){
    Create SqliteConnection from connectionString as Connection
    Query = "UPDATE Artist SET isExpanded = 1 WHERE SpotifyId = @spotifyId"
    Create SqliteCommand from Query and Connection as Command
    Replace @spotifyId with spotifyId in command
    Execute Command
}
```

AddArtistToDb

This method adds an artist to the database

```
SUBROUTINE AddArtistToDb(ArtistNode artist){
    Create SqliteConnection from connectionString as Connection
    Query = "INSERT INTO Artist (ArtistName, isExpanded, SpotifyID) VALUES (@artistName, 0, @spotifyId)".
    Create SqliteCommand from Query and Connection as Command
    Replace @artistName with artist.Name in command
    Replace @spotifyid with artist.SpotifyId in command
    Execute Command
}
```

Connections Repository SQL statements

Here are all the functions in the connection repository

IsConnectionInDb

This method checks whether two artists have a connection between each other

```
FUNCTION IsConnectionInDb(string spotifyid1, string spotifyId2){
    Create SqliteConnection from connectionString as Connection
    Query = "SELECT * FROM Connections
    WHERE (SpotifyId1 = @spotifyId1 AND SpotifyId2 = @spotifyId2)
    OR (SpotifyId1 = @spotifyId2 AND SpotifyId2 = @spotifyId1);"
    Create SqliteCommand from Query and Connection
    Replace @spotifyId1 with spotifyId1 in command
    Replace @spotifyId2 with spotifyId2 in command
    Reader = Command.ExecuteReader()
    If reader reads{
        return true;
    }
    Else{
        return false;
    }
}
```

AddConnectionToDb

This method adds a connection between two artists

```
FUNCTION AddConnectionToDb(string spotifyId1, string spotifyId2, double weight){
    Create SqliteConnection from connectionString as Connection
```

```

        query = "INSERT INTO Connections (SpotifyId1, SpotifyId2, Weight)
VALUES (@spotifyId1, @spotifyId2, @strength);"
        Create SqliteCommand from Query and Connection as Command
        Replace @spotifyId1 with spotifyId1
        Replace @spotifyId2 with spotifyId2
        Replace @weight with weight
        Execute Command
    }

```

Representing Network at runtime

Overview

When the server is running, I will load a network from a database. To handle network traversal logic, I designed a class which will represent a network: ArtistNetwork. In essence, ArtistNode represents an artist in the network, and the ArtistNetwork consists of a 2D Dictionary; where the Key is an ArtistNode, and the Value is a dictionary of ArtistNodes and doubles, which represent the weight. The ArtistNetwork also includes the key method FindPathWithDijkstras, which handles the path-finding logic

ArtistNode

I designed the class ArtistNode to facilitate moving all the key data about the artist in one object. It consists of the fields ArtistName and SpotifyId. As I am using the ArtistNode as the key in my dictionary, I need to override the Equals and GetHashCode methods in the class to use the SpotifyId instead of the entire ArtistNode object. This is because hashing strings is much more computationally efficient, and it avoids issues with object identity, where two different instances of ArtistNode with the same SpotifyId would otherwise be treated as distinct keys. By basing equality and hashing solely on SpotifyId, it also makes sure that dictionary lookups, insertions, and deletions remain efficient and consistent. This also simplifies comparisons, reducing the risk of unintended duplication in the adjacency matrix. It is not just tied to the ArtistNetwork but gets used throughout the solution.

ArtistNetwork

From my theoretical knowledge of networks, I know that a way to represent a network is by using an adjacency matrix. As decided earlier, I will be representing my matrix using a Dictionary, with this specific structure:

Key	Value
ArtistNode	Dictionary<ArtistNode, double>

Previously, I had another object called ArtistConnections, which had fields of the starting ArtistNode and the ending ArtistNode, as well as the weight of connection. I was representing the adjacency matrix as a dictionary keys of ArtistNodes and a values which were lists of these ArtistConnections. This is not as efficient as my current implementation because there were 2

instances of the same ArtistNode in the same row. Also, using a dictionary will be more efficient than using a list as it allows there to be linear time complexity lookups of the weight between two artists.

The key reason why I need to represent the artists in a network is because of the method FindPathWithDijkstras(). Given two ArtistNode objects, it uses Dijkstras algorithm to find the shortest path between any two artists. () Here is a pseudocode of how my implementation of Dijkstras works

```

Initialize an empty priority queue (priorityQueue)

Initialize an empty dictionary (distances) to store shortest distances to each artist
Initialize an empty dictionary (previous) to store previous nodes in the shortest path
Initialize an empty list (Path) to store the final path.

For each artist in the adjacency matrix:
    Set distance to infinity
    Set previous node to null.

Set distance of startArtistNode to 0
Enqueue startArtistNode into priorityQueue with priority 0

While priorityQueue is not empty:
    Dequeue the artist with the smallest distance (currentArtist).
    Get the current distance from distances dictionary.
    If currentArtist is the endArtistNode:
        Break the loop.
    For each connection of currentArtist:
        Get the neighbor node and connection weight.
        Calculate new distance (newDist) as currentDistance + connection weight.
        If newDist is smaller than the stored distance for the neighbor:
            Update the distance in distances dictionary.
            Update previous node in previous dictionary.
            Enqueue neighbor into priorityQueue with newDist as priority.

Set pathArtistNode to endArtistNode.
While pathArtistNode is not null:
    Insert pathArtistNode at the beginning of Path list.
    Move to the previous node in the path.

Return Path list.

```

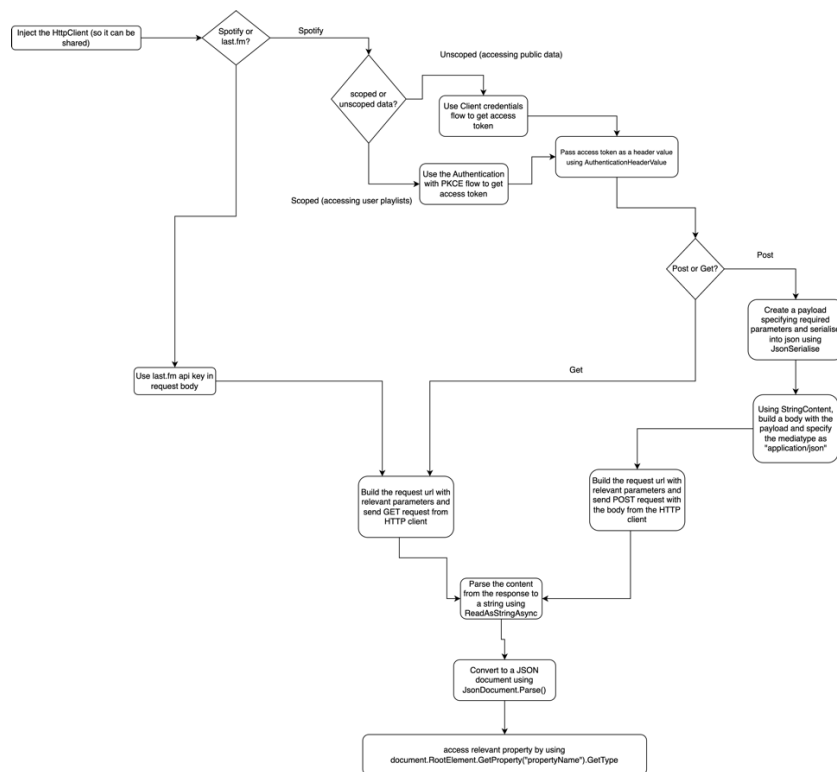
Calling External Services

Overview

I will need to use external APIs extensively throughout my code for these main goals:

- Using last.fm to get a list of artists which are like a starting artist + a score of how similar they are.
- Using spotify data to check whether the last.fm artist has a spotify profile + assigning a unique spotify id for an artist + removing any artists with too small of a popularity score
- Once a path of artists has been found between any two artists, finding songs for each artist + checking whether any two artists have a shared feature
- Once a list of songs has been found, creating a playlist in the user library and adding the songs to the user library

Here is a flowchart of how requests work:



LastFmApiExpanderService

There are two main methods which I use to be called relevant data from the Last.Fm api: Getting the list of 5 similar artists from an artist, as well as their weight, and getting the top tracks for an artist (for cross-reference with the spotify top tracks). For unscoped data last.fm does not

require any authorization process except giving my unique api key in the request, which I can obtain via registering for application privileges on their website,

GetSimilarArtists

The purpose of this function is, given a name of an artist, to get a dictionary of the top 5 most similar artists according to last.fm, where the key is the name of the similar artist and the value is the strength of connection between the given artist and the similar artist

artist.getSimilar

Get all the artists similar to this artist

Example URLs

JSON: /2.0/?method=artist.getsimilar&artist=cher&api_key=YOUR_API_KEY&format=json
XML: /2.0/?method=artist.getsimilar&artist=cher&api_key=YOUR_API_KEY

Params

limit (Optional): Limit the number of similar artists returned
artist (Required unless mbid): The artist name
autocorrect (0|1) (Optional): Transform misspelled artist names into correct artist names, returning the correct version instead. The corrected artist name will be returned in the response.
mbid (Optional): The musicbrainz id for the artist
api_key (Required): A Last.fm API key.

Auth

This service does **not** require authentication.

Sample Response

```
<similarartists artist="Cher">
  <artist>
    <name>Sonny & Cher</name>
    <mbid>3d6e4b6d-2788-458c-9722-9021965a8164</mbid>
    <match>1</match>
    <url>http://www.last.fm/music/Sonny%26%20Cher</url>
    <image size="small">http://userserve-ak.last.fm/serve/34/71168888.png</image>
    <image size="medium">http://userserve-ak.last.fm/serve/64/71168888.png</image>
    <image size="large">http://userserve-ak.last.fm/serve/126/71168888.png</image>
    <image size="extralarge">http://userserve-ak.last.fm/serve/252/71168888.png</image>
    <image size="mega">http://userserve-ak.last.fm/serve/368/71168888/Sonny+Cher.png</image>
    <streamable>1</streamable>
  </artist>
  ...
</similarartists>
```

```
ASYNC FUNCTION Dictionary<string, double> GetSimilarArtists(string artistName) {
    set request Url BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/", similarArtistParameters)
    wait for response from GET Request to RequestURL
    parse JsonDocument from Response as document
    parse artists from similarartists from root element of document

    foreach artist in artists
        if artist does not have properties "name" or "match"
            throw exception "Artist does not have name and match"
        parse foundArtist from element as Name
        parse match from JsonElement as MatchScore
        if foundArtist does not contain '&' then // Gets Rid of double artists like
            similarArtists[foundArtist] = MatchScore
    }
    Return similarArtists
}
```

GetTopTracksFromLastFm

The purpose of this function is to get the top tracks as a list, so it can be cross referenced to the spotify top tracks for the artist. This will hopefully act to verify whether two artists are the same even if they do not have the same name on the two platforms (**Travis \$cott** vs **Travis Scott**)

artist.getTopTracks

Get the top tracks by an artist on Last.fm, ordered by popularity

Example URLs

JSON: /2.0/?method=artist.gettoptracks&artist=cher&api_key=YOUR_API_KEY&format=json
XML: /2.0/?method=artist.gettoptracks&artist=cher&api_key=YOUR_API_KEY

Params

artist (Required unless mbid): The artist name
mbid (Optional): The musicbrainz id for the artist
autocorrect (0|1) (Optional): Transform misspelled artist names into correct artist names, returning the correct version instead. The corrected artist name will be returned in the response.
page (Optional): The page number to fetch. Defaults to first page.
limit (Optional): The number of results to fetch per page. Defaults to 50.
api_key (Required): A Last.fm API key.

Auth

This service does **not** require authentication.

Sample Response

```
<toptracks artist="Cher">
  <track name="1">
    <name>Believe</name>
    <mbid>
    <playcount>56135</playcount>
    <listeners>23217</listeners>
    <url>http://www.last.fm/music/Cher/_/Believe</url>
    <image size="small">...</image>
    <image size="medium">...</image>
    <image size="large">...</image>
  </track>
  ...
</toptracks>
```

```

ASYNC FUNCTION List<string> GetTopTracksFromLastFm(string artistName) {
    set request Url BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/", similarArtistParameters)
    send GET Request to RequestURL as Response
    parse JsonDocument from Response as document

    if document does not contain "toptracks" or "track" then
        throw InvalidOperationException("Artist has no top tracks")

    foreach JsonElement in
document.RootElement.GetProperty("toptracks").GetProperty("track").EnumerateArray() {
        parse trackName from JsonElement as Name
        Remove anything in brackets from trackName
        Add trackName.ToLower() to topTracks list
    }
    Return topTracks
}

```

SpotifyClientCredentialAuthorization

Since the expander runs entirely on the backend and doesn't need access to any user-specific or session-scoped data, using SpotifyClientCredentials for authentication is completely fine. SpotifyClientCredentials is designed for server-to-server authentication, meaning it doesn't require user interaction. It works by sending a POST request to the Spotify Accounts service with my client ID and client secret. In response, Spotify returns an access token that can be used to make API requests.³

GetAccessToken(),

Here is the key method, which returns a string int tuple of the accessToken and when expiry time is

```

ASYNC FUNCTION GetAccessToken{
    set HttpAuthHeader to ClientId, ClientSecret
    create StringContent with granttype=client_credentials
    Post HttpResponseMessage to https://accounts.spotify.com/api/token, stringcontent
    Wait for response as response
    Parse JsonDocument from response
    Parse AccessToken from JsonDocument as AccessToken
    Parse expires_in from JsonDocument as ExpiresIn
    return AccessToken as tuple (AccessToken, expires_in)
}

```

SpotifyExpanderApiService

The SpotifyExpanderApiService controls all the actions which gets unscoped data from spotify required for expanding the network. It is loosely coupled with the spotifyClientCredentials class via an interface and handles the refreshing of the spotifyClientCredentials accessToken, as well as getting the two main pieces of data needed to verify artists and add them to the spotify Database: the spotifyId, spotifyName and popularity of an artist and the top 10 popular songs of an artist, as well as their date of release

RefreshAccessToken

This subroutine is called before any API call. It checks whether the accesstoken is going to expire in the next minute, and if so, it uses the spotifyclientcredentials class to get another accesstoken

```

ASYNC SUBROUTINE RefreshAccessToken{
    if (access token expiry time is less than a minute)
        Access Token = await for SpotifyClientCredentialAuthorization.GetAccessToken()

```

GetArtistDetails

This method gets all the relevant information for an artist from the spotify v1 search endpoint, and returns it as a KVP with the structure of: Key(SpotifyId, name) Value(Popularity)

```

ASYNC FUNCTION KeyValuePair<(string, string), int> GetArtistDetails(string artistName){
    Wait for RefreshAccessToken()
    Set AuthorizationHeader with the Bearer Scheme and the AccessToken
    set request Url https://api.spotify.com/v1/search?q=artist:"{artistName}"&type=artist&limit=1
    send GET Request to RequestURL as Response
    parse JsonDocument from Response as document
    parse id from document as SpotifyId
    parse name from document as Name
    parse popularity from document as Popularity
    Return KeyValuePair <(spotifyId, name), popularity>

```

GetTopTracks

The purpose of this method is to get a dictionary of the top

```

ASYNC FUNCTION Dictionary<(string, DateTime> GetTopTracks(string spotifyId){
    Wait for RefreshAccessToken()
    Set AuthorizationHeader with the Bearer Scheme and the AccessToken
    set request Url https://api.spotify.com/v1/get-top-tracks"{spotifyId}"&type=artist&limit=1
    send GET Request to RequestURL as Response
    parse JsonDocument from Response as document
    foreach var JsonElement in JsonDocument.RootElement.EnumerateArray(){
        parse trackName from Element as Name
        Delete anything in brackets
        parse date time from document as dateTime
        Dictionary[trackName.ToLower()] = dateTime
    }
    Return Dictionary
}

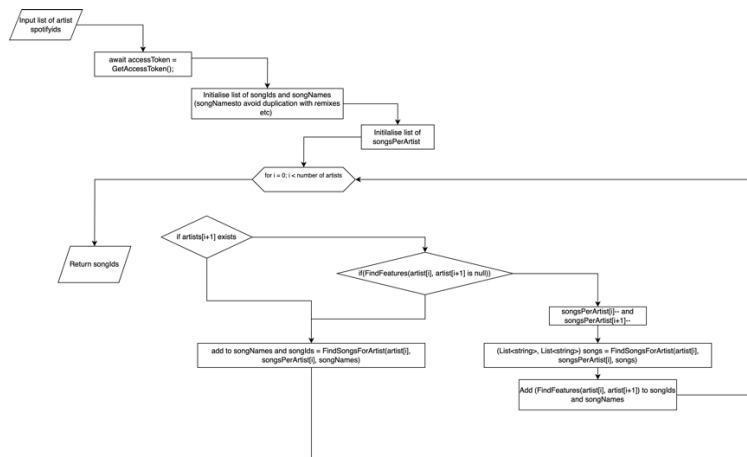
```

SpotifyGetPlaylistSongs

This class has the main priority of handling getting the playlist songs, given a path of artist ids. Whilst it uses the SpotifyClientCredentialAuthroziation flow to get the access token, it does not hold the token to be refreshed as the token is only needed when a request is made, unlike the backend process of SpotifyExpanderService where the token is being used very often.

GetPlaylistSongIds is the public method which is called from the outside

As this is quite a complicated query, I will use a flowchart to represent the general logic



NormaliseSongName

To avoid duplicate song names which may be under different spotify listings but with remixes/features/re-releases, I used regex to normalise song names. It gets rid of anything inbetween brackets, anything after a dash, eliminates any punctuation, and standardises whitespace

```

private string NormaliseSongName(songName)
    songName = songName.ToLower()
    songName = Regex.Replace(songName, @"\s*-\s*.*|\s*\(.?\)", "")
    songName = Regex.Replace(songName, @"[\^w\s]", "")
    songName = Regex.Replace(songName, @"\s{2,}", " ").Trim()
    return songName
  
```

The first regex `@"\s\*-\s\*.\*|\s\*\(.?\)"` removes features, versions, and additional artist info. The left side `"\s\*-\s\*.\*"` matches any whitespace before a dash, the dash itself, any whitespace after the dash, and then everything that follows. This removes text like " - Radio Edit" or " - feat. Artist Name". The right side `"\s\*\(.?\)"` matches any whitespace followed by an opening parenthesis, then all characters inside the parentheses and the closing parenthesis.

The second regex `@"[\^w\s]"` matches any character that is neither a word character nor whitespace. Word characters include letters, numbers, and underscores, so this regex effectively targets all punctuation and special characters.

The third regex `@"\s{2,}"` finds any sequence of two or more whitespace characters and replaces them with a single space. This ensures consistent spacing throughout the song name. The consecutive `.Trim()` method then removes any leading or trailing whitespace.

So after these transformations, song names like "Shape of You (feat. Ed Sheeran) [Radio Edit]" and "Shape of You - Acoustic Version" would both be normalised to "shape of you", making it easier to identify and prevent duplicates in the playlist generation process.

FindFeatures

This uses the spotify v1 endpoint to search for a feature between two artists, returning null if a feature does not exist. Only searches for features which only have the two artists on the song

```

ASYNC FUNCTION (string id, string normalizedName)? FindFeature(ArtistNode artist1, ArtistNode artist2)

    set query to UrlEncode("{artist1.Name} {artist2.Name}")
    set requestUrl to "https://api.spotify.com/v1/search?q={query}&type=track&limit=10"

    send GET Request to requestUrl as response
    parse JsonDocument from response as document

    if document doesn't have "tracks" property OR tracks doesn't have "items" property
        Return null

    foreach var track in document.tracks.items.EnumerateArray()
        set artists to track.artists.EnumerateArray()
        set hasArtist1 to false
        set hasArtist2 to false

        foreach var artist in artists
            if Compare(artist.name, artist1.Name, ignore case) equals true
                set hasArtist1 to true

            if Compare(artist.name, artist2.Name, ignore case) equals true
                set hasArtist2 to true

        set artistCount to track.artists.length

        if hasArtist1 AND hasArtist2 AND artistCount equals 2
            parse trackName from track as name
            parse songId from track as id
            set normalizedName to NormalizeSongName(trackName)
            Return (songId, normalizedName)
    Return null

```

FindSongsForArtists

This uses the get-top-tracks method to find a random list of songs for an artist. It uses the existing songNames to make sure that there are no duplicate songs / songs with the same name

```

ASYNC FUNCTION Tuple<List<string>, List<string>> FindSongsForArtist(ArtistNode artist, int
tracksPerArtist, List<string> songNames)
    Initialize List<string> songIds as empty
    Initialize List<string> newSongNames as empty

    Set AuthorizationHeader with the Bearer Scheme and the AccessToken
    set requestUrl to "https://api.spotify.com/v1/artists/{artist.SpotifyId}/top-tracks?market=US"

    send GET Request to requestUrl as response

    parse JsonDocument from response as document
    set topSongs to document.tracks.EnumerateArray()

    Initialize Dictionary<string, string> potentialSongs as empty

    foreach var song in topSongs
        set artistCount to song.artists.length
        set hasFeatures to artistCount > 1

        if NOT hasFeatures
            parse songId from song as id
            parse trackName from song as name
            set normalizedSongName to NormalizeSongName(trackName)

            set isDuplicate to false
            foreach var existingName in songNames
                if existingName contains " {normalizedSongName} " OR
                normalizedSongName contains " {existingName} " OR
                existingName equals normalizedSongName {
                    set isDuplicate to true
                    break

            if NOT isDuplicate

```

```

        Add (songId, normalizedSongName) to potentialSongs

    if potentialSongs count > 0
        Initialize randomGenerator
        set selectedSongs to random selection of tracksPerArtist songs from potentialSongs

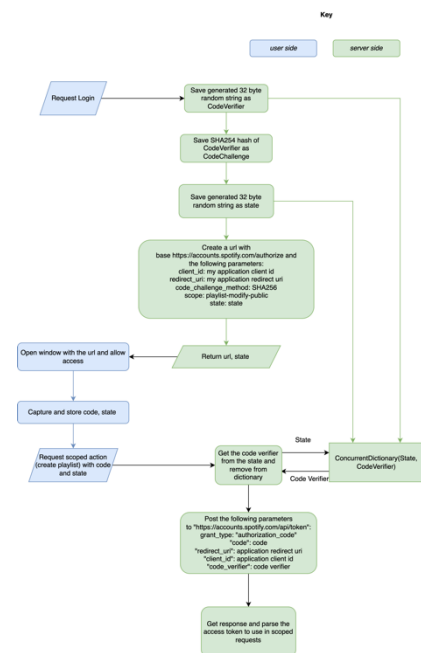
    foreach var song in selectedSongs
        Add song.Key to songIds
        Add song.Value to newSongNames

    Return (songIds, newSongNames)

```

SpotifyPkceCodeAuthorizationFlow

To access scoped data (which in my case is to be able to modify users playlists), I cannot use the Spotify Client Credential flow. Instead, I must use a different flow. The two choices I have are the Authorization flow or the Authorization with PKCE code flow. I will use the latter as I will use a website and that is not able to store the access token safely. To handle multiple users, I will use a concurrent dictionary to store the randomly generated code verifiers. As it requires two asynchronous systems to authenticate, it is easier to explain the flow in a flowchart. I will explain the methods in the actual code here:



GetAuthorizationURL

This method is called at the beginning of the authorization flow to get the URL. Returns an AuthUrl with the relevant parameters and a randomly generated state

ExchangeCode

This is a public method which is called once the user has authenticated to trade their code for an accesstoken. The user never handles the access token, instead it is put straight into relevant method

CreatePlaylistService

This is the class which handles the creation of the playlist once the songs for the playlist have been found. It uses an access token which is generated via the SpotifyPkceCodeAuthorizationFlow to access the private scope of playlist-modify-public for the user. So that it can handle concurrent users, whenever scoped data needs to be accessed, I won't set the authentication header for the entire http client but rather the just for the request

GetUserId

This method uses the /me search endpoint to get the userId of the current user

```

ASYNC FUNCTION STRING GetUserID(string accessToken)
    Initialize HttpRequestMessage request with:
        - Method: HttpMethod.Get
        - URL: "https://api.spotify.com/v1/me"
    Set request.Headers.Authorization to (Bearer scheme, accessToken)
    Send request async using _httpClient as response
    IF NOT response.IsSuccessStatusCode THEN
        Set errorContent to await response.Content.ReadAsStringAsync()
        THROW HttpRequestException("Failed to get user ID. Status: {response.StatusCode}. Response: {errorContent}")
    END IF
    Set jsonResponse to await response.Content.ReadAsStringAsync()
    Parse jsonResponse as JsonDocument named document
    RETURN document.RootElement.GetProperty("id").GetString()

```

AddPlaylistToUserLibrary

This method is a private helper method which creates the playlist with the right name and description in the user library

```

ASYNC FUNCTION STRING AddPlaylistToUserLibrary(List<string> songIds, string firstArtist, string lastArtist, string accessToken, string userId)
    Initialize HttpRequestMessage request with:
        - Method: HttpMethod.Get
        - URL: "https://api.spotify.com/v1/me"

    Initialize content object with:
        - name: "from {firstArtist} to {lastArtist}"
        - description: "Creates a playlist that transitions between {firstArtist} and {lastArtist}."
        - public: true

    Set request.Content to StringContent with:
        - JsonSerializer.Serialize(content)
        - Encoding.UTF8
        - "application/json"

    Set request.Headers.Authorization to "Bearer {accessToken}"
    Send request async using _httpClient as response

    IF NOT response.IsSuccessStatusCode THEN
        THROW HttpRequestException("Error creating playlist: {response.StatusCode}")
    END IF

    Set jsonResponse to response.Content.ReadAsStringAsync().Result
    Log "Playlist created successfully: {jsonResponse}"

    Parse jsonResponse as JsonDocument named document
    Extract playlistId from document.RootElement.GetProperty("id").GetString()

    Log "Playlist ID: {playlistId}"
    RETURN playlistId

```

AddTracksToPlaylists

The purpose of this method is to add the songs which have been already found to the playlist.

```

ASYNC FUNCTION VOID AddTracksToPlaylist(string playlistId, List<string> songIds, string accessToken) {
    Initialize HttpRequestMessage request with:
        - Method: HttpMethod.Post
        - URL: "https://api.spotify.com/v1/playlists/{playlistId}/tracks"

    Set request.Headers.Authorization to "Bearer {accessToken}"

    Initialize requestBody object with:
        - uris: songIds.Select(id => "spotify:track:{id}").ToArray()

```

```

        Set request.Content to StringContent with:
        - JsonSerializer.Serialize(requestBody)
        - Encoding.UTF8
        - "application/json"

        Send request async using _httpClient
    }

```

CreatePlaylist

This method is the public method which is called with parameters of the list of songs, the name of the first and last artist and the access token. It returns the link of the playlist. It contains two private helper methods which handle the two actions required to create the playlist: first creating the new playlist in the user library and add the songs to the playlist

```

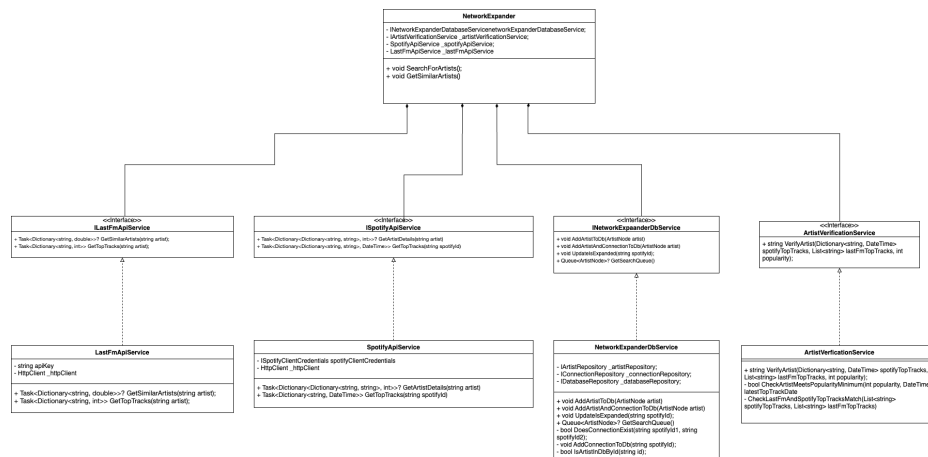
FUNCTION Async Task<string> CreatePlaylist(List<string> songIds, string firstArtist, string
lastArtist, string accessToken)
    PlaylistId = AddPlaylistToUserLibrary(firstArtist, lastArtist, accessToken)
    AddSongsToPlaylist(songIds, playlistId, accessToken)
}

```

Expanding the Network

Overview

As shown in my prototype, I know that I will need to use the Spotify API to verify the data received from last.fm, due to duplicate and dud artists. I need to design classes which follow my design paradigms of being loosely coupled via interfaces and modularity. Here is my design for the classes which handle the logic of expanding the network given a starting artist.



Normalising last.fm Data + Adding Spotify data

One drawback with last.fm is that there are a lot of “dud” artists. One key problem is that, when two artists have a song together, for example “Drake & Future”, on last.fm this is registered as one artist, rather than two separate artists. Luckily, all artists which this happens to have specifically an & in their name, so I can filter to exclude any artist names which include an “&”,

as it is likely to be a dud artist, the only major exceptions being duo acts like “Krept & Konan” or “Simon & Garfunkel”.

Another drawback of last.fm is that they do not store unique identifiers for every artist, only the musicbrainz id which many recent artists do not have. To kill two birds with one stone, I will save the Spotify ID alongside the artist’s name. This will require me to use the V1 Search Endpoint, querying with the name of the artist and parsing key information from the array of artist objects that is returned.

However, this search query introduces its own challenges. Firstly, it is not 100% accurate in returning the same artist which the last.fm name would indicate. This is quite hard to overcome, as we cannot filter for exact name matches as artists like Travis Scott have a last.fm name of “Travis \$cott”. However, we can use another call of the last.fm API: artist.GetTopTracks, and compare it against the result of the Spotify search to see if there are any matches. If none of the top tracks of last.fm match any of the top tracks on Spotify, then it is safe to assume that the Spotify artist is different to the last.fm artist.

ArtistVerificationService

This is a helper class for the NetworkExpander class. It contains methods for two checks:

- Whether the spotify artist which was the result of a v1 search is the same as the artist that was being referenced as a similar artist to the last.fm artist
- Whether the artist has a high enough popularity to be included in the database

VerifyArtist is the main public method

It takes into the date and name of the top spotify tracks, and the popularity of the artist, and returns a string which indicates the status of whether the artist is valid or not

```
FUNCTION string VerifyArtist(Dictionary<string, DateTime> spotifyTopTracks, List<string>
lastFmTopTracks, int popularity)
    set artistVerificationStatus = "Valid Artist"
    set latestTopTrackDate = Max(spotifyTopTracks.Values)
    if (spotifyTopTracks.Keys.ToList().Any(lastFmTopTracks).Contains) == false then
        set artistVerificationStatus = "The last fm and spotify top tracks do not match up"

    else if CheckArtistMeetsPopularityMinimum(popularity, latestTopTrackDate) == false then
        set artistVerificationStatus = "The popularity is too low: Popularity:{popularity},
Age:{(int)(DateTime.UtcNow - latestTopTrackDate).TotalDays/365}."

    Return artistVerificationStatus
```

Here is the **CheckArtistMeetsPopularityMinimum** method

```
FUNCTION boolean CheckPopularityMeetsMinimumRequirement(int popularity, DateTime latestTopTrackDate){
    set timeSinceRelease to CurrentDateTime minus latestTopTrackDate
    set yearsSinceRelease to (timeSinceRelease.TotalDays divided by 365) as integer

    set minPopularity to 65

    if yearsSinceRelease < 1
        set minPopularity to 50
    else if yearsSinceRelease <= 5
        set minPopularity to 55
    else if yearsSinceRelease <= 10
        set minPopularity to 60

    Return true if popularity >= minPopularity, false otherwise
}
```

ArtistNetworkExpanderDatabaseService

This is a class which aggregates interfaces from the 3 databaserepositories and allows the artsitnetworkexpander to access the following methods:

```
Queue<ArtistNode> GetSearchQueue();
void UpdateIsExpanded(string spotifyId);
void AddArtistToDb(ArtistNode artist);
bool IsArtistInDbById(string spotifyId);
```

As well as this it adds unique logic to add artists to the database dependent on whether a connection already exists between two artists

```
SUBROUTINE AddArtistAndConnectionToDb(Artist artist1, Artist artist2)
if IsArtistInDbById(artist1.SpotifyId) AND IsConnectionInDb(artist1.SpotifyId, artist2.SpotifyId)
    return

else if IsArtistInDbById(artist2.SpotifyId)
    if NOT IsArtistInDbById(artist1.SpotifyId)
        AddArtistToDb(artist1)
        AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight)
    else
        AddArtistToDb(artist2)
        AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight)
        Print in Green "Added {artist2.Name} to the artistNetworkDb and added a new connection between
{artist1.Name} and {artist2.Name}"
        Reset Console Color
```

ArtistNetworkExpander

This is a key class which brings together the LastFmExpanderService and the SpotifyExpanderService web with the goal of using a breadth-first search to try to combine all the elgiible artists into a connected network. To do this, I used a breadth-first search, first checking whether there is any artists in the database who have not had their similar artists found, and loading them into a queue. If not, I give a default artist (Drake) for the expander to load into the queue.

FindSimilarArtists

The purpose of this method is that, given a starting artist, it tries to find up to 5 similar artists and their corresponding weight to the starting artist.

```
FUNCTION Dictionary<ArtistNode, double> FetchVerifiedSimilarArtists(ArtistNode startingArtistNode)
lastFmSimilarArtists = GetSimilarArtists(startingArtistNode.Name) similarArtists = new
Dictionary<ArtistNode, double>()
vbnet
CopyEdit
FOR EACH similarArtist IN lastFmSimilarArtists
    similarArtistName = similarArtist.Key
    similarArtistWeight = similarArtist.Value
    lastFmTopTracks = GetTopTracks(similarArtistName)
    spotifyArtistDetails = GetArtistDetails(similarArtistName)
    spotifyId = spotifyArtistDetails.Key.Item1
    name = spotifyArtistDetails.Key.Item2
    popularity = spotifyArtistDetails.Value
    spotifyTopTracks = GetTopTracks(spotifyId)
    artistVerificationStatus = ArtistVerificationService.VerifyArtist(spotifyTopTracks,
lastFmTopTracks, popularity)
```

```

IF artistVerificationStatus # "Valid Artist"
    CONTINUE
foundArtist = new ArtistNode(name, spotifyId)
similarArtists[foundArtist] = similarArtistWeight

RETURN similarArtists

```

SearchForArtists

This method implements the breadth first search, enqueueing the results from FindSimilarArtists if they are not already in the network. This uses the artistverification, spotifyapiexpander and last.fm api expander service to return adictionary of simialr artists and their ocrreesponding weight to the starting artist.

```

load queue from database
if queue is empty
    enqueue default artist
callCount = 0
WHILE queue.Count > 0 AND spotifyApi is not timed out
    queueLength = queue.Count
    FOR i = 0 TO queueLength - 1
        currentArtist = Dequeue(queue)
        UpdateIsExpanded(currentArtist.SpotifyId)
        similarArtists = GetSimilarArtists(currentArtist)
        callCount = callCount + 1

    FOR EACH similarArtist IN similarArtists
        IF NOT IsArtistInDbById(similarArtist.Key.SpotifyId)
            Enqueue(queue, similarArtist.Key)
            AddArtistAndConnectionToDb(currentArtist, similarArtist.Key, similarArtist.Value)

```

Web Server

Overview

To serve the website, I will need to write my own API for my website to communicate with the backend. To do this, I will use the ASP.NET Core framework to create an API controller class that handles various HTTP requests. For local requests, my website will request at <http://localhost:5048/api/{path}>.

In my implementation, I have an ApiController class that inherits from ControllerBase. The [ApiController] attribute enables features like automatic model validation and improved error handling, while [Route("api")] defines the base route for all endpoints. The class constructor uses dependency injection to access core services, such as IArtistNetwork for pathfinding, IWebServerDatabaseService for querying the database, and ISpotifyPkceCodeAuthorizer for user authentication with Spotify.

Each endpoint is structured to handle a specific request:

- The ValidateArtist method checks whether a given artist is in the database, using webServerDatabaseService, and returns the name of the artist case corrected if it does
- FindPath calls artistNetwork.FindPathWithDijkstras to determine the shortest path between two artists.
- Authenticate initiates Spotify's authentication process using the PKCE flow.

- CreatePlaylist generates a playlist from the path found, retrieving song data via getPlaylistSongsService before calling createPlaylistService.
- ReportIssue writes user-reported issues to my
- GetAllArtists retrieves a list of all stored artists.

Initialising the server

The API class depends on several services I've written. To keep things loosely coupled, I use dependency injection to provide the necessary services as singletons. This lets me inject them into the API in ASP.NET Core.

Here is the flow of initialising the server at run time

```
Get databasePath, spotifyClientId, spotifyClientSecret, redirectUri as constructor arguments

Initialise WebApplicationBuilder builder
Add HttpClient as singleton to builder
Initilaise ArtistRepository, DatabaseRepository, ArtistNetworkDatabaseService with the database path
Add WebServerDatabaseService as a singleton to builder
Initialise SpotifyClientCredentialAuthorizer with spotifyClientId, spotifyClientSecret and HttpClient
as a singleton
Add SpotifyClientCredentialAuthorizer as singleton into builder
Initialise SpotifyPkceCodeAuthorizer with the spotifyClientId and redirect Uri
Add SpotifyPkceCodeAuthorizer as singleton into builder
Add GetPlaylistSongsService as singleton into builder
Add CreatePlaylistService as singleton into builder
Initialise ArtistNetworkDatabaseServiceFrom with databasePath
Initialise ArtistNetwork with ArtistNetworkDatabaseServiceFrom
Add ArtistNetwork as singleton

Add apiController to builder
Allow access from "http://127.0.0.1:8080"

Build app from builder
```

Api Class

Here is where I can provide endpoints for my website to communicate with.

ValidateArtist (GET)

Query: (http://localhost:5048/api/validate-artist?artistName ={artistName})

The purpose of this endpoint is to check whether a user-inputed artist exists in the database or not. It is able to deal with incorrect casing, returning the artist name with the correct casing as seen in the databaes

```
FUNCTION IActionResult ValidateArtist(string artistName)
    TRY
        Set exists to webServerDatabaseService.IsArtistInDbByName(artistName)
        if exists = true
            spotifyId = webServerDatabaseService.GetArtistIdByName(artistName)
            artistName = webserverDatabaseService.GetArtistNameById(spotifyId)
            return status code ok, ArtistExist = true, ArtistName = artistName
        return status code ok, ArtistExist = false, ArtistName = null
    CATCH exception
        RETURN StatusCode 500, Error = "An error occurred while checking the artist."
```

FindPath (GET)

Query: `http://localhost:5048/api/find-path?artist1={artist1}&artist2={artist2}`

The purpose of this endpoint is to find the shortest path between two artists, using the network loaded from the database

```

ASYNC FUNCTION IActionResult FindPath(string artist1, string artist2) {
    TRY
        Create ArtistNode artist1Node with name artist1 and ID from
        webServerDatabaseService.GetIdFromName(artist1)
        Create ArtistNode artist2Node with name artist2 and ID from
        webServerDatabaseService.GetIdFromName(artist2)

        path = artistNetwork.FindPathWithDijkstras(artist1Node, artist2Node)

        RETURN StatusCode Ok, PathId = path.Select(a => a.SpotifyId).ToList(), PathName =
        path.Select(a => a.Name).ToList()

    CATCH exception
        RETURN StatusCode 500 with { Error = "An error occurred while finding the path." }

```

Authenticate (GET)

The purpose of this result is to start the authentication process with the user. The authentication url, which is described in the PKCE authentication process, is given to the user for the user to log into spotify. The user can keep the code in their local browser safe because a code and state, not the final access token, is given to the user after authentication. The access token is created when required (when creating the playlist)

```

ASYNC FUNCTION IActionResult Authenticate() {
    TRY
        set (authUrl, state) to wait for spotifyPkceCodeAuthorizer.GetAuthorizationUrl()
        RETURN Ok response with { AuthUrl = authUrl, State = state }
    CATCH exception
        RETURN StatusCode 500 with { Error = "Authentication initialization failed" }

```

CreatePlaylist (POST)

This is where the creation of the playlist is handled. As this is a more complex request, it is a POST, not GET request, with the paramaters being passed in the body. This means that I had to create a data transfer object PlaylistRequest to parse the data from the post

```

class PlaylistRequest
    public string Code
    public string Path
    public string State

```

Here is the actual logic for the creation of playlist. It gets the name for each artists from the database from the id which is in the request, and gets the songs for the playlist according to the logic of the get songs class. The playlist is then created according to the playlist creation logic

```

ASYNC FUNCTION IActionResult CreatePlaylist(PlaylistRequest request) {
    TRY
        Set accessToken to await spotifyPkceCodeAuthorizer.ExchangeCode(request.Code, request.State)
        Initialize List<ArtistNode> artists as empty

        FOREACH spotifyId IN request.Path DO
            Set artistName to webServerDatabaseService.GetNameFromId(spotifyId)
            Add new ArtistNode(artistName, spotifyId) to artists

```

```

Set songIds to await getPlaylistSongsService.GetPlaylistSongIds(artists)

FOREACH songId IN songIds DO
    Log songId

Set playlistLink to await createPlaylistService.CreatePlaylist(
    songIds,
    artists.First().Name,
    artists.Last().Name,
    accessToken
RETURN Ok status code, Message = "Playlist created successfully.", PlaylistLink = playlistLink

CATCH exception
    RETURN StatusCode 500 with
        Error = "An error occurred while creating the playlist.",
        Details = exception.Message

```

ReportIssue

The purpose of this endpoint is to allow the user to report issues. Once user has reported an issue it is put inside an issue log text

```

FUNCTION IActionResult ReportIssue(string issue)
    TRY
        IF issue is null or empty THEN
            RETURN BadRequest with "Artist name cannot be empty."

        Set path to
        "/Users/jonathanlyria/RiderProjects/NettspendToSautiSol/NettspendToSautiSol/webserver/reportissue.txt"

        IF file does not exist at path THEN
            Create file at path and dispose

        Append issue to file at path

        RETURN Ok with "Issue reported for artist: {issue}"
    CATCH exception
        RETURN StatusCode 500 with "Internal server error: {exception.Message}"

```

GetAllArtists

The purpose of this endpoint is to get a list of all the artists and their name for the view-all-artists table

```

FUNCTION IActionResult GetAllArtists() {
    TRY
        artists = webServerDatabaseService.GetAllArtistNodes()
        RETURN Ok status code, Artists = artists.Select(a => new {name = a.Name, spotifyId = a.SpotifyId}).ToList()
    }
    CATCH exception
        RETURN StatusCode 500 with
            Error = "Failed to retrieve artists",
            Details = exception.Message

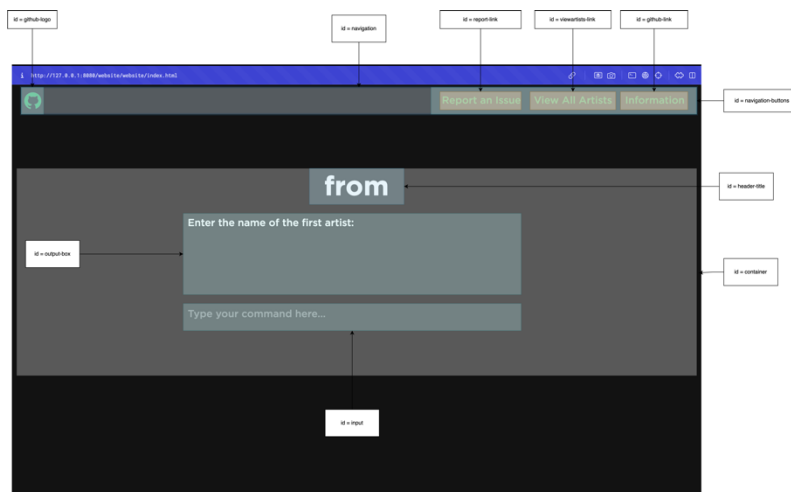
```

Website

Overview

To serve my client side, I will design a website with the logic being written in Javascript, the styles being done in CSS and the website structure being designed with HTML.

index.html

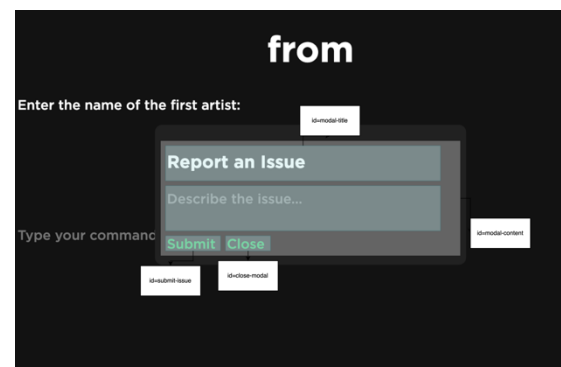


This page is where the main user input occurs. It functions similar to a terminal-based app, with the user entering text at the bottom and prompts being shown for what is needed from the user. The navigation pane at the top contains a link to github through the logo, a button to open the report issue modal (by changing the class of the

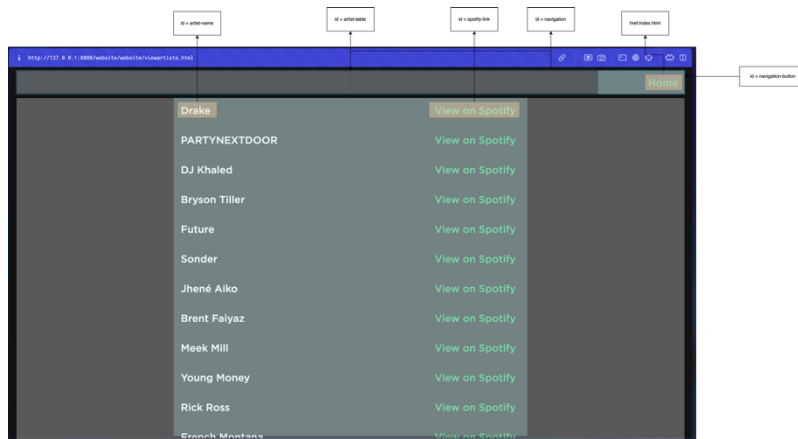
modal from.hidden to .modal-content), a button to open the view all artists page and a button to open the github readme.md file. The container in the center is centered in the screen, with the title which dynamically updates, an output box for prompts, and a textbox to type in prompts. Font-sizes dynamically change based on the size of the view window.

report issue modal

This is the modal which opens up when report an issue is clicked. There is a text box for the user to type in any complaints, rejecting any null entries. the button close changes the modal class back to .hidden. Submit sends the message to the server, to the api endpoint report-issue



view-all-artists.html



The purpose of this page is to show all the artists in the database, so that the user can check which artists they can try to find the connection between. When the page is loaded, the get all artists call from my webserver is called. I use inner html to create the table of artists. The navigation at the top borrows from the

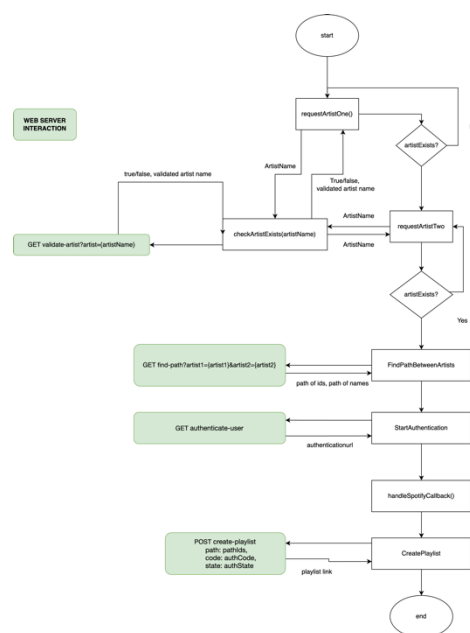
same stylesheet as the index.html to maintain consistency

callback.html

This is the page I specify in my spotify web api portal to handle user authentication callback. The purpose of this page is to extract the query parameters from the authentication flow. It gets the authentication code and the authentication state from the user logging in, and sends the data to the main page for the process to handle accordingly

Logic (script.js)

Here is the logic of the user. The final code is written in javascript. I have created a flowchart for a broader overview, with each function being shown more indepth in pseudocode.



GLOBAL VARIABLES:

Here are the global variables in this code:

const input: the input element (user input box in website)

const output: the output element (output box in website)

headerTitle: the header title

ProcessFlow()

This function starts the entire process. It sequentially requests two artists, finds a path between them, and if a valid path is found, it proceeds with authentication.

```
FUNCTION processFlow()
  artist1 = requestArtistOne()
  artist2 = requestArtistTwo(artist1)
  pathIds = findPathBetweenArtists(artist1, artist2)
  IF pathIds ≠ NULL
    requestAuthentication(pathIds)
```

RequestArtistOne()

This function prompts the user to enter the first artist's name. It keeps asking until a valid artist is found.

```
FUNCTION requestArtistOne() RETURNS STRING
  DISPLAY "Enter the name of the first artist:"
  WHILE TRUE
    WAIT FOR userInput
    inputName = TRIM(userInput)
    DISPLAY "> " + inputName
    (exists, artistName) = checkArtistExists(inputName)
    IF exists
      SET document.title = "from " + artistName
      SET headerTitle = "from " + artistName
      RETURN artistName
    ELSE
      DISPLAY "Artist not found. Please try again."
```

RequestArtistTwo()

This function asks the user to enter a second artist's name, ensuring it's valid before continuing.

```
FUNCTION requestArtistTwo(artist1) RETURNS STRING
  DISPLAY "Enter the name of the second artist:"
  WHILE TRUE
    WAIT FOR userInput
    inputName = TRIM(userInput)
    DISPLAY "> " + inputName
    (exists, artistName) = checkArtistExists(inputName)
    IF exists
      SET document.title = "from " + artist1 + " to " + artistName
      SET headerTitle = "from " + artist1 + " to " + artistName
      RETURN artistName
    ELSE
      DISPLAY "Artist not found. Please try again."
```

CheckArtistExists()

This function checks whether a given artist exists by sending a request to an external API.

```
FUNCTION checkArtistExists(artistName) RETURNS (BOOLEAN, STRING)
```

```

TRY
    SEND REQUEST TO API_BASE_URL + "/validate-artist?artistName=" + ENCODE(artistName)
    response = RECEIVE RESPONSE
    RETURN (response.artistExist, response.artistName)
CATCH ERROR
    RETURN (FALSE, NULL)

```

FindPathBetweenArtists()

This function attempts to find a path between the two selected artists by sending a request to an API.

```

FUNCTION findPathBetweenArtists(artist1, artist2) RETURNS LIST
    DISPLAY "Finding path between " + artist1 + " and " + artist2 + "..."
    TRY
        SEND REQUEST TO API_BASE_URL + "/find-path?artist1=" + ENCODE(artist1) + "&artist2=" +
        ENCODE(artist2)
        response = RECEIVE RESPONSE
        IF response.status = OK
            DISPLAY "Path found: " + JOIN(response.pathName, " -> ")
            RETURN response.pathId
        ELSE
            DISPLAY "Error: " + response.Error
            RETURN NULL
    CATCH ERROR
        DISPLAY "An error occurred: " + ERROR.MESSAGE
        RETURN NULL

```

RequestAuthentication()

This function prompts the user to press Enter to begin Spotify authentication.

```

FUNCTION requestAuthentication(pathIds)
    DISPLAY "Press Enter to sign into Spotify."
    WAIT FOR userInput
    DISPLAY "> Authenticating..."
    startAuthentication(pathIds)

```

StartAuthentication()

This function starts the authentication process by opening a login window and waiting for a response.

```

FUNCTION startAuthentication(pathIds)
    TRY
        SEND REQUEST TO API_BASE_URL + "/authenticate-user"
        response = RECEIVE RESPONSE
        authState = response.state
        OPEN WINDOW response.authUrl AS "Spotify Auth"

        WAIT FOR MESSAGE FROM WINDOW
        IF message.origin ≠ window.origin
            DISPLAY "Authentication failed: Origin mismatch"
            RETURN

        (code, state) = message.data
        IF state ≠ authState
            DISPLAY "Authentication failed: State mismatch"
            RETURN

        DISPLAY "Authentication successful."
        requestPlaylistCreation(pathIds, code, state)
    CATCH ERROR
        DISPLAY "Authentication failed: " + ERROR.MESSAGE

```

RequestPlaylistCreation()

This function asks the user to press Enter to proceed with playlist creation.

```
FUNCTION requestPlaylistCreation(pathIds, authCode, authState)
  DISPLAY "Press Enter to create the playlist."
  WAIT FOR userInput
  createPlaylist(pathIds, authCode, authState)
```

CreatePlaylist()

This function sends a request to create a playlist using the previously authenticated credentials.

```
FUNCTION createPlaylist(pathIds, authCode, authState)
  DISPLAY "Creating playlist, wait for 7ish seconds..."
  TRY
    SEND POST REQUEST TO API_BASE_URL + "/create-playlist"
    WITH BODY { path: pathIds, code: authCode, state: authState }
    response = RECEIVE RESPONSE
    IF response.status = OK
      DISPLAY "Playlist created successfully! Link: " + response.playlistLink
      OPEN LINK response.playlistLink
    ELSE
      DISPLAY "Error: " + response.error
  CATCH ERROR
    DISPLAY "An error occurred: " + ERROR.MESSAGE

CALL processFlow()
```

Technical Solution

Technical skills

Skill	Project//Class	Explanation
Dictionaries	DatabaseServices// DatabaseRepository	2D Dictionary of key ArtistNode and Value of Dictionary key ArtistNode and value weight are used as a return type from the SQL Server to return the artist network
	ExternalWebService// LastFmApiExpander	Dictionaries are used when getting the similar artists by using the artist name as a key and the weight as real
	ExternalWebService// SpotifyApiExpander	Dictionaries are used when getting top tracks of an artist with the key being a tuple of the name and of the track and the value being the date of creation
	NetworkExpander// ArtistNetworkExpander	Dictionaries are used when finding the similar artist in the GetSimilarArtists() method
	WebServer//ArtistNetwork	2D Dictionary of key ArtistNode and Value of Dictionary key ArtistNode and value weight is used to represent the network at runtime
Text Files	Web Server//Api.cs (reportissue.txt)	Text files are used to log user concerns
Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model	ExternalWebServices Every Class	Every class in external services calls parameterised web service APIs and parses complex JSON to service my complex client-server model. I use a wide variety of endpoints and POST + GET requests for my code. I handle complex server authentication with 3 different authentication flows, 2 of which are of the OAuth2.0 standard
	Website//Script.js	I call my own API to serve my website
Server-side scripting using request and response objects and server-side	WebServer//Api.cs	I have created my own web server using ASP.NET core. I have complex POST and GET endpoints, some of which require bespoke data transfer objects. All

extensions for a complex client-server model		complex operations happen on server-side to comply with separation of concerns
Hashing	ExternalWebServices// SpotifyPkceCodeAuthorizer.cs	I use SHA256 to hash a randomly generated string to get my codeverifier for my PKCE Authentication flow
Queues/Queue operations	DatabaseServices// DatabaseRepository.cs	I use queues as a return type for the GetSearchQueue method
	NetworkExpander// ArtistNetworkExpander.cs	I use a queue data structure to facilitate the breadth-first search
	WebServer//ArtistNetwork	I use a priority queue as a part of dijkstras algorithm
Lists	Almost all projects	I use lists extensively throughout all my code to store collections of data of the same time
List Operations	ExternalWebService// CreatePlaylist	I use list operations to convert the songIds to the correct type from a general spotify indicator to the required type spotify:track:{id}
	ExternalWebService// GetPlaylistSongs	I use AddRange list operations to add songNames to lists, as well as .Any and .Contains to determine whether a songName is already in the playlist
	ArtistNetworkExpander// ArtistVerificationService	I convert dictionary keys into a list. I also use .Any and .Contains to determine whether there is a shared song between two lists
	WebServer//ArtistNetwork	I use .Insert to convert the found artists on the path into a list
Graphs/Graph Operations	WebServer// ArtistNetwork	I use a dictionary to represent a weighted connected network where the nodes are the artists and the edges are the weight of connections between artists. I use dijkstras algorithm to find the shortest path between two artists on this network
	NetworkExpander// ArtistNetworkExpander	I use a breadth-first search to expand the network and find new artists and connections between artists
Cross table parameterised SQL	DatabaseServices// DatabaseRepository	I use an SQL SELECT JOIN statement to join the Artist and Connection table on the SpotifyId key
Data model in database	DatabaseServices	I use a database to store the relevant data for my code
Inheritance		

Interfaces	I use inheritance, interfaces and composition for almost all my code. All classes interact with each other through the interface for the class type
Composition	

Programming techniques

Technique	Explanation of how I have done it
Meaningful identifier names	Throughout my code, I have very descriptive identifier names which indicate what the variable's role is
Well-designed user interface	I have a well designed website with dynamic sizes for containers
Modularisation of code	All my code is contained in descriptive classes with descriptive with modules
Self Documented Code	I use comments where necessary to explain particularly difficult concepts or methods
Good use of local variables/Minimal use of global variables	I use local values where relevant/I sparingly use global variables throughout my code except where it would be smart to. For example, I use global variables for sharing the HttpClient to avoid socket exhaustion
Managed casting of types	I explicitly class all my types to increase readability
Use of constants	I use constants where required, for example when composing classes from interfaces
Appropriate indentation	My IDE (Rider) indents my code where necessary
Consistent style throughout	I use camelCase for local variable names, _snakeCase with a leading underscore for private global variables and PascalCase for class names
File Path Parameterized	I not only parameterise my file path but also pass it in the configuration
Modules (subroutines) with appropriate interfaces.	My functions and subroutines have appropriate interfaces. My classes only interact with each other using interfaces
Loosely coupled modules (subroutines) – module code interacts with other parts of the program through its interface only.	My classes are very loosely coupled as they only interact with each other through their interfaces, with the concrete class being injected at runtime
Cohesive modules (subroutines) – module code does just one thing/ Modules(collections of subroutines) – subroutines with common purpose grouped.	My classes are very cohesive
Defensive programming/Good exception handling.	I have great exception handling throughout my code through a use of try catch blocks. Specifically when dealing with external data, I catch specific

Global Types

ArtistNode

```
namespace GlobalTypes
{
    public class ArtistNode(string name, string spotifyId) // Used as a data type when handling artist data
    {
        public readonly string Name = name;
        public readonly string SpotifyId = spotifyId;

        public override bool Equals(object? obj) // needed so it can act as the key for a dictionary
        {
            if (obj is ArtistNode otherNode)
            {
                return SpotifyId == otherNode.SpotifyId; // checks whether other nodes spotify id is the same as this spotify id
            }
            return false;
        }

        public override int GetHashCode() // needed so it can act as the key for a dictionary
        {
            return SpotifyId.GetHashCode(); // just hashes the spotifyid
        }
    }
}
```

Database Services

Interfaces:

```
using GlobalTypes;

namespace DatabaseServices.Interfaces;

public interface IArtistRepository
{
    bool IsArtistInDbByName(string artistName);
    bool IsArtistInDbById(string id);
    string GetIdFromName(string artistName);
    string GetNameFromId(string spotifyId);
    List<ArtistNode> GetAllArtistNodes();
    void AddArtistToDb(ArtistNode artist);
    void UpdateIsExpanded(string spotifyId);
    Queue<ArtistNode> GetSearchQueue();
}
```

```
namespace DatabaseServices.Interfaces;

public interface IConnectionRepository
{
    bool IsConnectionInDb(string spotifyId1, string spotifyId2);
    void AddConnectionToDb(string spotifyId1, string spotifyId2, double weight);
}
```

```
using GlobalTypes;

namespace DatabaseServices.Interfaces;

public interface IDatabaseRepository
```

```

{
    Dictionary<ArtistNode, Dictionary<ArtistNode, double>> GetNetwork();

    void InitialiseDatabase();
}

```

ArtistRepository

```

using DatabaseServices.Interfaces;
using GlobalTypes;
using Microsoft.Data.Sqlite;

namespace DatabaseServices;

public class ArtistRepository : IArtistRepository
{
    private readonly string _connectionString;

    public ArtistRepository(string databasePath)
    {
        _connectionString = "Data Source=" + databasePath;
    }

    public bool IsArtistInDbByName(string artistName) // used by WebServer from user input
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();

            string selectQuery = "SELECT * FROM Artist WHERE LOWER(ArtistName) = LOWER(@artistName)";
            using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
            {
                cmd.Parameters.AddWithValue("@artistName", artistName);
                using (SqliteDataReader reader = cmd.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    public bool IsArtistInDbById(string id) // used by NetworkExpanderDbService
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();

            string selectQuery = "SELECT SpotifyID FROM Artist WHERE SpotifyID = @id";
            using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
            {
                cmd.Parameters.AddWithValue("@id", id);
                using (SqliteDataReader reader = cmd.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return true;
                    }
                }
            }
        }
        return false;
    }
}

```

```

public string GetIdFromName(string artistName) // Used by WebServerDbService
{
    string spotifyId = "";

    using (SqliteConnection connection = new SqliteConnection(_connectionString))
    {
        connection.Open();

        string selectQuery = @"SELECT SpotifyId FROM Artist WHERE ArtistName = @artistName COLLATE
NOCASE";
        using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
        {
            cmd.Parameters.AddWithValue("@artistName", artistName);
            using (SqliteDataReader reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    spotifyId = reader.GetString(0);
                }
            }
        }

        return spotifyId;
    }

    public string GetNameFromId(string spotifyId) // Used by WebServer to get name from spotifyId
    {
        string name = "";

        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();
            string selectQuery = @"SELECT ArtistName FROM Artist WHERE SpotifyId = @spotifyId COLLATE
NOCASE";
            using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
            {
                cmd.Parameters.AddWithValue("@spotifyId", spotifyId);
                using (SqliteDataReader reader = cmd.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        name = reader.GetString(0);
                    }
                }
            }

            return name;
        }

        public List<ArtistNode> GetAllArtistNodes() // Used by WebServer to get name, artistId of all the
        artists
        {
            List<ArtistNode> artists = new List<ArtistNode>();

            using (SqliteConnection connection = new SqliteConnection(_connectionString))
            {
                connection.Open();

                string selectQuery = "SELECT SpotifyId, ArtistName FROM Artist;";
                using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
                {
                    using (SqliteDataReader reader = cmd.ExecuteReader())
                    {
                        while (reader.Read())
                        {
                            string spotifyId = reader.GetString(0);
                            string artistName = reader.GetString(1);
                            artists.Add(new ArtistNode(artistName, spotifyId));
                        }
                    }
                }
            }
        }
    }

```

```

    }
}

return artists;
}

public void UpdateIsExpanded(string spotifyId) // used by ArtistExpander to indicate when the
similar artists have been found
{
    using (SqlConnection connection = new SqlConnection(_connectionString))
    {
        connection.Open();

        string updateQuery = "UPDATE Artist SET isExpanded = 1 WHERE SpotifyId = @spotifyId";
        using (SqlCommand cmd = new SqlCommand(updateQuery, connection))
        {
            cmd.Parameters.AddWithValue("@spotifyId", spotifyId);
            cmd.ExecuteNonQuery();
        }
    }
}

public void AddArtistToDb(ArtistNode artist) // used by artistexpander to add an artist to the
database
{
    using (SqlConnection connection = new SqlConnection(_connectionString))
    {
        connection.Open();

        string insertQuery =
            "INSERT INTO Artist (ArtistName, isExpanded, SpotifyID) VALUES (@artistName, 0,
@spotifyId)";
        using (SqlCommand cmd = new SqlCommand(insertQuery, connection))
        {
            cmd.Parameters.AddWithValue("@artistName", artist.Name);
            cmd.Parameters.AddWithValue("@spotifyId", artist.SpotifyId);
            cmd.ExecuteNonQuery();
        }
    }
}

public Queue<ArtistNode> GetSearchQueue() // used by artist expander to get a list of artists who
havent had their similar artists found
{
    Queue<ArtistNode> queue = new Queue<ArtistNode>();
    using (SqlConnection connection = new SqlConnection(_connectionString))
    {
        connection.Open();

        string selectQuery = @"
SELECT ArtistName, SpotifyID FROM Artist WHERE isExpanded = 0";

        using (SqlCommand cmd = new SqlCommand(selectQuery, connection))
        {
            using (SqlDataReader reader = cmd.ExecuteReader())
            {
                if (!reader.HasRows)
                    throw new NullReferenceException("Queue is empty");
                while (reader.Read())
                {
                    string artistName = reader.GetString(0);
                    string spotifyId = reader.GetString(1);

                    queue.Enqueue(new ArtistNode(artistName, spotifyId));
                }
            }
        }
    }
    return queue;
}

```

```

    }
}

```

Connection Repository

```

using DatabaseServices.Interfaces;
using Microsoft.Data.Sqlite;

namespace DatabaseServices;

public class ConnectionRepository : IConnectionRepository
{
    private readonly string _connectionString;
    public ConnectionRepository(string databasePath)
    {
        _connectionString = $"Data Source={databasePath}";
    }
    public bool IsConnectionInDb(string spotifyId1, string spotifyId2) // used by artistexpander to check whether a connection already exists before adding the connection
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();

            string selectQuery = @"
                SELECT * FROM Connections
                WHERE (SpotifyId1 = @spotifyId1 AND SpotifyId2 = @spotifyId2)
                OR (SpotifyId1 = @spotifyId2 AND SpotifyId2 = @spotifyId1);";

            using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
            {
                cmd.Parameters.AddWithValue("@spotifyId1", spotifyId1);
                cmd.Parameters.AddWithValue("@spotifyId2", spotifyId2);
                using (SqliteDataReader reader = cmd.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return true;
                    }
                    return false;
                }
            }
        }
    }
    public void AddConnectionToDb(string spotifyId1, string spotifyId2, double weight) // adds a connection to the database
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();
            string insertQuery = @"
                INSERT INTO Connections (SpotifyId1, SpotifyId2, Weight)
                VALUES (@spotifyId1, @spotifyId2, @strength);";
            using (SqliteCommand cmd = new SqliteCommand(insertQuery, connection))
            {
                cmd.Parameters.AddWithValue("@spotifyId1", spotifyId1);
                cmd.Parameters.AddWithValue("@spotifyId2", spotifyId2);
                cmd.Parameters.AddWithValue("@strength", weight);
                cmd.ExecuteNonQuery();
            }
        }
    }
}

```

Database Repository

```

using DatabaseServices.Interfaces;
using Microsoft.Data.Sqlite;

namespace DatabaseServices;

public class ConnectionRepository : IConnectionRepository
{
    private readonly string _connectionString;
    public ConnectionRepository(string databasePath)
    {
        _connectionString = $"Data Source={databasePath}";
    }
    public bool IsConnectionInDb(string spotifyId1, string spotifyId2) // used by artistexpander to
    check whether a connection already exists before adding the connection
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();

            string selectQuery = @"
                SELECT * FROM Connections
                WHERE (SpotifyId1 = @spotifyId1 AND SpotifyId2 = @spotifyId2)
                OR (SpotifyId1 = @spotifyId2 AND SpotifyId2 = @spotifyId1);";

            using (SqliteCommand cmd = new SqliteCommand(selectQuery, connection))
            {
                cmd.Parameters.AddWithValue("@spotifyId1", spotifyId1);
                cmd.Parameters.AddWithValue("@spotifyId2", spotifyId2);
                using (SqliteDataReader reader = cmd.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return true;
                    }
                    return false;
                }
            }
        }
    }
    public void AddConnectionToDb(string spotifyId1, string spotifyId2, double weight) // adds a
    connection to the database
    {
        using (SqliteConnection connection = new SqliteConnection(_connectionString))
        {
            connection.Open();
            string insertQuery = @"
                INSERT INTO Connections (SpotifyId1, SpotifyId2, Weight)
                VALUES (@spotifyId1, @spotifyId2, @strength);";
            using (SqliteCommand cmd = new SqliteCommand(insertQuery, connection))
            {
                cmd.Parameters.AddWithValue("@spotifyId1", spotifyId1);
                cmd.Parameters.AddWithValue("@spotifyId2", spotifyId2);
                cmd.Parameters.AddWithValue("@strength", weight);
                cmd.ExecuteNonQuery();
            }
        }
    }
}

```

External Web Services

Interfaces

```
namespace ExternalWebServices.Interfaces;

public interface ICreatePlaylistService
{
    Task<string> CreatePlaylist(List<string> songIds, string firstArtist, string lastArtist, string accessToken);
}
```

```
using GlobalTypes;

namespace ExternalWebServices.Interfaces;

public interface IGetPlaylistSongsService
{
    Task<List<string>> GetPlaylistSongIds(List<ArtistNode> pathOfArtists);
}
```

```
namespace ExternalWebServices.Interfaces;

public interface ILastFmApiService
{
    Task<Dictionary<string, double>>? GetSimilarArtists(string artist);
    Task<List<string>> GetTopTracks(string artist);
}
```

```
namespace ExternalWebServices.Interfaces;

public interface ISpotifyClientCredentialAuthorizer
{
    Task<(string AccessToken, int ExpiresIn)> GetAccessToken();
}
```

```
namespace ExternalWebServices.Interfaces;

public interface ISpotifyExpanderService
{
    Task<KeyValuePair<(string, string), int>> GetArtistDetails(string artistName);
    Task<Dictionary<string, DateTime>> GetTopTracks(string spotifyId);
}
```

```
namespace ExternalWebServices.Interfaces;

public interface ISpotifyPkceCodeAuthorizer
{
    (string AuthUrl, string State) GetAuthorizationUrl();
    string ExchangeCode(string code, string state);
}
```

CreatePlaylistService

```
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;
using ExternalWebServices.Interfaces;

namespace ExternalWebServices;

public class CreatePlaylistService: ICreatePlaylistService
```

```

{
    private readonly HttpClient _httpClient;
    public CreatePlaylistService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    // used by server to create playlist once songAgs have been found
    public async Task<string> CreatePlaylist(List<string> songIds, string firstArtist, string
lastArtist, string accessToken)
    {
        try
        {
            string playlistId = await AddPlaylistToUserLibrary(firstArtist, lastArtist, accessToken);
            await AddSongsToPlaylist(songIds, playlistId, accessToken);
            return $"https://open.spotify.com/playlist/{playlistId}";
        }
        catch (HttpRequestException ex)
        {
            throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
        }
        catch (JsonException ex)
        {
            throw new InvalidOperationException("Failed to parse Spotify API JSON response.", ex);
        }
        catch (FormatException ex)
        {
            throw new InvalidOperationException($"Invalid date format or value: {ex.Message}", ex);
        }
        finally
        {
            _httpClient.DefaultRequestHeaders.Authorization = null;
        }
    }

    private async Task<string> AddPlaylistToUserLibrary(string firstArtist, string lastArtist, string
accessToken)
    {
        string userId = await GetUserId(accessToken);

        try
        {
            var request = new HttpRequestMessage(
                HttpMethod.Post,
                $"https://api.spotify.com/v1/users/{userId}/playlists"
            );

            var content = new
            {
                name = $"from {firstArtist} to {lastArtist}",
                description = $"Creates a playlist that transitions between" +
                    $" {firstArtist} and {lastArtist}.",
                @public = true
            };

            request.Content = new StringContent(
                JsonSerializer.Serialize(content),
                Encoding.UTF8,
                "application/json"
            );

            request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);

            var response = await _httpClient.SendAsync(request);

            if (!response.IsSuccessStatusCode)
            {
                throw new HttpRequestException($"Error creating playlist: {response.StatusCode}");
            }

            string jsonResponse = response.Content.ReadAsStringAsync().Result;
            Console.WriteLine($"Playlist created successfully: {jsonResponse}");
        }
    }
}

```

```

        JsonDocument document = JsonDocument.Parse(jsonResponse);
        var playlistId = document.RootElement.GetProperty("id").GetString();

        Console.WriteLine($"Playlist ID: {playlistId}");
        return playlistId;
    }
    catch (HttpRequestException ex)
    {
        throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
    }
    catch (JsonException ex)
    {
        throw new InvalidOperationException("Failed to parse Spotify API JSON response.", ex);
    }
    catch (FormatException ex)
    {
        throw new InvalidOperationException($"Invalid date format or value: {ex.Message}", ex);
    }
    catch (Exception ex)
    {
        throw new InvalidOperationException($"Something went wrong: {ex.Message}", ex);
    }
    finally
    {
        _httpClient.DefaultRequestHeaders.Authorization = null;
    }
}

private async Task AddSongsToPlaylist(List<string> songIds, string playlistId, string accessToken)
{
    var request = new HttpRequestMessage(
        HttpMethod.Post,
        $"https://api.spotify.com/v1/playlists/{playlistId}/tracks"
    );

    request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);

    var requestBody = new
    {
        uris = songIds.Select(id => $"spotify:track:{id}").ToArray() // spotify can only parse
        uris with this format
    };

    request.Content = new StringContent(
        JsonSerializer.Serialize(requestBody),
        Encoding.UTF8,
        "application/json"
    );

    try
    {
        HttpResponseMessage response = await _httpClient.SendAsync(request);
        if (!response.IsSuccessStatusCode)
        {
            throw new Exception($"Failed to add tracks to the playlist {response.StatusCode}:
            {response.ReasonPhrase}.");
        }
    }
    catch (HttpRequestException ex)
    {
        throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
    }
    catch (JsonException ex)
    {
        throw new InvalidOperationException("Failed to send Spotify playlist response: json
        error", ex);
    }
    catch (FormatException ex)
    {
        throw new InvalidOperationException($"Invalid date format or value: {ex.Message}", ex);
    }
}

```

```

        finally
        {
            _httpClient.DefaultRequestHeaders.Authorization = null;
        }

    }

    private async Task<string> GetUserId(string accessToken)
    {
        try
        {
            var request = new HttpRequestMessage(
                HttpMethod.Get,
                $"https://api.spotify.com/v1/me"
            );

            request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);
            HttpResponseMessage response = await _httpClient.SendAsync(request); // I use SendAsync
            instead of PostAsync so that the webserver can handle concurrent sending
            if (!response.IsSuccessStatusCode)
            {
                string errorContent = await response.Content.ReadAsStringAsync();
                throw new HttpRequestException($"Failed to get user ID. Status: {response.StatusCode}.
                Response: {errorContent}");
            }

            string jsonResponse = await response.Content.ReadAsStringAsync();
            JsonDocument document = JsonDocument.Parse(jsonResponse);
            if (document.RootElement.GetProperty("id").GetString() == null)
                throw new Exception("Failed to retrieve user ID.");

            return document.RootElement.GetProperty("id").GetString();
        }
        catch (HttpRequestException ex)
        {
            throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
        }
        catch (JsonException ex)
        {
            throw new InvalidOperationException("Failed to send Spotify playlist response: json
            error", ex);
        }
        catch (FormatException ex)
        {
            throw new InvalidOperationException($"Invalid Format {ex.Message}", ex);
        }
        catch (Exception ex)
        {
            throw new Exception($"Failed to retrieve user ID: {ex.Message}");
        }
        finally
        {
            _httpClient.DefaultRequestHeaders.Authorization = null;
        }
    }
}

```

GetPlaylistSongsService

```

using System.Net.Http.Headers;
using System.Text.Json;
using System.Text.RegularExpressions;

```

```

using ExternalWebServices.Interfaces;
using GlobalTypes;

namespace ExternalWebServices
{
    public class GetPlaylistSongsService : IGetPlaylistSongsService
    {
        private readonly ISpotifyClientCredentialAuthorizer _clientCredentialAuthorizer;
        private readonly HttpClient _httpClient;

        public GetPlaylistSongsService(ISpotifyClientCredentialAuthorizer clientCredentialAuthorizer,
        HttpClient httpClient)
        {
            _httpClient = httpClient;
            _clientCredentialAuthorizer = clientCredentialAuthorizer;
        }

        private async Task<string> GetAccessToken()
        {
            try
            {
                var tokenData = await _clientCredentialAuthorizer.GetAccessToken();
                string accessToken = tokenData.AccessToken;

                Console.ForegroundColor = ConsoleColor.Cyan;
                Console.WriteLine("Access token refreshed successfully.");
                Console.ResetColor();
                return accessToken;
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error refreshing access token: {ex.Message}");
                throw new Exception("Failed to refresh access token.", ex);
            }
        }

        public async Task<List<string>> GetPlaylistSongIds(List<ArtistNode> pathOfArtists)
        {
            try
            {
                string accessToken = await GetAccessToken();

                List<string> songIds = new List<string>();
                List<string> songNames = new List<string>();
                List<int> songsPerArtist = pathOfArtists.Select(_ => 3).ToList();

                for (int i = 0; i < pathOfArtists.Count; i++)
                {
                    if (i + 1 < pathOfArtists.Count)
                    {
                        try
                        {
                            var featureResult = await FindFeature(pathOfArtists[i],
                                pathOfArtists[i + 1],
                                accessToken);

                            if (featureResult != null)
                            {
                                string featureId = featureResult.Value.id;
                                string featureName = featureResult.Value.name;

                                songsPerArtist[i]--;
                                songsPerArtist[i + 1]--; // this is so that there is only 3 songs per
artist

                                var artistSongs = await FindSongsForArtist(
                                    pathOfArtists[i],
                                    songsPerArtist[i],
                                    songNames, accessToken);

                                songIds.AddRange(artistSongs.Item1);
                                songNames.AddRange(artistSongs.Item2);

                                songIds.Add(featureId);

```

```

        songNames.Add(featureName);
    }
    else
    {
        (List<string>, List<string>) artistSongs = await FindSongsForArtist(
            pathOfArtists[i],
            songsPerArtist[i],
            songNames, accessToken);

        songIds.AddRange(artistSongs.Item1);
        songNames.AddRange(artistSongs.Item2);
    }
}
catch (Exception ex)
{
    Console.WriteLine($"Error processing artists {pathOfArtists[i].Name} and
{pathOfArtists[i + 1].Name}: {ex.Message}");

    try
    {
        (List<string>, List<string>) artistSongs = await FindSongsForArtist(
            pathOfArtists[i],
            songsPerArtist[i],
            songNames, accessToken);

        songIds.AddRange(artistSongs.Item1);
        songNames.AddRange(artistSongs.Item2);
    }
    catch (Exception songEx)
    {
        Console.WriteLine($"Failed to get songs for artist
{pathOfArtists[i].Name}: {songEx.Message}");
    }
}
}
else
{
    try
    {
        (List<string>, List<string>) artistSongs = await FindSongsForArtist(
            pathOfArtists[i],
            songsPerArtist[i],
            songNames, accessToken);

        songIds.AddRange(artistSongs.Item1);
        songNames.AddRange(artistSongs.Item2);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Failed to get songs for artist
{pathOfArtists[i].Name}: {ex.Message}");
    }
}

foreach (var trackId in songIds)
{
    Console.WriteLine(trackId);
}

return songIds;
}
catch (Exception ex)
{
    Console.WriteLine($"Critical error in GetPlaylistSongIds: {ex.Message}");
    throw new Exception("Failed to generate playlist song IDs", ex);
}
}

private async Task<(string id, string name)?> FindFeature(ArtistNode artist1, ArtistNode
artist2, string accessToken)
{
    try
    {

```

```

        Console.WriteLine($"Searching for feature between {artist1.Name} and {artist2.Name}");

        _httpClient.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", accessToken);

        string query = Uri.EscapeDataString($"{artist1.Name} {artist2.Name}");
        string url = $"https://api.spotify.com/v1/search?q={query}&type=track&limit=10";

        HttpResponseMessage response = await _httpClient.GetAsync(url);
        response.EnsureSuccessStatusCode();

        string jsonResponse = await response.Content.ReadAsStringAsync();
        JsonDocument document = JsonDocument.Parse(jsonResponse);

        if (!document.RootElement.TryGetProperty("tracks", out var tracks) ||
            !tracks.TryGetProperty("items", out var items))
        {
            Console.WriteLine($"No tracks found in search response for {artist1.Name} and
{artist2.Name}");
            return null;
        }

        foreach (JsonElement track in items.EnumerateArray())
        {
            var artists = track.GetProperty("artists").EnumerateArray();

            // verifies if the song actually has the first artist
            bool hasArtist1 = artists.Any(a => string.Equals(
                a.GetProperty("name").GetString(),
                artist1.Name,
                StringComparison.OrdinalIgnoreCase));

            // verifies if the song actually has the second artist
            bool hasArtist2 = artists.Any(a => string.Equals(
                a.GetProperty("name").GetString(),
                artist2.Name,
                StringComparison.OrdinalIgnoreCase));

            int artistCount = track.GetProperty("artists").GetArrayLength();

            // if there isnt more than 2 artists per song
            if (hasArtist1 && hasArtist2 && artistCount == 2)
            {
                string trackName = track.GetProperty("name").GetString();
                string songId = track.GetProperty("id").GetString();
                string normalizedName = NormalizeSongName(trackName);
                Console.WriteLine($"Found Feature: {trackName}");
                return (songId, normalizedName);
            }
        }

        Console.WriteLine($"No feature found between {artist1.Name} and {artist2.Name}");
        return null;
    }
    catch (HttpRequestException ex)
    {
        Console.WriteLine($"HTTP request failed when finding feature: {ex.Message}");
        throw new Exception($"Failed to search for feature between {artist1.Name} and
{artist2.Name}", ex);
    }
    catch (JsonException ex)
    {
        Console.WriteLine($"JSON parsing error when finding feature: {ex.Message}");
        throw new Exception($"Failed to parse response for feature between {artist1.Name} and
{artist2.Name}", ex);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Unexpected error finding feature: {ex.Message}");
        throw new Exception($"Unexpected error searching for feature between {artist1.Name}
and {artist2.Name}", ex);
    }
}

```

```

private async Task<(List<string>, List<string>>> FindSongsForArtist(ArtistNode artist, int
tracksPerArtist, List<string> previousSongNames, string accessToken)
{
    List<string> songIds = new List<string>();
    List<string> songNames = new List<string>();
    songNames.AddRange(previousSongNames);

    try
    {
        _httpClient.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", accessToken);

        string url = $"https://api.spotify.com/v1/artists/{artist.SpotifyId}/top-
tracks?market=US";
        Console.WriteLine(url);

        HttpResponseMessage response = await _httpClient.GetAsync(url);
        response.EnsureSuccessStatusCode();

        string jsonResponse = await response.Content.ReadAsStringAsync();
        JsonDocument document = JsonDocument.Parse(jsonResponse);
        var topSongs = document.RootElement.GetProperty("tracks").EnumerateArray();

        Dictionary<string, string> potentialSongs = new Dictionary<string, string>();
        foreach (JsonElement song in topSongs)
        {
            // i only want solo songs
            int artistCount = song.GetProperty("artists").GetArrayLength();
            bool hasFeatures = artistCount > 1;

            if (!hasFeatures)
            {
                string songId = song.GetProperty("id").GetString();
                string trackName = song.GetProperty("name").GetString();
                string normalizedSongName = NormalizeSongName(trackName);

                bool isDuplicate = songNames.Any(existingName =>
                    existingName.Contains($" {normalizedSongName} ") ||
                    normalizedSongName.Contains($" {existingName} ") ||
                    existingName.Equals(normalizedSongName));

                if (!isDuplicate)
                {
                    potentialSongs.Add(songId, normalizedSongName);
                }
            }
        }

        if (potentialSongs.Count > 0)
        {
            Random random = new Random();
            var selectedSongs = potentialSongs
                .OrderBy(x => random.Next())
                .Take(tracksPerArtist)
                .ToList();

            foreach (var song in selectedSongs)
            {
                songIds.Add(song.Key);
                songNames.Add(song.Value);
            }
        }

        return (songIds, songNames);
    }
    catch (HttpRequestException ex)
    {
        Console.WriteLine($"HTTP request failed for artist {artist.Name}: {ex.Message}");
        throw new Exception($"Failed to get top tracks for artist {artist.Name}", ex);
    }
    catch (JsonException ex)
    {
        Console.WriteLine($"JSON parsing error for artist {artist.Name}: {ex.Message}");
    }
}

```

```

        throw new Exception($"Failed to parse response for artist {artist.Name}", ex);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Unexpected error finding songs for artist {artist.Name}: {ex.Message}");
        throw new Exception($"Unexpected error getting songs for artist {artist.Name}", ex);
    }
}

private string NormalizeSongName(string songName)
{
    try
    {
        songName = songName.ToLower();
        songName = Regex.Replace(songName, @"\s*-\s*.*|\s*\(.?\)", "");
        songName = Regex.Replace(songName, @"^\w\s", "");
        songName = Regex.Replace(songName, @"\s{2,}", " ").Trim();
        return songName;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error normalizing song name: {ex.Message}");
        return songName;
    }
}
}
}

```

LastFmApiService

```

using System.Text.Json;
using System.Text.RegularExpressions;
using ExternalWebServices.Interfaces;

namespace ExternalWebServices;

public class LastFmApiService : ILastFmApiService
{
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;

    public LastFmApiService(string apiKey, HttpClient httpClient)
    {
        _httpClient = httpClient;
        _apiKey = apiKey;
    }

    public async Task<Dictionary<string, double>> GetSimilarArtists(string artistName)
    {
        var similarArtists = new Dictionary<string, double>();
        var similarArtistParameters = new Dictionary<string, string>
        {
            { "method", "artist.getSimilar" },
            { "artist", artistName },
            { "api_key", _apiKey },
            { "format", "json" },
            { "limit", "5" }
        };

        // this makes the last fm request url as its quite long
        string requestUrl = BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/",
            similarArtistParameters);

        try

```

```

{
    var response = await _httpClient.GetAsync(requestUrl);
    response.EnsureSuccessStatusCode();

    string responseContent = await response.Content.ReadAsStringAsync();

    // checks whether the response is empty
    if (string.IsNullOrEmpty(responseContent))
        throw new InvalidOperationException("Response content is empty.");

    using var document = JsonDocument.Parse(responseContent);

    // if it has the similar artists property
    if (!DoSimilarArtistsExist(document))
        throw new InvalidOperationException("Similar artists data is missing.");

    foreach (var artistData in document.RootElement.GetProperty("similarartists")
        .GetProperty("artist").EnumerateArray())
    {
        // if the artist is valid; if it actually has a name and a match
        if (!DoesLastFmArtistHaveNameAndMatch(artistData))
            throw new InvalidOperationException("Artist does not have name and match.");

        string foundArtist = artistData.GetProperty("name").GetString(!);
        double match = double.Parse(artistData.GetProperty("match").GetString(!));

        // gets rid of common Last.fm duplicate artist e.g. Drake & Future. Tradeoff: small %
        // of duo acts who actually have & in their names are filtered out
        if (!foundArtist.Contains('&'))
            similarArtists[foundArtist] = match;
    }

    return similarArtists;
}
catch (HttpRequestException ex)
{
    throw new HttpRequestException($"HTTP request failed: {ex.Message}", ex);
}
catch (JsonException ex)
{
    throw new InvalidOperationException("Failed to parse JSON response.", ex);
}
catch (Exception ex)
{
    throw new Exception($"Error {ex.Message}", ex);
}
}

public async Task<List<string>> GetTopTracks(string artistName)
{
    var topTracks = new List<string>();
    var topTrackParameters = new Dictionary<string, string>
    {
        { "method", "artist.getTopTracks" },
        { "artist", artistName },
        { "api_key", _apiKey },
        { "format", "json" },
    };

    // this makes the Last fm request url as its quite long
    string requestUrl = BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/",
topTrackParameters);

    try
    {
        var response = await _httpClient.GetAsync(requestUrl);
        response.EnsureSuccessStatusCode();

        string responseContent = await response.Content.ReadAsStringAsync();
        if (string.IsNullOrEmpty(responseContent))
            throw new InvalidOperationException("Response content is empty.");
    }
}

```

```

using var document = JsonDocument.Parse(responseContent);
if (!document.RootElement.TryGetProperty("toptracks", out var toptracks) ||
    !toptracks.TryGetProperty("track", out var tracksArray))
    throw new InvalidOperationException("Artist has no top tracks.");

foreach (var track in tracksArray.EnumerateArray())
{
    string topTrack = track.GetProperty("name").GetString() ?? "Unknown Track";
    if (topTrack == "Unknown Track")
        throw new InvalidOperationException("Top track name is unknown.");

    /*
     * Regex explanation:
     * gets rid of anything in brackets () or [], replaces with blank
     */
    topTrack = Regex.Replace(topTrack, @"\s*[\(\[\].*?[\]\)]\s*", "").Trim();

    topTracks.Add(topTrack.ToLower());
}

return topTracks;
}
catch (HttpRequestException ex)
{
    throw new HttpRequestException($"HTTP request failed: {ex.Message}", ex);
}
catch (JsonException ex)
{
    throw new InvalidOperationException("Failed to parse JSON response.", ex);
}
}

private string BuildUrlWithParams(string url, Dictionary<string, string> parameters)
{
    // creates the url and turns it into valid request
    List<string> paramList = new List<string>();
    foreach (KeyValuePair<string, string> param in parameters)
    {
        paramList.Add($"{param.Key}={param.Value}");
    }
    return $"{url}?{string.Join("&", paramList)}";
}

private bool DoSimilarArtistsExist(JsonDocument document)
{
    // checks whether the similar artist exists, if it does, checks whether it has the
    // artist array. returns true or false dependent
    return document.RootElement.TryGetProperty("similarartists", out JsonElement
similarArtistsElement) &&
        similarArtistsElement.TryGetProperty("artist", out _);
}

private bool DoesLastFmArtistHaveNameAndMatch(JsonElement artistData)
{
    // checks whether it has name and match property
    return artistData.TryGetProperty("name", out _) && artistData.TryGetProperty("match", out _);
}
}

```

SpotifyClientCredentialAuthorizer

```

using System.Text;
using System.Text.Json;
using ExternalWebServices.Interfaces;

// Citation of spotify Api Client Credentials
namespace ExternalWebServices;

public class SpotifyClientCredentialAuthorizer : ISpotifyClientCredentialAuthorizer
{
    private readonly string _clientId;
    private readonly string _clientSecret;
}

```

```

private const string TokenUrl = "https://accounts.spotify.com/api/token";
private readonly HttpClient _httpClient;

public SpotifyClientCredentialAuthorizer(string clientId, string clientSecret, HttpClient
httpClient)
{
    _httpClient = httpClient;
    _clientId = clientId;
    _clientSecret = clientSecret;
}

public async Task<(string AccessToken, int ExpiresIn)> GetAccessToken()
{
    string authHeader =
Convert.ToBase64String(Encoding.UTF8.GetBytes($"{{_clientId}}:{{_clientSecret}}"));
    _httpClient.DefaultRequestHeaders.Authorization = new
System.Net.Http.Headers.AuthenticationHeaderValue("Basic", authHeader);

    // from spotify api docs
    StringContent requestContent = new StringContent("grant_type=client_credentials",
Encoding.UTF8, "application/x-www-form-urlencoded");
    HttpResponseMessage response = await _httpClient.PostAsync(TokenUrl, requestContent);

    if (!response.IsSuccessStatusCode)
    {
        throw new Exception($"Failed to retrieve access token. Status code: {response.StatusCode},
Response: {await response.Content.ReadAsStringAsync()}");
    }

    string document = await response.Content.ReadAsStringAsync();
    JsonElement tokenData = JsonSerializer.Deserialize<JsonElement>(document);

    string? accessToken = tokenData.GetProperty("access_token").GetString();
    int expiresIn = tokenData.GetProperty("expires_in").GetInt32();

    if (string.IsNullOrEmpty(accessToken))
        throw new Exception("Access token not found in response.");

    Console.WriteLine($"Access token: {accessToken}");

    return (accessToken, expiresIn);
}
}

```

SpotifyExpanderService

```

using System.Net;
using System.Net.Http.Headers;
using System.Text.Json;
using System.Text.RegularExpressions;
using ExternalWebServices.Interfaces;

namespace ExternalWebServices;

public class SpotifyExpanderService : ISpotifyExpanderService
{
    private readonly ISpotifyClientCredentialAuthorizer _spotifyClientCredentialAuthorizer;
    private readonly HttpClient _httpClient;
    private string _accessToken; // global property so that it doesnt have to be refreshed every
single time it is called
    private DateTime _tokenExpiryTime;

    public SpotifyExpanderService(ISpotifyClientCredentialAuthorizer
spotifyClientCredentialAuthorizer, HttpClient httpClient)
    {
        _spotifyClientCredentialAuthorizer = spotifyClientCredentialAuthorizer;
    }

```

```

        _httpClient = httpClient;
        _accessToken = string.Empty;
        _tokenExpiryTime = DateTime.UtcNow;
    }

    private async Task RefreshAccessToken() // This ensures that the accessToken doesnt get refreshed unnecessarily
    {
        if (DateTime.UtcNow >= _tokenExpiryTime.AddMinutes(-1))
        {
            try
            {
                (string AccessToken, int ExpiresIn) tokenData = await
                _spotifyClientCredentialAuthorizer.GetAccessToken();
                _accessToken = tokenData.AccessToken;
                _tokenExpiryTime = DateTime.UtcNow.AddSeconds(tokenData.ExpiresIn);

                Console.ForegroundColor = ConsoleColor.Cyan;
                Console.WriteLine("Access token refreshed successfully.");
                Console.ResetColor();
                _httpClient.DefaultRequestHeaders.Authorization = new
                AuthenticationHeaderValue("Bearer", _accessToken);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error refreshing access token: {ex.Message}");
                throw new Exception("Failed to refresh access token.", ex);
            }
        }
    }

    public async Task<KeyValuePair<(string, string), int>> GetArtistDetails(string artistName)
    {
        await RefreshAccessToken();
        _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
        _accessToken);

        try
        {
            string url =
            $"https://api.spotify.com/v1/search?q=artist:{Uri.EscapeDataString(artistName)}&type=artist&limit=1";

            var response = await _httpClient.GetAsync(url);
            response.EnsureSuccessStatusCode();

            string responseContent = await response.Content.ReadAsStringAsync();
            if (string.IsNullOrEmpty(responseContent))
                throw new InvalidOperationException("Spotify API returned an empty response.");

            using var document = JsonDocument.Parse(responseContent);
            var artistData = document.RootElement
                .GetProperty("artists")
                .GetProperty("items")
                .EnumerateArray()
                .FirstOrDefault();

            // checks whether there is actually an artist in teh data returned
            if (artistData.ValueKind == JsonValueKind.Undefined)
                throw new InvalidOperationException("No artist found for the given name.");
            try
            {
                string spotifyId = artistData.GetProperty("id").GetString() ?? throw new
                InvalidOperationException();
                string name = artistData.GetProperty("name").GetString() ?? throw new
                InvalidOperationException();
                int popularity = artistData.GetProperty("popularity").GetInt32();
                return new KeyValuePair<(string, string), int>((spotifyId, name), popularity);
            }
            catch (NullReferenceException ex)
            {

```

```

        throw new InvalidOperationException("Artist does not have valid data.");
    }

}

catch (HttpRequestException ex)
{
    throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
}
catch (JsonException ex)
{
    throw new InvalidOperationException("Failed to parse Spotify API JSON response.", ex);
}
finally
{
    _httpClient.DefaultRequestHeaders.Authorization = null;
}
}

public async Task<Dictionary<string, DateTime>> GetTopTracks(string spotifyId)
{
    await RefreshAccessToken();
    Console.WriteLine($"{spotifyId}");
    _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
_accessToken);

    var topTracks = new Dictionary<string, DateTime>();

    try
    {
        string url = $"https://api.spotify.com/v1/artists/{spotifyId}/top-tracks";

        var response = await _httpClient.GetAsync(url);

        string responseContent = await response.Content.ReadAsStringAsync();

        if (string.IsNullOrEmpty(responseContent))
            throw new InvalidOperationException("Spotify API returned an empty response.");

        var document = JsonDocument.Parse(responseContent);
        foreach (var topTrack in document.RootElement.GetProperty("tracks").EnumerateArray())
        {
            string topTrackName = topTrack.GetProperty("name").GetString(!);

            // regex: gets rid of anything in brackets
            topTrackName = Regex.Replace(topTrackName, @"\s*[\(\[\].*?[\]\)]\s*", "").Trim();

            string releaseDateStr =
topTrack.GetProperty("album").GetProperty("release_date").GetString(!);

            /* Regex: checks whether the date is in the correct format
               Dates from spotify are sometimes just the year, or just the month, so it
               allows those to be null. Must at least have a year
            */
            if (!Regex.IsMatch(releaseDateStr, @"^\d{4}(-\d{2}(-\d{2})?)?$"))
            {
                throw new FormatException($"Invalid date format: {releaseDateStr}");
            }

            string[] parts = releaseDateStr.Split('-');
            int year = int.Parse(parts[0]);
            int month = parts.Length >= 2 ? int.Parse(parts[1]) : 1; // january is the default
month
            int day = parts.Length >= 3 ? int.Parse(parts[2]) : 1; // january first is the default
day

            DateTime topTrackDate;
            try
            {
                topTrackDate = new DateTime(year, month, day);
            }
            catch (ArgumentOutOfRangeException ex)
            {

```

```

        throw new FormatException($"Invalid date values in: {releaseDateStr}", ex);
    }

    topTracks[topTrackName.ToLower()] = topTrackDate;
}
return topTracks;
}
catch (HttpRequestException ex)
{
    throw new HttpRequestException($"Spotify API request failed: {ex.Message}", ex);
}
catch (JsonException ex)
{
    throw new InvalidOperationException("Failed to parse Spotify API JSON response.", ex);
}
catch (FormatException ex)
{
    throw new InvalidOperationException($"Invalid date format or value: {ex.Message}", ex);
}
finally
{
    _httpClient.DefaultRequestHeaders.Authorization = null;
}
}
}

```

SpotifyPkceCodeAuthorizer

```

using System.Collections.Concurrent;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using ExternalWebServices.Interfaces;

namespace ExternalWebServices;

public class SpotifyPkceCodeAuthorizer : ISpotifyPkceCodeAuthorizer
{
    private readonly string _clientId;
    private readonly string _redirectUri;
    private readonly ConcurrentDictionary<string, string> _codeVerifiers = new();
    private readonly HttpClient _httpClient;

    public SpotifyPkceCodeAuthorizer(HttpClient httpClient, string clientId, string redirectUri)
    {
        _httpClient = httpClient;
        _clientId = clientId;
        _redirectUri = redirectUri;
    }

    public async Task<(string AuthUrl, string State)> GetAuthorizationUrl()
    {
        string codeVerifier = GenerateCodeVerifier(); // generates random string
        string codeChallenge = GenerateCodeChallenge(codeVerifier); // hashes code verifier
        string state = GenerateState(); // generates reandom string

        _codeVerifiers.TryAdd(state, codeVerifier);

        string authUrl =
        $"https://accounts.spotify.com/authorize?response_type=code&client_id={_clientId}" +
        $"&redirect_uri={Uri.EscapeDataString(_redirectUri)}&code_challenge_method=S256" +
        $"&code_challenge={codeChallenge}&scope=playlist-modify-
        public&state={state}";
    }
}

```

```

        return (authUrl, state);
    }

    public async Task<string> ExchangeCode(string code, string state)
    {
        if (!_codeVerifiers.TryRemove(state, out string codeVerifier)) // remove the code verifier from the state
            throw new Exception("Invalid state or code verifier not found.");

        return await ExchangeAuthorizationCodeForAccessToken(code, codeVerifier);
    }

    private async Task<string> ExchangeAuthorizationCodeForAccessToken(string code, string codeVerifier) // get the access token
    {
        HttpResponseMessage response = await
        _httpClient.PostAsync("https://accounts.spotify.com/api/token",
            new FormUrlEncodedContent(new[]
            {
                new KeyValuePair<string, string>("grant_type", "authorization_code"),
                new KeyValuePair<string, string>("code", code),
                new KeyValuePair<string, string>("redirect_uri", _redirectUri),
                new KeyValuePair<string, string>("client_id", _clientId),
                new KeyValuePair<string, string>("code_verifier", codeVerifier)
            }));

        if (!response.IsSuccessStatusCode)
            throw new Exception($"Failed to exchange authorization code. Status: {response.StatusCode}");

        string jsonResponse = await response.Content.ReadAsStringAsync();
        JsonDocument document = JsonDocument.Parse(jsonResponse);
        return document.RootElement.GetProperty("access_token").GetString();
    }

    private string GenerateCodeVerifier()
    {
        byte[] bytes = new byte[32];
        using RandomNumberGenerator rng = RandomNumberGenerator.Create();
        rng.GetBytes(bytes);
        return Convert.ToBase64String(bytes)
            .Replace("+", "-")
            .Replace("/", "_")
            .Replace("=", "");
    }

    private string GenerateCodeChallenge(string codeVerifier)
    {
        using SHA256 sha256 = SHA256.Create();
        byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(codeVerifier));
        return Convert.ToBase64String(bytes)
            .Replace("+", "-")
            .Replace("/", "_")
            .Replace("=", "");
    }

    private string GenerateState()
    {
        byte[] bytes = new byte[32];
        using RandomNumberGenerator rng = RandomNumberGenerator.Create();
        rng.GetBytes(bytes);
        return Convert.ToBase64String(bytes)
            .Replace("+", "-")
            .Replace("/", "_")
            .Replace("=", "");
    }
}

```

NetworkExpander

Interfaces

```
namespace expander;

public interface IArtistVerificationService
{
    string VerifyArtist(Dictionary<string, DateTime> spotifyTopTracks, List<string> lastFmTopTracks,
        int popularity);
}
```

```
using GlobalTypes;

namespace DatabaseServices.Interfaces;

public interface INetworkExpanderDatabaseService
{
    Queue<ArtistNode> GetSearchQueue();
    void UpdateIsExpanded(string spotifyId);
    void AddArtistAndConnectionToDb(ArtistNode artist1, ArtistNode artist2, double weight);
    void AddArtistToDb(ArtistNode artist);

    bool IsArtistInDbById(string spotifyId);
}
```

Program.cs (program entry point #1)

```
using DatabaseServices;
using ExternalWebServices;
using GlobalTypes;

namespace expander
{
    public class Program
    {
        public static async Task Main(string[] args)
        {
            string databasePath = args[0]; // pass as constructor argument
            Console.WriteLine("Enter last fm api key");
            string lastFmApiKey = Console.ReadLine();

            Console.WriteLine("Enter Spotify Client Id");
            string spotifyClientId = Console.ReadLine();

            Console.WriteLine("Enter Spotify Client Secret");
            string spotifyClientSecret = Console.ReadLine();

            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine($"database path: {databasePath} last fm {lastFmApiKey} spotify {spotifyClientId} || {spotifyClientSecret}");
            Console.ResetColor();

            HttpClient client = new HttpClient();

            DatabaseRepository databaseRepository = new DatabaseRepository(databasePath);
            ArtistRepository artistRepository = new ArtistRepository(databasePath);
            ConnectionRepository connectionRepository = new ConnectionRepository(databasePath);

            SpotifyClientCredentialAuthorizer spotifyClientCredentialAuthorizer = new
            SpotifyClientCredentialAuthorizer(spotifyClientId, spotifyClientSecret, client);

            LastFmApiService lastFmApiService = new LastFmApiService(lastFmApiKey, client);
            SpotifyExpanderService spotifyExpanderService = new
            SpotifyExpanderService(spotifyClientCredentialAuthorizer, client);
            ArtistVerificationService artistVerificationService = new ArtistVerificationService();

            NetworkExpanderDatabaseService artistNetworkDatabaseService = new
```

```

NetworkExpanderDatabaseService(artistRepository, connectionRepository, databaseRepository);
    ArtistNetworkExpander artistExpander = new
ArtistNetworkExpander(artistNetworkDatabaseService, lastFmApiService, spotifyExpanderService,
artistVerificationService);

    ArtistNode startingArtistNode = new ArtistNode("Drake", "3TVXtAsR1Inumwj472S9r4");

    await artistExpander.SearchForArtists(startingArtistNode);

    }
}
}

```

NetworkExpanderDatabaseService

```

using DatabaseServices.Interfaces;
using GlobalTypes;

namespace expander;

public class NetworkExpanderDatabaseService: INetworkExpanderDatabaseService
{
    private readonly IArtistRepository _artistRepository;
    private readonly IConnectionRepository _connectionRepository;
    private readonly IDatabaseRepository _databaseRepository;

    public NetworkExpanderDatabaseService(IArtistRepository artistRepository, IConnectionRepository
connectionRepository, IDatabaseRepository databaseRepository)
    {
        _databaseRepository = databaseRepository;
        _artistRepository = artistRepository;
        _connectionRepository = connectionRepository;

        _databaseRepository.InitialiseDatabase();
    }

    public Queue<ArtistNode> GetSearchQueue()
    {
        return _artistRepository.GetSearchQueue();
    }

    public void AddArtistAndConnectionToDb(ArtistNode artist1, ArtistNode artist2, double weight)
    {
        if (IsArtistInDbById(artist1.SpotifyId) && IsConnectionInDb(artist1.SpotifyId,
artist2.SpotifyId))
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"{artist2.Name} is in artistNetworkDb and connection exists for
{artist1.Name} -> {artist2.Name}");
            Console.ResetColor();
        }
        else if (IsArtistInDbById(artist2.SpotifyId)) // if the artist is in but the connection isnt
        {
            // Check if artist1 exists, if not, add it
            if (!IsArtistInDbById(artist1.SpotifyId))
            {
                AddArtistToDb(artist1);
            }
            try
            {
                AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight);
                Console.ForegroundColor = ConsoleColor.Green;
                Console.WriteLine($"Added connection between {artist1.Name} and {artist2.Name}");
                Console.ResetColor();
            }
            catch (Exception ex)
            {
                throw new Exception(ex.Message);
            }
        }
        else // if the artist isnt in the database then the connection cant eb
        {
            try

```

```

        {
            AddArtistToDb(artist2);
            AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight);
        }
        catch (Exception ex)
        {
            throw new Exception(ex.Message);
        }
    }

    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine($"Added {artist2.Name} to the artistNetworkDb and added a new connection
between {artist1.Name} and {artist2.Name}");
    Console.ResetColor();
}
}

public void AddArtistToDb(ArtistNode artistNode)
{
    _artistRepository.AddArtistToDb(artistNode);
}

public void UpdateIsExpanded(string spotifyId)
{
    _artistRepository.UpdateIsExpanded(spotifyId);
}

public bool IsArtistInDbById(string spotifyId)
{
    return _artistRepository.IsArtistInDbById(spotifyId);
}

private bool IsConnectionInDb(string spotifyId1, string spotifyId2)
{
    return _connectionRepository.IsConnectionInDb(spotifyId1, spotifyId2);
}

private void AddConnectionToDb(string spotifyId1, string spotifyId2, double weight)
{
    _connectionRepository.AddConnectionToDb(spotifyId1, spotifyId2, weight);
}
}
}

```

ArtistVerificationService

```

namespace expander;

public class ArtistVerificationService : IArtistVerificationService
{
    public void VerifyArtist(Dictionary<string, DateTime> spotifyTopTracks, List<string>
lastFmTopTracks,
    int popularity)
    {
        DateTime latestTopTrackDate = spotifyTopTracks.Values.Max();
        if (!CheckLastFmAndSpotifyTopTracksMatch(spotifyTopTracks.Keys.ToList(), lastFmTopTracks))
        {
            throw new Exception("The last fm and spotify top tracks do not match up");
        }
        if (!CheckArtistMeetsPopularityMinimum(popularity, latestTopTrackDate))
        {
            throw new Exception(
                $"The popularity is too low: Popularity:{popularity}, Age:{(int)(DateTime.UtcNow -
latestTopTrackDate).TotalDays / 365}.");
        }
    }
}

private bool CheckArtistMeetsPopularityMinimum(int popularity, DateTime latestTopTrackDate)

```

```

{
    TimeSpan timeSinceRelease = DateTime.UtcNow - latestTopTrackDate;
    int yearsSinceRelease = (int)(timeSinceRelease.TotalDays / 365); // finds the age in years of
the artists last most popular song

    /*
    determines minimum popularity based on the date of the artists last popular song
    */

    int minPopularity = yearsSinceRelease switch
    {
        < 1 => 50,
        <= 5 => 55,
        <= 10 => 60,
        _ => 65
    };

    return popularity >= minPopularity;
}

private bool CheckLastFmAndSpotifyTopTracksMatch(List<string> spotifyTopTracks, List<string>
lastFmTopTracks)
{
    if (spotifyTopTracks.Any(lastFmTopTracks.Contains))
    {
        return true;
    }
    return false;
}
}

```

ArtistNetworkExpander

```

using System.Net;
using System.Text.Json;
using DatabaseServices.Interfaces;
using ExternalWebServices.Interfaces;
using GlobalTypes;

namespace expander
{
    public class ArtistNetworkExpander
    {
        private readonly INetworkExpanderDatabaseService _networkExpanderDatabaseService;
        private readonly IArtistVerificationService _artistVerificationService;
        private readonly ISpotifyExpanderService _spotifyExpanderService;
        private readonly ILastFmApiService _lastFmApiService;

        public ArtistNetworkExpander(INetworkExpanderDatabaseService networkExpanderDatabaseService,
            ILastFmApiService lastFmApiService, ISpotifyExpanderService spotifyExpanderService,
            IArtistVerificationService artistVerificationService)
        {
            _lastFmApiService = lastFmApiService;
            _spotifyExpanderService = spotifyExpanderService;
            _networkExpanderDatabaseService = networkExpanderDatabaseService;
            _artistVerificationService = artistVerificationService;
        }

        public async Task SearchForArtists(ArtistNode startingArtistNode)
        {
            Queue<ArtistNode> queue;
            try
            {
                queue = _networkExpanderDatabaseService.GetSearchQueue();
            }
            catch (NullReferenceException ex) // if the queue is empty
            {
                Console.WriteLine(ex.Message);
                queue = new Queue<ArtistNode>();
                queue.Enqueue(startingArtistNode); // enqueues starting artist (for me drake)
                _networkExpanderDatabaseService.AddArtistToDb(startingArtistNode); // adds starting
artist to database
            }
        }
    }
}

```

```

        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
            throw new Exception(ex.Message);
        }
        while (queue.Count > 0)
        {
            int queueLength = queue.Count;
            for (int i = 0; i < queueLength; i++)
            {
                Dictionary<ArtistNode, double> similarArtists = new Dictionary<ArtistNode,
double>(); // dictionary key artistNode, value strength of connection
                ArtistNode currentArtist = queue.Dequeue();

                try
                {
                    similarArtists = await FindSimilarArtists(currentArtist);
                }
                catch (HttpRequestException ex)
                {
                    Console.WriteLine(ex.Message);
                    if (ex.Message.Contains("429")) // spotify timeout code
                    {
                        Console.WriteLine("Timed out, Run again in 36-48 hrs");
                    }

                    break;
                }
                catch (Exception ex)
                {
                    Console.WriteLine(ex.Message);
                    continue;
                }

                _networkExpanderDatabaseService.UpdateIsExpanded(currentArtist.SpotifyId); //
changes the isExpanded column from 0 to 1

                foreach (KeyValuePair<ArtistNode, double> similarArtist in similarArtists)
                {
                    Console.WriteLine($"FOUND: {similarArtist.Key.SpotifyId} -
{similarArtist.Key.Name}");
                    if
(!_networkExpanderDatabaseService.IsArtistInDbById(similarArtist.Key.SpotifyId))
                    {
                        queue.Enqueue(similarArtist.Key); // enqueue foudn artist to serach queue

                        // determines whether the artist and its connection is able to be added to the
database or not
                        _networkExpanderDatabaseService.AddArtistAndConnectionToDb(currentArtist,
similarArtist.Key, similarArtist.Value);
                    }
                }
            }
        }

        private async Task<Dictionary<ArtistNode, double>> FindSimilarArtists(ArtistNode
startingArtistNode)
        {
            Dictionary<ArtistNode, double> similarArtists = new Dictionary<ArtistNode, double>();

            try
            {
                Dictionary<string, double> lastFmSimilarArtists = await
_lastFmApiService.GetSimilarArtists(startingArtistNode.Name);

                foreach (var similarArtist in lastFmSimilarArtists)
                {
                    string similarArtistName = similarArtist.Key;
                    double similarArtistWeight = similarArtist.Value;

                    try
                    {

```

```

        List<string> lastFmTopTracks = await
_lastFmApiService.GetTopTracks(similarArtistName);

        KeyValuePair<(string, string), int> spotifyArtistDetails = await
_spotifyExpanderService.GetArtistDetails(similarArtistName);

        string spotifyId = spotifyArtistDetails.Key.Item1;
        string name = spotifyArtistDetails.Key.Item2;
        int popularity = spotifyArtistDetails.Value;

        Dictionary<string, DateTime> spotifyTopTracks = await
_spotifyExpanderService.GetTopTracks(spotifyId);

        try
        {
            _artistVerificationService.VerifyArtist(spotifyTopTracks, lastFmTopTracks,
popularity);
        }
        catch (Exception ex) // throws exception if the top tracks dont match or the
popularity is too low
        {
            Console.WriteLine(ex.Message);
            continue;
        }

        ArtistNode foundArtist = new ArtistNode(name, spotifyId);
        similarArtists[foundArtist] = similarArtistWeight;
    }
    /*
    Error handling for when running:
    - Yellow means there was an issue with spotify
    - Red means there was an issue with Last.fm
    the only fatal error is HttpRequestException as that indicates broader issue which
needs to be dealt with
    */
    catch (HttpRequestException ex)
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"HTTP request failed for artist {similarArtistName}:
{ex.Message}");
        Console.ResetColor();
        throw new HttpRequestException(ex.Message);
    }
    catch (JsonException ex)
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"JSON parsing failed for artist {similarArtistName}:
{ex.Message}");
        Console.ResetColor();
    }
    catch (InvalidOperationException ex)
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"Invalid operation for artist {similarArtistName}:
{ex.Message}");
        Console.ResetColor();
    }
    catch (FormatException ex)
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"Invalid date format for artist {similarArtistName}:
{ex.Message}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"Unexpected error processing artist {similarArtistName}:
{ex.Message}");
        Console.ResetColor();
    }
}
}
catch (HttpRequestException ex)

```

```

        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"Failed to fetch similar artists from Last.fm: {ex.Message}");
            Console.ResetColor();
            throw new HttpRequestException(ex.Message);
        }
        catch (JsonException ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"Failed to parse Last.fm API response: {ex.Message}");
            Console.ResetColor();
        }
        catch (Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"Unexpected error in FindSimilarArtists: {ex.Message}");
            Console.ResetColor();
        }

        return similarArtists;
    }
}

```

Web Server

Interfaces

```

using GlobalTypes;

namespace webserver;

public interface IArtistNetwork
{
    List<ArtistNode> FindPathWithDijkstras(ArtistNode startArtistNode, ArtistNode endArtistNode);
    void DisplayAllConnections();
}

```

```

using GlobalTypes;

namespace DatabaseServices.Interfaces;

public interface IArtistNetworkDatabaseService
{
    Dictionary<ArtistNode, Dictionary<ArtistNode, double>> GetNetwork();
}

```

ArtistNetworkDatabaseService

```

using DatabaseServices.Interfaces;
using GlobalTypes;

namespace webserver;

public class ArtistNetworkDatabaseService : IArtistNetworkDatabaseService
{
    private readonly IDatabaseRepository _databaseRepository;

    public ArtistNetworkDatabaseService(IDatabaseRepository databaseRepository)
    {
        _databaseRepository = databaseRepository;
        databaseRepository.InitialiseDatabase();
    }

    public Dictionary<ArtistNode, Dictionary<ArtistNode, double>> GetNetwork()
    {
        return _databaseRepository.GetNetwork();
    }
}

```

```

    }
}

```

ArtistNetwork

```

using GlobalTypes;

namespace webserver
{
    public class ArtistNetwork : IArtistNetwork
    {
        private readonly Dictionary<ArtistNode, Dictionary<ArtistNode, double>> _adjacencyMatrix; //
        represents the network adjacency matrix
        public ArtistNetwork(IArtistNetworkDatabaseService artistNetworkDatabaseService)
        {
            _adjacencyMatrix = new Dictionary<ArtistNode, Dictionary<ArtistNode, double>>();
            _adjacencyMatrix = artistNetworkDatabaseService.GetNetwork();
            Console.WriteLine("Finished loading the artist network.");
        }

        public void DisplayAllConnections() // for backend monitoring
        {
            foreach (var kvp in _adjacencyMatrix)
            {
                ArtistNode artist = kvp.Key;
                foreach (var connection in kvp.Value)
                {
                    Console.WriteLine($"{artist.Name} -> {connection.Key.Name} (Weight:
{connection.Value})");
                }
            }
        }

        // DIJKSTRAS ALGORITHM
        public List<ArtistNode> FindPathWithDijkstras(ArtistNode startArtistNode, ArtistNode
endArtistNode)
        {
            PriorityQueue<ArtistNode, double> priorityQueue = new PriorityQueue<ArtistNode, double>();
            Dictionary<ArtistNode, double> distances = new Dictionary<ArtistNode, double>();
            Dictionary<ArtistNode, ArtistNode> previous = new Dictionary<ArtistNode, ArtistNode>();
            List<ArtistNode> Path = new List<ArtistNode>();

            foreach (ArtistNode artist in _adjacencyMatrix.Keys)
            {
                distances[artist] = double.MaxValue;
                previous[artist] = null;
            }
            distances[startArtistNode] = 0;

            priorityQueue.Enqueue(startArtistNode, 0);

            while (priorityQueue.Count > 0)
            {
                ArtistNode currentArtist = priorityQueue.Dequeue();
                double currentDistance = distances[currentArtist];

                if (currentArtist.SpotifyId == endArtistNode.SpotifyId)
                    break;

                foreach (KeyValuePair<ArtistNode, double> connection in
_adjacencyMatrix[currentArtist])
                {
                    ArtistNode neighbour = connection.Key;
                    double newDist = currentDistance + connection.Value;

                    if (newDist < distances[connection.Key])
                    {
                        distances[neighbour] = newDist;
                        previous[neighbour] = currentArtist;
                        priorityQueue.Enqueue(neighbour, newDist);
                    }
                }
            }

            return Path;
        }
    }
}

```

```

    }
    }
}

ArtistNode pathArtistNode = endArtistNode;
while (pathArtistNode != null)
{
    Path.Insert(0, pathArtistNode);
    pathArtistNode = previous[pathArtistNode];
}
return Path;
}
}
}

```

ArtistNetworkDatabaseService

```

using DatabaseServices.Interfaces;
using GlobalTypes;

namespace webserver;

public class ArtistNetworkDatabaseService : IArtistNetworkDatabaseService
{
    private readonly IDatabaseRepository _databaseRepository;

    public ArtistNetworkDatabaseService(IDatabaseRepository databaseRepository)
    {
        _databaseRepository = databaseRepository;
        databaseRepository.InitialiseDatabase();
    }

    public Dictionary<ArtistNode, Dictionary<ArtistNode, double>> GetNetwork()
    {
        return _databaseRepository.GetNetwork();
    }
}

```

API

```

using DatabaseServices.Interfaces;
using ExternalWebServices.Interfaces;
using GlobalTypes;
using Microsoft.AspNetCore.Mvc;

namespace webserver
{
    public class PlaylistRequest // data transfer object for playlist request as it is a post
    {
        public string Code { get; set; }
        public List<string> Path { get; set; }
        public string State { get; set; }
    }

    [ApiController]
    [Route("api")]
    public class Api(
        IArtistNetwork artistNetwork,

```

```

IWebServerDatabaseService webServerDatabaseService,
ISpotifyPkceCodeAuthorizer spotifyPkceCodeAuthorizer,
IGetPlaylistSongsService getPlaylistSongsService,
ICreatePlaylistService createPlaylistService)
: ControllerBase
{

    [HttpGet("validate-artist")]
    public IActionResult ValidateArtist([FromQuery] string artistName)
    {
        try
        {
            bool exists = webServerDatabaseService.IsArtistInDbByName(artistName);
            if (exists)
            {
                // this gets the verified name of the artist to correct wrong casing
                string artistId = webServerDatabaseService.GetIdFromName(artistName);
                artistName = webServerDatabaseService.GetNameFromId(artistId);
                return Ok(new
                {
                    ArtistExist = true,
                    ArtistName = artistName
                });
            }
            return Ok(new
            {
                ArtistExist = true,
            });
        }
        catch (Exception ex)
        {
            return StatusCode(500, new
            {
                Error = $"An error occurred while checking the artist. {ex.Message}"
            });
        }
    }

    [HttpGet("find-path")]
    public IActionResult FindPath([FromQuery] string artist1, [FromQuery] string artist2)
    {
        try
        {
            ArtistNode artist1Node = new ArtistNode(artist1,
webServerDatabaseService.GetIdFromName(artist1));
            ArtistNode artist2Node = new ArtistNode(artist2,
webServerDatabaseService.GetIdFromName(artist2));

            // uses artist network dijkstras algorithm to find the shortest path
            List<ArtistNode> path = artistNetwork.FindPathWithDijkstras(artist1Node, artist2Node);
            Console.WriteLine($"TRYING TO TRAVEL BETWEEN {artist1Node.Name} and
{artist2Node.Name}");
            foreach (string id in path.Select(a => a.SpotifyId).ToList())
            {
                Console.WriteLine(id);
            }

            return Ok(new
            {
                PathId = path.Select(a => a.SpotifyId).ToList(),
                PathName = path.Select(a => a.Name).ToList(),
            });
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { Error = "An error occurred while finding the path." });
        }
    }

    [HttpGet("authenticate-user")]
    public async Task<IActionResult> Authenticate()
    {

```

```

        try
        {
            // retrusn the authurl
            (string authUrl, string state) = await
spotifyPkceCodeAuthorizer.GetAuthorizationUrl();
            return Ok(new { AuthUrl = authUrl, State = state });
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { Error = $"Authentication initialization failed,
{ex.Message}" });
        }
    }

[HttpPost("create-playlist")]
public async Task<IActionResult> CreatePlaylist([FromBody] PlaylistRequest request)
{
    try
    {
        // if the path is empty
        if (!request.Path.Any())
        {
            Console.WriteLine("No path provided");
            return BadRequest(new { Error = "Path is empty" });
        }

        List<ArtistNode> artists = new();

        foreach (string spotifyId in request.Path)
        {
            string artistName = webServerDatabaseService.GetNameFromId(spotifyId);
            artists.Add(new ArtistNode(artistName, spotifyId));
        }

        List<string> songIds = await getPlaylistSongsService.GetPlaylistSongIds(artists);
        Console.WriteLine("Creating playlist");
        foreach (string songId in songIds)
            Console.WriteLine(songId);

        // gets the access token only when required
        string accessToken = await spotifyPkceCodeAuthorizer.ExchangeCode(request.Code,
request.State);
        string playlistLink = await createPlaylistService.CreatePlaylist(songIds,
artists.First().Name,
            artists.Last().Name, accessToken);

        return Ok(new { Message = "Playlist created successfully.", PlaylistLink =
playlistLink });
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
        return StatusCode(500, new { Error = "An error occurred while creating the playlist.",
Details = ex.Message });
    }
}

[HttpGet("report-issue")]
public IActionResult ReportIssue([FromQuery] string issue)
{
    try
    {
        if (string.IsNullOrEmpty(issue))
        {
            return BadRequest("Artist name cannot be empty.");
        }

        string path =
"/Users/jonathanlyria/RiderProjects/NettspendToSautiSol/NettspendToSautiSol/webserver/reportissue.txt"
;

```

```

        if (!System.IO.File.Exists(path))
        {
            System.IO.File.Create(path).Dispose();
        }

        System.IO.File.AppendAllText(path, $"{issue}{Environment.NewLine}");

        return Ok($"Issue reported: {issue}");
    }
    catch (Exception ex)
    {
        return StatusCode(500, $"Internal server error: {ex.Message}");
    }
}

[HttpGet("get-all-artists")]
public IActionResult GetAllArtists()
{
    try
    {
        var artists = webServerDatabaseService.GetAllArtistNodes();
        return Ok(new {
            Artists = artists.Select(a => new {
                name = a.Name,
                spotifyId = a.SpotifyId
            }).ToList() // returns the name and spotifyId for the getallartist web page (Link
// can be created with just the artist id)
        });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new {
            Error = "Failed to retrieve artists",
            Details = ex.Message
        });
    }
}
}
}

```

Program.cs (entry point 2)

```

using DatabaseServices;
using DatabaseServices.Interfaces;
using ExternalWebServices;
using ExternalWebServices.Interfaces;

namespace webserver
{
    public class Program
    {
        public static async Task Main(string[] args)
        {
            string databasePath = args[0];
            string spotifyClientId = args[1];
            string spotifyClientSecret = args[2];
            string redirectUri = args[3];

            WebApplicationBuilder builder = WebApplication.CreateBuilder();

            builder.Services.AddSingleton<HttpClient>(); // singleton = only one for program lifetime.
// prevents socket exhaustion

            ArtistRepository artistRepository = new ArtistRepository(databasePath);
            DatabaseRepository databaseRepository = new DatabaseRepository(databasePath);
            ArtistNetworkDatabaseService artistNetworkDatabaseService = new
            ArtistNetworkDatabaseService(databaseRepository);

            builder.Services.AddSingleton<IWebServerDatabaseService>(sp =>
                new WebServerDatabaseService(artistRepository, databaseRepository));

```

```

builder.Services.AddSingleton<ISpotifyClientCredentialAuthorizer>(sp =>
    new SpotifyClientCredentialAuthorizer(
        spotifyClientId,
        spotifyClientSecret,
        sp.GetRequiredService<HttpClient>()));

builder.Services.AddSingleton<ISpotifyPkceCodeAuthorizer>(sp =>
    new SpotifyPkceCodeAuthorizer(
        sp.GetRequiredService<HttpClient>(), spotifyClientId,
        redirectUri));

builder.Services.AddSingleton<IGetPlaylistSongsService, GetPlaylistSongsService>();
builder.Services.AddSingleton<ICreatePlaylistService, CreatePlaylistService>();

builder.Services.AddSingleton<IArtistNetwork>(sp =>
{
    var artistNetwork = new ArtistNetwork(artistNetworkDatabaseService);
    return artistNetwork;
});

builder.Services.AddControllers();

builder.Services.AddCors(options => // allows my local website to access the api
{
    options.AddPolicy("AllowSpecificOrigin", policy =>
    {
        policy.WithOrigins("http://localhost", "http://127.0.0.1:8080")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

WebApplication app = builder.Build();

Console.WriteLine("Starting the application...");

IArtistNetwork artistNetwork = app.Services.GetRequiredService<IArtistNetwork>();
Console.WriteLine("Waiting for the artist network to finish loading...");
await Task.Run(() => artistNetwork.DisplayAllConnections()); // makes sure the artist
network is loaded from the database
Console.WriteLine("Artist network loaded!");

app.UseRouting();
app.UseCors("AllowSpecificOrigin");
app.MapControllers();

await app.RunAsync();
}
}
}

```

WebServerDatabaseService

```

using DatabaseServices.Interfaces;
using GlobalTypes;

namespace webserver;

public class WebServerDatabaseService : IWebServerDatabaseService
{
    private readonly IArtistRepository _artistRepository;
    private readonly IDatabaseRepository _databaseRepository;

    public WebServerDatabaseService(IArtistRepository artistRepository, IDatabaseRepository
databaseRepository)
    {
        _artistRepository = artistRepository;
        _databaseRepository = databaseRepository;
        _databaseRepository.InitialiseDatabase();
    }
}

```

```

public string GetIdFromName(string name)
{
    return _artistRepository.GetIdFromName(name);
}

public string GetNameFromId(string spotifyId)
{
    return _artistRepository.GetNameFromId(spotifyId);
}

public bool IsArtistInDbByName(string name)
{
    return _artistRepository.IsArtistInDbByName(name);
}

public List<ArtistNode> GetAllArtistNodes()
{
    return _artistRepository.GetAllArtistNodes();
}
}

```

Website

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>from nettspend to sauti sol</title> <!-- Basic Title which will be changed once user enters
title -->
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div id="navigation">
        <a href="https://github.com/jonathanlyria/NettspendToSautiSol" target="_blank" id="github-
logo">
            
            
        </a>
        <div id="navigation-buttons">
            <a href="#report" id="report-link">Report an Issue</a>
            <a href="viewartists.html" id="viewartists-link">View All Artists</a>
            <a href="https://github.com/jonathanlyria/NettspendToSautiSol/blob/main/README.md"
target="_blank" id="github-link">Information</a>
        </div>
    </div>
    <div id="container">
        <h1 id="header-title">from</h1>
        <div id="output-box"></div>
        <input id="input" type="text" placeholder="Type your command here..." autofocus />
    </div>

    <!-- Report Issue Modal -->
    <div id="report-issue-modal" class="modal hidden">
        <div class="modal-content">
            <h2 class="modal-title">Report an Issue</h2>
            <textarea id="issue-text" placeholder="Describe the issue..."></textarea>
            <div id="issue-error" class="error hidden">Please provide a valid issue description.</div>
            <button id="submit-issue">Submit</button>
            <button class="close-modal">Close</button>
        </div>
    </div>

```

```

<script src="config.js"></script>
<script src="script.js"></script>
<script src="report.js"></script>
</body>
</html>

```

view-artists.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>View Artists</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<div id="navigation">
  <div id="navigation-buttons">
    <a href="index.html">Home</a>
  </div>
</div>

<div id="viewartists-container">
  <div id="viewartists-table"></div>
</div>

<script src="viewartists.js"></script>
</body>
</html>

```

callback.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Spotify Callback</title>
</head>
<body>
<script>
  <!--handles the callback logic of the spotify api. the specific path is specified in spotify-->
  const params = new URLSearchParams(window.location.search);
  window.opener.postMessage({
    code: params.get('code'),
    state: params.get('state')
  }, window.location.origin);
  window.close();
</script>
</body>
</html>

```

script.js

```

// api base url is based in config.js for now it is http://localhost:5048/api, which is where my
server is configured to

const input = document.getElementById('input');
const output = document.getElementById('output-box');
let headerTitle = document.getElementById('header-title');

```

```

function appendOutput(text, className = '') { // this handles adding output into the website
  const newLine = document.createElement('div');
  newLine.textContent = text;
  if (className) newLine.classList.add(className);
  output.appendChild(newLine);
  output.scrollTop = output.scrollHeight; // this means that once it reaches the scroll height, the
  // navigation is still visible
}

async function processFlow() {
  const artist1 = await requestArtistOne();
  const artist2 = await requestArtistTwo(artist1);
  const pathIds = await findPathBetweenArtists(artist1, artist2);
  if (pathIds) {
    await requestAuthentication(pathIds);
  }
}

function requestArtistOne() { // requests first artist
  return new Promise(resolve => {
    appendOutput("Enter the name of the first artist:");
    const handler = async (event) => {
      if (event.key === 'Enter') {
        const inputName = input.value.trim();
        input.value = '';
        appendOutput(`> ${inputName}`);
        input.removeEventListener('keydown', handler);

        const { exists, artistName } = await checkArtistExists(inputName);
        if (exists) {
          document.title = `from ${artistName}`; // change the tab name
          headerTitle.textContent = `from ${artistName}`; // change the title name
          resolve(artistName);
        } else {
          appendOutput("Artist not found. Please try again.", "error");
          const artist1 = await requestArtistOne();
          resolve(artist1);
        }
      }
    };
    input.addEventListener('keydown', handler);
  });
}

function requestArtistTwo(artist1) { // requests second artist
  return new Promise(resolve => {
    appendOutput("Enter the name of the second artist:");
    const handler = async (event) => {
      if (event.key === 'Enter') {
        const inputName = input.value.trim();
        input.value = '';
        appendOutput(`> ${inputName}`);
        input.removeEventListener('keydown', handler);

        const { exists, artistName } = await checkArtistExists(inputName);
        if (exists) {
          document.title = `from ${artist1} to ${artistName}`; // change the tab name
          headerTitle.textContent = `from ${artist1} to ${artistName}`; // change the title
          // name
          resolve(artistName);
        } else {
          appendOutput("Artist not found. Please try again.", "error");
          const artist2 = await requestArtistTwo(artist1);
          resolve(artist2);
        }
      }
    };
    input.addEventListener('keydown', handler);
  });
}

async function checkArtistExists(artistName) { // checks whetehr the artist exists
  try {

```

```

    const response = await fetch(`${API_BASE_URL}/validate-artist?artistName=${encodeURIComponent(artistName)}`);
    const data = await response.json();

    return { exists: data.artistExist, artistName: data.artistName };
  } catch (error) {
    return { exists: false, artistName: null };
  }
}

async function findPathBetweenArtists(artist1, artist2) { // finds the path between two artist
  try {
    appendOutput(`Finding path between "${artist1}" and "${artist2}"...`);
    const response = await fetch(`${API_BASE_URL}/find-path?artist1=${encodeURIComponent(artist1)}&artist2=${encodeURIComponent(artist2)}`);
    const data = await response.json();

    if (response.ok) {
      const pathNames = data.pathName.join(" -> ");
      appendOutput(`Path found: ${pathNames}`, "success");
      return data.pathId;
    } else {
      appendOutput(`Error: ${data.Error}`, "error");
      return null;
    }
  } catch (error) {
    appendOutput(`An error occurred: ${error.message}`, "error");
    return null;
  }
}

function requestAuthentication(pathIds) {
  return new Promise(resolve => {
    appendOutput("Press Enter to sign into Spotify.");
    const handler = async (event) => {
      if (event.key === 'Enter') {
        input.removeEventListener('keydown', handler);
        appendOutput("> Authenticating...");
        await startAuthentication(pathIds);
        resolve();
      }
    };
    input.addEventListener('keydown', handler);
  });
}

async function startAuthentication(pathIds) {
  try {
    const response = await fetch(`${API_BASE_URL}/authenticate-user`);
    const data = await response.json();
    const authState = data.state;

    // open window for user authentication
    const authWindow = window.open(data.authUrl, 'Spotify Auth', 'width=600,height=800');

    // Create a promise to handle the callback
    const authPromise = new Promise(resolve => {
      const callbackHandler = (event) => {
        // verify event origin = window origin
        if (event.origin !== window.location.origin) {
          return;
        }

        const { code, state } = event.data;

        // if the user state and received states are different
        if (state !== authState) {
          appendOutput("Authentication failed: State mismatch", "error");
          return;
        }

        // Remove the event listener
        window.removeEventListener('message', callbackHandler);
        appendOutput("Authentication successful.", "success");
      }
    });
  }
}

```

```

        resolve({ code, state });
    });

    // Listen to messages
    window.addEventListener('message', callbackHandler);
});

appendOutput("Waiting for Spotify authentication...");

// Wait for authentication to complete
const authResult = await authPromise;
await requestPlaylistCreation(pathIds, authResult.code, authResult.state);

} catch (error) {
    appendOutput(`Authentication failed: ${error.message}`, "error");
    console.error('Authentication error:', error);
}
}

function requestPlaylistCreation(pathIds, authCode, authState) {
    return new Promise(resolve => {
        appendOutput("Press Enter to create the playlist.");
        const handler = async (event) => {
            if (event.key === 'Enter') {
                input.removeEventListener('keydown', handler);
                await createPlaylist(pathIds, authCode, authState);
                resolve();
            }
        };
        input.addEventListener('keydown', handler);
    });
}

async function createPlaylist(pathIds, authCode, authState) {
    try {
        appendOutput("Creating playlist, wait for 7ish seconds...");
        const requestBody = { // creates request body for post request
            path: pathIds,
            code: authCode,
            state: authState
        };

        const response = await fetch(`${API_BASE_URL}/create-playlist`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(requestBody),
        });
        const data = await response.json();

        if (response.ok) {
            appendOutput(`Playlist created successfully! Link: ${data.playlistLink}`, "success");
            window.open(data.playlistLink, '_blank');
        } else {
            appendOutput(`Error: ${data.error}`, "error");
        }
    } catch (error) {
        appendOutput(`An error occurred: ${error.message}`, "error");
    }
}

processFlow();

```

report.js

```

// watched modal tutorial on youtube
document.getElementById('report-link').addEventListener('click', () => {
    document.getElementById('report-issue-modal').classList.remove('hidden');
});

document.getElementById('submit-issue').addEventListener('click', async () => {

```

```

const issueText = document.getElementById('issue-text').value.trim();
const errorElement = document.getElementById('issue-error');

if (!issueText) {
  errorElement.classList.remove('hidden');
  return;
}
errorElement.classList.add('hidden');

try {
  const response = await fetch(`${API_BASE_URL}/report-issue?issue=${encodeURIComponent(issueText)}`);
  const message = await response.text();

  if (response.ok) {
    alert(message);
  } else {
    alert(`Error: ${message}`);
  }
} catch (error) {
  alert(`Error: ${error.message}`);
}
});

document.querySelectorAll('.close-modal').forEach(button => {
  button.addEventListener('click', () => {
    document.getElementById('report-issue-modal').classList.add('hidden');
    document.getElementById('request-artist-modal').classList.add('hidden');
  });
});

```

styles.css

```

body {
  background-color: #121212;
  color: white;
  font-family: "Gotham-Medium", monospace;
  font-size: clamp(1.5rem, 2vw, 2rem);
}

@font-face {
  font-family: 'Gotham';
  src: url('Gotham-Medium.woff') format('woff');
  font-weight: normal;
  font-style: normal;
}

/*navigation*/

#navigation{
  display: flex;
  align-items: center;
  justify-content: space-between;
  width: 100%;
  border-radius: 8px;
}

#github-logo {
  position: relative;
  display: inline-block;
  margin-left: 1%;
  width: 50px;
  height: 50px;
}

#github-logo img {
  position: absolute;
  top: 0;
  left: 0;
  transition: opacity 0.3s ease;
}

```

```

#github-logo .hover-logo {
  opacity: 1;
}

#github-logo:hover .hover-logo {
  opacity: 0;
}

#github-logo:hover .default-logo {
  opacity: 1;
}

#navigation-buttons {
  display: flex;
  width: 100%;
  padding-right: 1%;
  justify-content: flex-end;
}

#navigation-buttons a {
  color: #1DB954;
  text-decoration: none;
  font-family: "Gotham-Medium", monospace;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  transition: color 0.3s;
  padding: 1%;
  border-radius: 8px;
}

#navigation-buttons a:hover {
  color: white;
  border-radius: 8px;
}

#navigation-buttons a:focus {
  outline: none;
}

/*index.html main section*/

#header-title {
  position: sticky;
  text-align: center;
  margin: 40px 0;
  font-size: 2em;
  font-family: "Gotham-Medium", monospace;
  color: white;
  border: 1px;
}

#container {
  display: flex;
  background-color: #121212;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 60vh;
  width: 100%;
}

#output-box {
  width: 100%;
  max-width: 800px;
  min-height: 20vh;
  color: white;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  overflow-y: auto;
  position: relative;
}

#input{
  color: #1DB954;
  background-color: #121212;

```

```

outline: none;
width: 100%;
max-width: 800px;
font-family: "Gotham-Medium", monospace;
font-size: clamp(1rem, 1.5vw, 1.5rem);
border: 1px;
padding-top: 20px;
}

/*report issue modal*/
.modal {
  display: flex;
  justify-content: center;
  align-items: center;
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  z-index: 1000;
}

.modal.hidden {
  display: none;
}

.modal-content {
  background-color: #212121;
  padding: 1%;
  border-radius: 10px;
  text-align: left;
  width: 400px;
}

.modal-title {
  color: white;
  font-size: 1.5rem;
  padding: 1%;
}

textarea{
  width: 95%;
  background-color: #212121;
  color: white;
  padding: 1%;
  font-family: "Gotham-Medium", monospace;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  resize: none;
  border: 1px;
  text-align: left;
  border-radius: 1px;
}

button {
  background: #212121;
  color: #1DB954;
  padding: 1%;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  cursor: pointer;
  transition: color 0.3s;
  font-family: "Gotham-Medium", monospace;
  border: 1px;
}

button:hover {
  color: white;
}

/*style classes*/

.error {
  color: red;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  margin-top: 5px;
}

```

```

.hidden {
  display: none;
}

.console-item {
  padding: 15px 0;
  border-bottom: 1px solid #1DB95433;
  display: flex;
  justify-content: space-between;
  align-items: center;
  animation: terminalEntry 0.5s ease-in-out;
}

.data-field {
  color: #fff;
  font-family: "Gotham-Medium", monospace;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
}

.terminal-link {
  color: #1DB954;
  text-decoration: none;
  font-family: "Gotham-Medium", monospace;
  padding: 5px 10px;
  border: 1px solid #1DB95433;
  transition: all 0.3s ease;
}

.terminal-link:hover {
  background-color: #1DB954;
  color: black;
  border-color: #1DB954;
}

/*viewartists.html*/

#viewartists-container {
  display: flex;
  background-color: #121212;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 90%;
  width: 100%;
}

#viewartists-table {
  width: 100%;
  max-width: 800px;
  min-height: 20vh;
  color: white;
  font-size: clamp(1rem, 1.5vw, 1.5rem);
  overflow-y: auto;
  position: relative;
}

.artist-name {
  padding: 15px 0;
  border-bottom: 1px solid rgba(29, 185, 84, 0.1);
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.artist-name span {
  flex-grow: 1;
  margin-right: 20px;
}

.spotify-link {
  color: #1DB954;
  text-decoration: none;
}

```

```
padding: 5px 10px;  
border-radius: 8px;  
transition: background-color 0.3s, color 0.3s;  
}  
  
.spotify-link:hover {  
  color: white;  
}
```

Testing

Overview

To test my solution, I have split the testing into white box and black box testing. Black box testing has been split into two sections: a video for user flow and a video to show the expander working at a slow pace.

Black Box Testing

Video 1 controls testing of all the user input. I incorporate normal, erroneous and boundary tests to determine whether my code is robust and fully satisfies the criteria

Sub-Success Criteria	Input data type	Expected Result	Test (if not obvious)	Time	Pass/Fail
The website prompts the user to enter the first artist.	Normal	A prompt displayed to ask the user to enter the first artist		0:00	Pass
User is prompted repeatedly until a valid artist is entered.	Normal	Next prompt is displayed		0:05	Pass
	Erroneous	Red error message saying this artist is not in the database	Testing with null data	1:50	Pass
			Test with artist name not in database	2:01-2:06	Pass
			Testing with weird characters	2:02	Pass
	Boundary	Red error message saying this artist is not in the database	Testing with artist written in wrong case	0:54	Pass
The process is then repeated for the second artist.	Normal	Next prompt is displayed		0:05	Pass
	Erroneous	Red error message saying this artist is not in the database	Testing with null data	2:18	Pass
			Testing with artist not in database		Pass
			Testing with weird characters	2:18	Pass
	Boundary	Automatically changes to the correct case for the artist	Testing with artist written in wrong case	1:05	Pass

Website title starts with "from"	Normal	Website title changes starts with from		0:00	Pass
Website title changes to from (first artist) to once a (first artist) has been successfully entered	Normal	Website title changes from Drake to, and then once Osamason is entered it changes to from Drake to Osamason		0:00-0:12	Pass
	Boundary	Website title changes to name with the correct case, from Nettpsend to Cleo Solk	Entered artists with the wrong case	1:00-1:10	Pass
Program displays artists on the shortest path between two artists with a ->	Normal	List of artists connected with arrows	Drake to Osamason	0:00	Pass
			Nettpsend to Cleo Sol	1:09	Pass
	Boundary	Artist is displayed	Drake -> Drake	2:31	Pass
The website must display a prompt to press enter to login to Spotify.	Normal	Prompt to press enter is displayed		0:12	Pass
A new window is opened for the user to login to Spotify.	Normal	New window opens		0:15	Pass
	Boundary	New window opens for both windows	Test with two simultaneous users		Pass
Once the user logs in, the window closes.	Normal	Window closes, next prompt is displayed		0:18	Pass
The website prompts the user to press enter to create the playlist.	Normal	Next prompt is displayed saying creating playlist		0:18	Pass
Once the playlist is created, a new window opens to display the playlist.	Normal	Playlist is created		0:28	Pass
	Boundary	Playlist is created for both users	Try to simultaneously create a playlist	3:13	Pass
The playlist link is also displayed for the user to copy.	Normal	Playlist link is displayed		0:32	Pass

Design a page in the website where users can view all artists in the network	Normal	Page to view all artists is displayed		1:33-1:42	Pass
Data is displayed in a table format with the columns: Artist Name, Similar Artists, and View Artist On Spotify.	Normal	Page is there to see all artists			Pass
Report issue link must open report issue modal	Normal	Report issue modal opens		4:03	Pass
User must be able to report issues in the report issue modal to be logged in text file	Normal	Allows user to report issue and it shows in my text file	Try report issue	4:04	Pass
	Erroneous	Display error: cannot report issue	Null entry	4:10	Pass
View all artists must open view all artists page	Normal	Shows list of artists in table		1:49	Pass
Website follows user design requirements (fonts, colours etc)	Normal	Website should have the fonts and colours as specified in objectives		Throug out video	Pass

Video 2 shows the network expander working to expand the network, and showing the network being saved in the database. As this is a backend process, this is just to show that the network expansion works. The specific objectives being met are shown in the White Box Testing section. I have slowed it down by using a Time.Sleep() method not present in my actual code

White Box Testing

To test objectives whose code is not immediately obvious in the video due to being backend processes, I have employed white box testing to test whether I meet the objectives I made out to start the project.

Objective	Code Snippet Evidence	Explanation//Expected//Result (Pass/Fail)
Program must use last.fm data to get a list of verified similar	<pre>var similarArtists = new Dictionary<string, double>(); var similarArtistParameters = new Dictionary<string, string> { { "method", "artist.getSimilar" }, { "artist", artistName }, { "api_key", _apiKey }, { "format", "json" }, }</pre>	Explanation: This code queries the last fm database with the

artists and their similarity score to the given artist	<pre> { "limit", "5" } }; string requestUrl = BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/", similarArtistParameters); try { var response = await _httpClient.GetAsync(requestUrl); response.EnsureSuccessStatusCode(); string responseContent = await response.Content.ReadAsStringAsync(); if (string.IsNullOrEmpty(responseContent)) throw new InvalidOperationException("Response content is empty."); using var document = JsonDocument.Parse(responseContent); if (!DoSimilarArtistsExist(document)) throw new InvalidOperationException("Similar artists data is missing."); foreach (var artistData in document.RootElement.GetProperty("similarartists") .GetProperty("artist").EnumerateArray()) { if (!DoesLastFmArtistHaveNameAndMatch(artistData)) throw new InvalidOperationException("Artist does not have name and match."); string foundArtist = artistData.GetProperty("name").GetString(); double match = double.Parse(artistData.GetProperty("match").GetString()); if (!foundArtist.Contains('&')) similarArtists[foundArtist] = match; } return similarArtists; } </pre>	given artist name and gets the name and match of the 5 most similar artist Expected: Code which can query last.fm using their api to get the name and weight of connection to the started artist Result: Pass
Program must use last.fm data to get the top tracks of a given artist	<pre> var topTracks = new List<string>(); var topTrackParameters = new Dictionary<string, string> { { "method", "artist.getTopTracks" }, { "artist", artistName }, { "api_key", _apiKey }, { "format", "json" }, }; string requestUrl = BuildUrlWithParams("https://ws.audioscrobbler.com/2.0/", topTrackParameters); try { var response = await _httpClient.GetAsync(requestUrl); response.EnsureSuccessStatusCode(); string responseContent = await response.Content.ReadAsStringAsync(); if (string.IsNullOrEmpty(responseContent)) throw new InvalidOperationException("Response content is empty."); using var document = JsonDocument.Parse(responseContent); if (!document.RootElement.TryGetProperty("toptracks", out var toptracks) !toptracks.TryGetProperty("track", out var tracksArray)) throw new InvalidOperationException("Artist has no top tracks."); foreach (var track in tracksArray.EnumerateArray()) { string topTrack = track.GetProperty("name").GetString() ?? "Unknown Track"; if (topTrack == "Unknown Track") throw new InvalidOperationException("Top track name is unknown."); topTrack = Regex.Replace(topTrack, @"\" + @"[\[\].*?[\(\)]\" + @"\s*", "").Trim(); topTracks.Add(topTrack.ToLower()); } return topTracks; } </pre>	Explanation: The code queries the last fm api with the given artist name and gets the top tracks Expected: Code which can get the top tracks of a given artist using the last.fm api Status: Pass

Program must use spotify data to get the spotifyId of a given artist	<pre> string url = \$"https://api.spotify.com/v1/search?q=artist:{Uri.EscapeDataString(artistName)}\&type=artist&limit=1"; var response = await _httpClient.GetAsync(url); response.EnsureSuccessStatusCode(); string responseContent = await response.Content.ReadAsStringAsync(); if (string.IsNullOrEmpty(responseContent)) throw new InvalidOperationException("Spotify API returned an empty response."); using var document = JsonDocument.Parse(responseContent); var artistData = document.RootElement .GetProperty("artists") .GetProperty("items") .EnumerateArray() .FirstOrDefault(); if (artistData.ValueKind == JsonValueKind.Undefined) throw new InvalidOperationException("No artist found for the given name."); string spotifyId = artistData.GetProperty("id").GetString(); string name = artistData.GetProperty("name").GetString(); int popularity = artistData.GetProperty("popularity").GetInt32(); return new KeyValuePair<string, string>, int>((spotifyId, name), popularity); </pre>	Explanation: Uses the spotify API to get the spotifyId and name of a given artist Expected: Code which can get the spotifyId and name of an artist using the spotify Api Status: Pass
Program must use spotify data to get the top tracks of a given artist	<pre> string url = \$"https://api.spotify.com/v1/artists/{spotifyId}/top-tracks"; var response = await _httpClient.GetAsync(url); string responseContent = await response.Content.ReadAsStringAsync(); if (string.IsNullOrEmpty(responseContent)) throw new InvalidOperationException("Spotify API returned an empty response."); var document = JsonDocument.Parse(responseContent); foreach (var topTrack in document.RootElement.GetProperty("tracks").EnumerateArray()) { string topTrackName = topTrack.GetProperty("name").GetString(); topTrackName = Regex.Replace(topTrackName, @"\"s*[\(\[\].*?[\]\]]\"s*", "").Trim(); string releaseDateStr = topTrack.GetProperty("album").GetProperty("release_date").GetString(); if (!Regex.IsMatch(releaseDateStr, @"^\"d{4}\"(-\"d{2}\"(-\"d{2})?)?\$\"")) { throw new FormatException(\$"Invalid date format: {releaseDateStr}"); } string[] parts = releaseDateStr.Split('-'); int year = int.Parse(parts[0]); int month = parts.Length >= 2 ? int.Parse(parts[1]) : 1; int day = parts.Length >= 3 ? int.Parse(parts[2]) : 1; DateTime topTrackDate; try { topTrackDate = new DateTime(year, month, day); } catch (ArgumentOutOfRangeException ex) { throw new FormatException(\$"Invalid date values in: {releaseDateStr}", ex); } topTracks[topTrackName.ToLower()] = topTrackDate; } </pre>	Explanation: Uses the spotify API to get a dictionary of top tracks for an artist as well as the date for the artist Expected: Code uses the spotifyAPI to get the name and date of an artists top tracks Status: Pass
Program must have a way to compare the top tracks from last.fm artist and spotify to check whether there are any matches	<pre> if (spotifyTopTracks.Any(lastFmTopTracks.Contains)) { return true; } return false; </pre>	Explanation: Compares the spotifyTopTracks with the lastFmTopTracks to check whether there is any correlation Expected: Compares spotify top tracks to check whether there are any matches

		Result: Pass
Program must reject any artist whose last popular track was less than a year ago + has a popularity score of <50	<pre> TimeSpan timeSinceRelease = DateTime.UtcNow - latestTopTrackDate; int yearsSinceRelease = (int)(timeSinceRelease.TotalDays / 365); int minPopularity = yearsSinceRelease switch { < 1 => 50, <= 5 => 55, <= 10 => 60, _ => 65 }; return popularity >= minPopularity; </pre>	Explanation: Sets a minimum popularity which is required for the artist to pass the popularity requirement. Then checks whether the artist meets the minimum popularity requirement Expected: Rejects any artist whose popularity is below the specified thresholds Result: Pass
Program must reject any artist whose last popular track was between a year and 5 years ago and has a popularity score of <55		
Program must reject any artist whose last popular track was between 5 years and 10 years ago and has a popularity score of <60		
Program must reject any artist whose last popular track was more than 10 years ago and has a popularity score of <65		
Database must be able to store the spotifyId, name and whether an artist has had their similar artists found	<pre> string createArtistTableQuery = @" CREATE TABLE IF NOT EXISTS Artist (SpotifyId TEXT NOT NULL PRIMARY KEY, ArtistName TEXT NOT NULL, isExpanded BIT NOT NULL);"; using (SqliteCommand cmd = new SqliteCommand(createArtistTableQuery, connection)) { cmd.ExecuteNonQuery(); } connection.Open(); string insertQuery = "INSERT INTO Artist (ArtistName, isExpanded, SpotifyID) VALUES (@artistName, 0, @spotifyId);"; using (SqliteCommand cmd = new SqliteCommand(insertQuery, connection)) { cmd.Parameters.AddWithValue("@artistName", artist.Name); cmd.Parameters.AddWithValue("@spotifyId", artist.SpotifyId); cmd.ExecuteNonQuery(); } </pre>	Explanation: First code snippet from DatabaseRepository, Second code snippet from ArtistRepository, shows that there is a table called artist which is created, and also a way to update the data with the relevant data. Expected Program can store artist key data in a persistent format Result: Pass
Database must be able to store the two spotifyIds and strength of connection between two artists	<pre> string createConnectionsTableQuery = @" CREATE TABLE IF NOT EXISTS Connections (SpotifyId1 TEXT NOT NULL REFERENCES Artist(SpotifyId), SpotifyId2 TEXT NOT NULL REFERENCES Artist(SpotifyId), Weight REAL NOT NULL);"; using (SqliteCommand cmd = new SqliteCommand(createConnectionsTableQuery, connection)) { cmd.ExecuteNonQuery(); } connection.Open(); string insertQuery = @" </pre>	Explanation: First code snippet from DatabaseRepository, Second code snippet from Connection, shows that there is a table called Connections which is

	<pre> INSERT INTO Connections (SpotifyId1, SpotifyId2, Weight) VALUES (@spotifyId1, @spotifyId2, @strength);"; using (SQLiteCommand cmd = new SQLiteCommand(insertQuery, connection)) { cmd.Parameters.AddWithValue("@spotifyId1", spotifyId1); cmd.Parameters.AddWithValue("@spotifyId2", spotifyId2); cmd.Parameters.AddWithValue("@strength", weight); cmd.ExecuteNonQuery(); } </pre>	created, and also a way to update the data with the relevant data Expected: a way to store the strenght of connection between two artists Result: Pass
Program must check whether there are any artists in the database who have not been searched for, and create a queue if there are	<pre> Queue<ArtistNode> queue = new Queue<ArtistNode>(); using (SQLiteConnection connection = new SQLiteConnection(_connectionString)) { connection.Open(); string selectQuery = @" SELECT ArtistName, SpotifyID FROM Artist WHERE isExpanded = 0"; using (SQLiteCommand cmd = new SQLiteCommand(selectQuery, connection)) { using (SQLiteDataReader reader = cmd.ExecuteReader()) { if (!reader.HasRows) throw new Exception("Queue is empty"); while (reader.Read()) { string artistName = reader.GetString(0); string spotifyId = reader.GetString(1); queue.Enqueue(new ArtistNode(artistName, spotifyId)); } } } } return queue; </pre>	Explanation: Code snippet creates a queue of all artists whose expanded column has not been updated to 1 (only done by the networkExpander) Expected: Code loads a queue of all the artists who hasn't been expanded Result: Pass
If no artists exist in database, at programmers discretion, choose an artist to enqueue to the searchQueue	<pre> Queue<ArtistNode> queue; try { queue = _networkExpanderDatabaseService.GetSearchQueue(); } catch (NullReferenceException ex) { Console.WriteLine(ex.Message); queue = new Queue<ArtistNode>(); queue.Enqueue(startingArtistNode); _networkExpanderDatabaseService.AddArtistToDb(startingArtistNode); queue = _networkExpanderDatabaseService.GetSearchQueue(); } </pre>	Explanation: Code snippet tries to check whether the search queue is empty or not, if it is , startingArtistNode (chosen when I run the program) is loaded as the first artist Expected: Code enqueues specified starting artist if there are no unexpanded artists in the database (database is empty) Result: Pass
For each artist in the queue program must find the similar verified (as per requirements earlier) artists and the corresponding strength of connection to that artist	<pre> Dictionary<ArtistNode, double> similarArtists = new Dictionary<ArtistNode, double>(); Dictionary<string, double> lastFmSimilarArtists = await _lastFmApiService.GetSimilarArtists(startingArtistNode.Name); foreach (var similarArtist in lastFmSimilarArtists) { string similarArtistName = similarArtist.Key; double similarArtistWeight = similarArtist.Value; try { List<string> lastFmTopTracks = await _lastFmApiService.GetTopTracks(similarArtistName); KeyValuePair<(string, string), int> spotifyArtistDetails = await _spotifyExpanderService.GetArtistDetails(similarArtistName); string spotifyId = spotifyArtistDetails.Key.Item1; string name = spotifyArtistDetails.Key.Item2; } catch { } } </pre>	Explanation: Uses Last.Fm api to find similar artists and corresponding weight to starting artist. Verifies artist data with artist verification service and rejects if there is anything wrong Adds each artist to the dictionary and returns dictionary

	<pre> int popularity = spotifyArtistDetails.Value; Dictionary<string, DateTime> spotifyTopTracks = await _spotifyExpanderService.GetTopTracks(spotifyId); try { _artistVerificationService.VerifyArtist(spotifyTopTracks, lastFmTopTracks, popularity); } catch (Exception ex) { Console.WriteLine(ex.Message); continue; } ArtistNode foundArtist = new ArtistNode(name, spotifyId); similarArtists[foundArtist] = similarArtistWeight; } return similarArtists; </pre>	Expected: Find artist and weight of connection between artist. Able to verify that each artist found is verified. Rejects any other artist. Result: Pass
Program must add these artists to the queue and add the connections between the starting artist and the found artist to the database, if it does not already exist	<pre> Dictionary<ArtistNode, double> similarArtists = await FindSimilarArtists(currentArtist); foreach (KeyValuePair<ArtistNode, double> similarArtist in similarArtists) { Console.WriteLine(\$"FOUND: {similarArtist.Key.SpotifyId} - {similarArtist.Key.Name}"); if (!_networkExpanderDatabaseService.IsArtistInDbById(similarArtis t.Key.SpotifyId)) { queue.Enqueue(similarArtist.Key); } _networkExpanderDatabaseService.AddArtistAndConnectionToDb(curr entArtist, similarArtist.Key, similarArtist.Value); } if (IsArtistInDbById(artist1.SpotifyId) && IsConnectionInDb(artist1.SpotifyId, artist2.SpotifyId)) { Console.ForegroundColor = ConsoleColor.Red; Console.WriteLine(\$"Artist {artist2.Name} is in artistNetworkDb and connection exists for {artist1.Name} -> {artist2.Name}"); Console.ResetColor(); } else if (IsArtistInDbById(artist2.SpotifyId)) { if (!IsArtistInDbById(artist1.SpotifyId)) { AddArtistToDb(artist1); } try { AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight); Console.ForegroundColor = ConsoleColor.Green; Console.WriteLine(\$"Added connection between {artist1.Name} and {artist2.Name}"); Console.ResetColor(); } catch (Exception ex) { throw new Exception(ex.Message); } } else { try { AddArtistToDb(artist2); AddConnectionToDb(artist1.SpotifyId, artist2.SpotifyId, weight); } catch (Exception ex) { throw new Exception(ex.Message); } Console.ForegroundColor = ConsoleColor.Green; Console.WriteLine(\$"Added {artist2.Name} to the artistNetworkDb and added a new connection between {artist1.Name} and {artist2.Name}"); Console.ResetColor(); } } </pre>	Explanation: Snippet one shows that program uses breadth first search queue to expand the network. enqueues all the artists found in FindSimilarArtists to the search queue Snippet two shows logic which checks whether artist is already in database and the connection between the two artists already exists (don't do anything), whether the artist already exists but the connection doesn't exist (add connection) and whether the artist does not exist in the database (add the artist and the connection to the database) Expected: Code can create a search queue which continuously enqueues artists. Adds relevant artist and connections to database. Doesn't add artists or connections to database which already exist Status: Pass

Program must load artists and connections from database	<pre>connection.Open(); string selectQuery = @" SELECT a1.ArtistName, c.SpotifyId1, a2.ArtistName, c.SpotifyId2, c.Weight FROM Connections c JOIN Artist a1 ON c.SpotifyId1 = a1.SpotifyId JOIN Artist a2 ON c.SpotifyId2 = a2.SpotifyId"; using (SQLiteCommand cmd = new SQLiteCommand(selectQuery, connection)) { using (SQLiteDataReader reader = cmd.ExecuteReader()) { while (reader.Read()) { string artistName1 = reader.GetString(0); string spotifyId1 = reader.GetString(1); ArtistNode artistNode1 = new ArtistNode(artistName1, spotifyId1); string artistName2 = reader.GetString(2); string spotifyId2 = reader.GetString(3); ArtistNode artistNode2 = new ArtistNode(artistName2, spotifyId2); double weight = reader.GetDouble(4); if (!artistNetwork.ContainsKey(artistNode1)) { artistNetwork.Add(artistNode1, new Dictionary<ArtistNode, double>()); } if (!artistNetwork.ContainsKey(artistNode2)) { artistNetwork.Add(artistNode2, new Dictionary<ArtistNode, double>()); } artistNetwork[artistNode1][artistNode2] = weight; artistNetwork[artistNode2][artistNode1] = weight; } } connection.Open(); private readonly Dictionary<ArtistNode, Dictionary<ArtistNode, double>> _adjacencyMatrix; public ArtistNetwork(IArtistNetworkDatabaseService artistNetworkDatabaseService) { _adjacencyMatrix = new Dictionary<ArtistNode, Dictionary<ArtistNode, double>>(); _adjacencyMatrix = artistNetworkDatabaseService.GetNetwork(); Console.WriteLine("Finished loading the artist network."); } }</pre>	Explanation: Snippet one shows that code uses an sql join to load a 2D Dictionary from the database which represents the network. Expected: Objective 1 Code loads network from database Objective 2 Code represents network as a 2D Dictionary Status: Pass
Program must represent the network as a 2D Dictionary of structure Dictionary(Artist, Dictionary(Artist, Weight))		
Artist object has hashing method which allows O(1) look up time	<pre>public readonly string Name = name; public readonly string SpotifyId = spotifyId; public override bool Equals(object? obj) { if (obj is ArtistNode otherNode) { return SpotifyId == otherNode.SpotifyId; } return false; } public override int GetHashCode() { return SpotifyId.GetHashCode(); }</pre>	Explanation: Snippet shows ArtistNode object which is used as the key to the dictionary with a modified equals and gethascode function to depend on the spotifyId so that the dictionary can have O(1) Lookup time for keys. Expected: Dictionary has keys which allow O(1) lookup time Status: Pass
Program must be able to use Dijkstras algorithm to find the shortest path between two artists	<pre>PriorityQueue<ArtistNode, double> priorityQueue = new PriorityQueue<ArtistNode, double>(); Dictionary<ArtistNode, double> distances = new Dictionary<ArtistNode, double>(); Dictionary<ArtistNode, ArtistNode> previous = new Dictionary<ArtistNode, ArtistNode>(); List<ArtistNode> Path = new List<ArtistNode>(); foreach (ArtistNode artist in _adjacencyMatrix.Keys) { distances[artist] = double.MaxValue; previous[artist] = null; }</pre>	Explanation: Code snippet shows use of dijkstras algorithm to find the shortest path between any two artists

	<pre> distances[artist] = double.MaxValue; previous[artist] = null; } distances[startArtistNode] = 0; priorityQueue.Enqueue(startArtistNode, 0); while (priorityQueue.Count > 0) { ArtistNode currentArtist = priorityQueue.Dequeue(); double currentDistance = distances[currentArtist]; if (currentArtist.SpotifyId == endArtistNode.SpotifyId) break; foreach (KeyValuePair<ArtistNode, double> connection in _adjacencyMatrix[currentArtist]) { ArtistNode neighbour = connection.Key; double newDist = currentDistance + connection.Value; if (newDist < distances[connection.Key]) { distances[neighbour] = newDist; previous[neighbour] = currentArtist; priorityQueue.Enqueue(neighbour, newDist); } } } ArtistNode pathArtistNode = endArtistNode; while (pathArtistNode != null) { Path.Insert(0, pathArtistNode); pathArtistNode = previous[pathArtistNode]; } return Path; </pre>	Expected: Dijkstras algorithm implemented to find the shortest path between two artists Status: Pass
Program must use the spotify data to search for if two artists have a shared song	<pre> string query = Uri.EscapeDataString(\$"{artist1.Name} {artist2.Name}"); string url = \$"https://api.spotify.com/v1/search?q={query}&type=track&limit= 10"; HttpResponseMessage response = await _httpClient.GetAsync(url); response.EnsureSuccessStatusCode(); string jsonResponse = await response.Content.ReadAsStringAsync(); JsonDocument document = JsonDocument.Parse(jsonResponse); if (!document.RootElement.TryGetProperty("tracks", out var tracks) !tracks.TryGetProperty("items", out var items)) { Console.WriteLine(\$"No tracks found in search response for {artist1.Name} and {artist2.Name}"); return null; } foreach (JsonElement track in items.EnumerateArray()) { var artists = track.GetProperty("artists").EnumerateArray(); bool hasArtist1 = artists.Any(a => string.Equals(a.GetProperty("name").GetString(), artist1.Name, StringComparison.OrdinalIgnoreCase)); bool hasArtist2 = artists.Any(a => string.Equals(a.GetProperty("name").GetString(), artist2.Name, StringComparison.OrdinalIgnoreCase)); int artistCount = track.GetProperty("artists").GetArrayLength(); if (hasArtist1 && hasArtist2 && artistCount == 2) { string trackName = track.GetProperty("name").GetString(); string songId = track.GetProperty("id").GetString(); string normalizedName = NormalizeSongName(trackName); Console.WriteLine(\$"Found Feature: {trackName}"); return (songId, normalizedName); } } </pre>	Explanation: spotify data v1 search endpoint is used to search whether two artists have a shared song by querying endpoint with name of two artists. returns list of songs which have both artists without any other artists. Expected; finds whether two artists have a shared song together and finds the key details about that song. Result: Pass
Program must reject any song which has more than the two artists		

Program must use the spotify data to find the top 10 popular songs for an artist	<pre> List<string> songIds = new List<string>(); List<string> songNames = new List<string>(); songNames.AddRange(previousSongNames); try { _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", accessToken); string url = \$"https://api.spotify.com/v1/artists/{artist.SpotifyId}/top- tracks"; Console.WriteLine(url); HttpResponseMessage response = await _httpClient.GetAsync(url); response.EnsureSuccessStatusCode(); string jsonResponse = await response.Content.ReadAsStringAsync(); JsonDocument document = JsonDocument.Parse(jsonResponse); var topSongs = document.RootElement.GetProperty("tracks").EnumerateArray(); Dictionary<string, string> potentialSongs = new Dictionary<string, string>(); foreach (JsonElement song in topSongs) { int artistCount = song.GetProperty("artists").GetArrayLength(); bool hasFeatures = artistCount > 1; if (!hasFeatures) { string songId = song.GetProperty("id").GetString(); string trackName = song.GetProperty("name").GetString(); string normalizedSongName = NormalizeSongName(trackName); bool isDuplicate = songNames.Any(existingName => existingName.Contains(\$" {normalizedSongName} ") normalizedSongName.Contains(\$" {existingName} ") existingName.Equals(normalizedSongName)); if (!isDuplicate) { potentialSongs.Add(songId, normalizedSongName); } } } if (potentialSongs.Count > 0) { Random random = new Random(); var selectedSongs = potentialSongs .OrderBy(x => random.Next()) .Take(tracksPerArtist) .ToList(); foreach (var song in selectedSongs) { songIds.Add(song.Key); songNames.Add(song.Value); } } return (songIds, songNames); </pre>	Explanation: finds the required number of songs for an artist by querying the spotify api get-top-tracks method to find the top tracks of an artist. Chooses at random the number of tracks needed. Expected: Finds required number of songs for artist from artists top track s Result: Pass
Songs must not have any other artists on it		
For each feature that has been found for an adjacent artist, take one away from the total number of songs to be found	<pre> var featureResult = await FindFeature(pathOfArtists[i], pathOfArtists[i + 1], accessToken); if (featureResult != null) { string featureId = featureResult.Value.id; string featureName = featureResult.Value.name; songsPerArtist[i]--; songsPerArtist[i + 1]--; var artistSongs = await FindSongsForArtist(pathOfArtists[i], songsPerArtist[i], songNames, accessToken); songIds.AddRange(artistSongs.Item1); songNames.AddRange(artistSongs.Item2); songIds.Add(featureId); songNames.Add(featureName); } </pre>	Explanation: Code is called when feature has been found. Removes one of the number of songs which is being searched for Expected: Only 3 actual songs for each artist Result: Pass
Program must use user authentication	<pre> private readonly string _clientId; private readonly string _redirectUri; private readonly ConcurrentDictionary<string, string> _codeVerifiers = new(); private readonly HttpClient _httpClient; </pre>	Explanation: Code snippet shows logic for

with PKCE to access
private scopes

```
public SpotifyPkceCodeAuthorizer(HttpClient httpClient, string
clientId, string redirectUri)
{
    _httpClient = httpClient;
    _clientId = clientId;
    _redirectUri = redirectUri;
}

public async Task<(string AuthUrl, string State)>
GetAuthorizationUrl()
{
    string codeVerifier = GenerateCodeVerifier();
    string codeChallenge = GenerateCodeChallenge(codeVerifier);
    string state = GenerateState();

    _codeVerifiers.TryAdd(state, codeVerifier);

    string authUrl =
$"https://accounts.spotify.com/authorize?response_type=code&cli
ent_id={_clientId}" +

$"&redirect_uri={Uri.EscapeDataString(_redirectUri)}&code_chall
enge_method=S256" +

$"&code_challenge={codeChallenge}&scope=playlist-modify-
public&state={state}";

    return (authUrl, state);
}

public async Task<string> ExchangeCode(string code, string
state)
{
    if (!_codeVerifiers.TryRemove(state, out string
codeVerifier))
        throw new Exception("Invalid state or code verifier not
found.");

    return await ExchangeAuthorizationCodeForAccessToken(code,
codeVerifier);
}

private async Task<string>
ExchangeAuthorizationCodeForAccessToken(string code, string
codeVerifier)
{
    HttpResponseMessage response = await
_httpClient.PostAsync("https://accounts.spotify.com/api/token",
new FormUrlEncodedContent(new[]
{
    new KeyValuePair<string, string>("grant_type",
"authorization_code"),
    new KeyValuePair<string, string>("code", code),
    new KeyValuePair<string, string>("redirect_uri",
_redirectUri),
    new KeyValuePair<string, string>("client_id",
_clientId),
    new KeyValuePair<string, string>("code_verifier",
codeVerifier)
})));

    if (!response.IsSuccessStatusCode)
        throw new Exception($"Failed to exchange authorization
code. Status: {response.StatusCode}");

    string jsonResponse = await
response.Content.ReadAsStringAsync();
    JsonDocument document = JsonDocument.Parse(jsonResponse);
    return
document.RootElement.GetProperty("access_token").GetString();
}

private string GenerateCodeVerifier()
{
    byte[] bytes = new byte[32];
    using RandomNumberGenerator rng =
RandomNumberGenerator.Create();
    rng.GetBytes(bytes);
    return Convert.ToBase64String(bytes)
        .Replace("+", "-")
        .Replace("/", "_")
        .Replace("=", "");
}

private string GenerateCodeChallenge(string codeVerifier)
{
    using SHA256 sha256 = SHA256.Create();
    byte[] bytes =
sha256.ComputeHash(Encoding.UTF8.GetBytes(codeVerifier));
    return Convert.ToBase64String(bytes)
        .Replace("+", "-")
        .Replace("/", "_")
        .Replace("=", "");
}

private string GenerateState()
{

```

handling PKCE
authentication
Expected : Code can
handle PKCE
authentication flow
Result: Pass

	<pre> byte[] bytes = new byte[32]; using RandomNumberGenerator rng = RandomNumberGenerator.Create(); rng.GetBytes(bytes); return Convert.ToBase64String(bytes) .Replace("+", "-") .Replace("/", "_") .Replace("=", ""); } </pre>	
Playlist must have name “from __ to __” where the gaps are the first and last artists the user entered respectively	<pre> string userId = await GetUserId(accessToken); try { var request = new HttpRequestMessage(HttpMethod.Post, \$"https://api.spotify.com/v1/users/{userId}/playlists"); var content = new { name = \$"from {firstArtist} to {lastArtist}", description = \$"Creates a playlist that transitions between" + \$" {firstArtist} and {lastArtist}.", @public = true }; request.Content = new StringContent(JsonSerializer.Serialize(content), Encoding.UTF8, "application/json"); request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken); var response = await _httpClient.SendAsync(request); if (!response.IsSuccessStatusCode) { throw new HttpRequestException(\$"Error creating playlist: {response.StatusCode}"); } string jsonResponse = response.Content.ReadAsStringAsync().Result; Console.WriteLine(\$"Playlist created successfully: {jsonResponse}"); JsonDocument document = JsonDocument.Parse(jsonResponse); var playlistId = document.RootElement.GetProperty("id").GetString(); Console.WriteLine(\$"Playlist ID: {playlistId}"); return playlistId; } var request = new HttpRequestMessage(HttpMethod.Post, \$"https://api.spotify.com/v1/playlists/{playlistId}/tracks"); request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken); var requestBody = new { uris = songIds.Select(id => \$"spotify:track:{id}").ToArray() }; request.Content = new StringContent(JsonSerializer.Serialize(requestBody), Encoding.UTF8, "application/json"); try { HttpResponseMessage response = await _httpClient.SendAsync(request); if (!response.IsSuccessStatusCode) { throw new Exception(\$"Failed to add tracks to the playlist {response.StatusCode}: {response.ReasonPhrase}."); } } </pre>	Explanation: Code snippet clearly shows all playlists will be created with given parameters Expected: All playlists have the name from_ to __ where the gaps are the names of the path Result: Pass
Playlist must be added automatically to user library with the songs found for each artist added in order	<pre> request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken); var response = await _httpClient.SendAsync(request); if (!response.IsSuccessStatusCode) { throw new HttpRequestException(\$"Error creating playlist: {response.StatusCode}"); } string jsonResponse = response.Content.ReadAsStringAsync().Result; Console.WriteLine(\$"Playlist created successfully: {jsonResponse}"); JsonDocument document = JsonDocument.Parse(jsonResponse); var playlistId = document.RootElement.GetProperty("id").GetString(); Console.WriteLine(\$"Playlist ID: {playlistId}"); return playlistId; } var request = new HttpRequestMessage(HttpMethod.Post, \$"https://api.spotify.com/v1/playlists/{playlistId}/tracks"); request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken); var requestBody = new { uris = songIds.Select(id => \$"spotify:track:{id}").ToArray() }; request.Content = new StringContent(JsonSerializer.Serialize(requestBody), Encoding.UTF8, "application/json"); try { HttpResponseMessage response = await _httpClient.SendAsync(request); if (!response.IsSuccessStatusCode) { throw new Exception(\$"Failed to add tracks to the playlist {response.StatusCode}: {response.ReasonPhrase}."); } } </pre>	Explanation: Code snippet clearly shows the path of songs being added to the playlist of the user Expected: Program can add songs to a playlist in the user library Result: Pass

Evaluation

Objectives Self Evaluation

Main Objective	Has this objective been met?	Comments
1. The program must have a way to expand network of artists given a starting artist and store it in a persistent manner	YES	Yes, I have managed to do this with my network expander and database classes. My artist database is persistent
2. Program must have a way to find the shortest path between any two artists	YES	Yes, I have done this by using dijkstras algorithm, which finds the shortest path very well between any two artists
3. Program must have a way to find songs for each artist in the path of artists a playlist in the user library	YES	Yes, I have managed to do this get songs classes. It can find 3 songs for each artist and finds features along the path for each artist
4. Program must have a way to create a playlist in the user library	YES	Yes, through my pkce authentication and create playlists class, I have managed to do this
5. Program must take the user through the process from authentication to playlist creation	YES	Yes, I have designed a website which I am able to tell the user what needs to be entered at which point.
6. Website design must follow client and shareholder requirements	YES	Yes, I have consistently used gotham bold and black where decided in my user interviews

User Feedback

Playlist transition feedback

To collect end user feedback, I surveyed my shareholders to test playlist combinations and give feedback. As I have not deployed my website yet, I created the playlists on my account

Name	Playlist Requested	Feedback + suggestions	Rating of transition	Would you recommend? If not why?
Mikel Antonio	Destroy Lonely -> Jhene Aiko	Its good. I would suggest having a way to change the length of the playlist	7/10	Yes, definetly
Jamal Munene	Talib Kweli -> Redveil	Wow its really good. Don't have any suggestions. The transitions were really good	9/10	Yes I would recommend
Seth Rodrigo	Nettspend – Cleo Sol	Its quite good I'm impressed but I feel like there was a huge jump for Cash Cobain. I wish he wasn't in the playlist and also he had too many songs	6/10	If you added a way to change the number fo songs for each artist then I think it would be better
Shobhit Taneja	From SZA to Emei	Its quite good but it goes through some weird artists to get there. I think there couldve been a more direct route	5/10	I think the logic for traversal needs to be changed tbh its ok but could be better
Abby English	Mitski -> Playboi Carti	Its very good. I listened to it over a few days and I was surprised how well it was transitioning. I think a few artists could've had a few less songs but apart from that it is very good	9/10	Yes definielley
Lorna Smith	Drake -> Osamason	I think there was a point where there was an artist who didn't really make sense, i think it kind of ruined the experience.	5/10	I think if yuou could make it so that there was a minimum similarity requirement that would be good
James Milnor	Gucci Mane -> Don Tolliver	It was very good. I tihnk it transioend really wel and opened my eyes to some very good artists	10/10	Yes

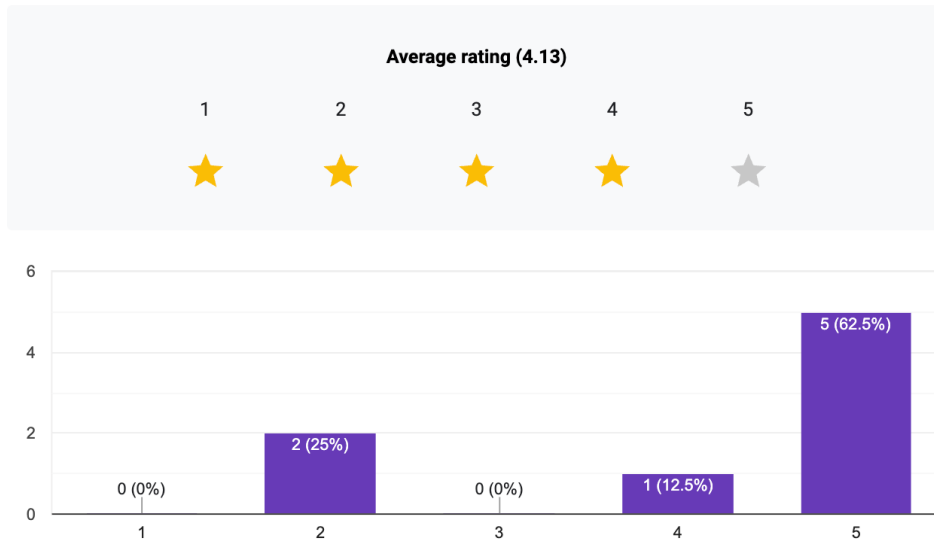
Design Survey feedback

I gave the website to a sample of shareholders for their quick feedback on the website

How would you rate the layout of the website compared to your expectations

[Copy chart](#)

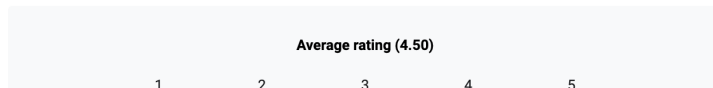
8 responses



How would you rate the responsiveness fo the website compared to your expectations

[Copy chart](#)

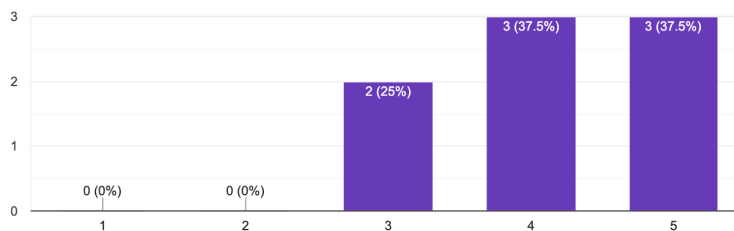
8 responses



How likely are you to recommend the website to a friend

[Copy chart](#)

8 responses



Interview with main client

I took my main client through the process of choosing artists to find the path between. They tested with a variety of artists without my guidance. Here is the structured interview I conducted to hear their opinions:

Q: First off, how would you rate the design of the website? Is it satisfactory to your standards?

A: Yeah, overall, the design is really solid. The color scheme looks great, and it definitely has a modern feel to it. The only issue I ran into was on the artist page—scrolling felt a bit awkward because it seemed like I had to refresh sometimes to get things to work smoothly.

Q: What about the responsiveness of the website? How smooth was it when performing actions?

A: The site was very responsive. I never really noticed any major lag, aside from the obvious processing time when generating the playlist. Everything else felt smooth and quick.

Q: What about user input? How was that?

A: At desktop size, there was no major issues at all, it worked as expected. However, when I resized the website to mobile size, it became unresponsive and harder to type into the text box. Also the headers at the top became a bit unresponsive

Q: Just looking at the artist names, would you say the transitions made sense?

A: For the most part, yes, but there were definitely a few weird jumps. It also seems like some artists act as "gateways" between certain genres, which can lead to some odd transitions. For example, when moving from R&B to underground hip-hop, there always seems to be a jump from Travis Scott -> Jackboys -> Mikeeysmind -> Yeat. The issue is that Mikeeysmind and Yeat don't really match at all, which makes that transition feel a bit off. Might be worth looking into.

Q: How was the login process? Did everything work as expected?

Yeah, no problems there, it worked perfectly fine.

Q: Finally, after listening to the playlist, what do you think of the transitions between the two artists?

Overall, they're pretty good. Some transitions feel really natural, but there are a few spots where the jumps feel a bit abrupt, like the mikeeysmind one I said before, or sometimes cash cobain is a crux point.

Q: Any other miscellaneous feedback

I think the user should have a bit more customization over whether the playlist is public or private in their library. Also it would be cool if the playlist image was a bit linked to the first and last artist!

Improvements + Final Conclusion

After collecting feedback from my clients, here are some things I would change if I did again:

Change the network expansion algorithm to filter out artists who are weakly connected

One big thing that was suggested is that there are few artists who seem to be too weakly connected. I think I would change my network expansion algorithm so that it would try to get more than 5 artists but filter out any artists with a low weight score, instead of an even 5 similar artists per artist

Change the network expansion algorithm to be automatic

I think I would change my network expansion algorithm to work instead of being manually run by me every 36-48 hours into a background process which runs until spotify rate limits and restarts automatically after 36-48 hours

Allow the user to change the number of songs per artist

I would change the flow of user input so that it allows the user to change the number of songs per artist. This allows the user a wider range of customization

Allow the user to change the minimum strength of connection/How important the connections are

I would give the user the ability to say how much the strength of connection is factored into the path finding algorithm. Maybe by timesing the weight by itself to make the difference between strong and weak connections bigger.

Allow the user to choose whether the playlist will be private or public

I would give the user the ability to choose whether the playlist is private or public. This would be aske before the authentication starts so that only the relevant scope is granted to the application

Make a full responsive mobile website

Currently I have a desktop website which resizes to mobile relatively well, but I would make a mobile site which would put the menu into a hidden corner to not take up too much space.

Change the playlist cover image

The playlist cover image could be the spotify profile pictures of the two artists, with an arrow connecting them

Conclusion

All in all, I think I have managed to solve the gap in the market for my clients and my shareholders. This is a personal issue which interests me, and I think I would use this website very often when curious about finding new artists between any two artists.

References

W3Schools (n.d.) Dijkstra's algorithm. Available at:

https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php.

GeeksforGeeks (n.d.) Introduction to Dijkstra's shortest path algorithm. Available at:

<https://www.geeksforgeeks.org/introduction-to-dijkstras-shortest-path-algorithm/>.

FreeCodeCamp (n.d.) Dijkstra's algorithm explained with a pseudocode example. Available at:

<https://www.freecodecamp.org/news/dijkstras-algorithm-explained-with-a-pseudocode-example/>.

Microsoft (n.d.) SQLite in .NET. Available at: [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/?tabs=net-cli)

[us/dotnet/standard/data/sqlite/?tabs=net-cli](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/?tabs=net-cli).

Microsoft (n.d.) SQLite types in .NET. Available at: [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/types)

[us/dotnet/standard/data/sqlite/types](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/types).

Microsoft (n.d.) SQLite parameters in .NET. Available at: [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/parameters)

[us/dotnet/standard/data/sqlite/parameters](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/parameters).

Microsoft (n.d.) SQLite database errors in .NET. Available at: [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/database-errors)

[us/dotnet/standard/data/sqlite/database-errors](https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/database-errors).

Spotify (n.d.) Web API documentation. Available at:

<https://developer.spotify.com/documentation/web-api>.

Spotify (n.d.) Authorization in the Spotify Web API. Available at:

<https://developer.spotify.com/documentation/web-api/concepts/authorization>.

Spotify (n.d.) Spotify Web API scopes. Available at:

<https://developer.spotify.com/documentation/web-api/concepts/scopes>.

Spotify (n.d.) Authorization code with PKCE flow. Available at:

<https://developer.spotify.com/documentation/web-api/tutorials/code-pkce-flow>.

Spotify (n.d.) Client credentials flow. Available at:

<https://developer.spotify.com/documentation/web-api/tutorials/client-credentials-flow>.

YouTube (n.d.) OAuth PKCE | OAuth Proof Key for Code Exchange explained X [Video]. Available

at: https://www.youtube.com/watch?v=h_1JA3JPKl.

YouTube (n.d.) How to Authenticate and use Spotify Web API. Available at:

<https://www.youtube.com/watch?v=1vR3m0HupGI>.

Last.fm (n.d.) Get similar artists API. Available at:

<https://www.last.fm/api/show/artist.getSimilar>.

Last.fm (n.d.) API authentication guide. Available at: <https://www.last.fm/api/authentication>.

tutorialsEU - C# (2023). *Using JSON IN C#! Serialization & Deserialization made easy!* [online] YouTube. Available at: <https://www.youtube.com/watch?v=w6M-Bj-tfv4>

Stack Overflow (n.d.) Correct way to override equals and gethashcode. Available at: <https://stackoverflow.com/questions/9317582/correct-way-to-override-equals-and-gethashcode>

Questpond (2011). Regular Expressions in C# with Practical Demonstration | C# Tutorial for Beginners | Regex in CSharp. [online] YouTube. Available at: <https://www.youtube.com/watch?v=C2zm0roE-Uc>

Microsoft (n.d.) C# async keyword documentation. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/async>

Microsoft (n.d.) Building your first ASP.NET Core Web API. Available at: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api?view=aspnetcore-9.0&tabs=visual-studio>.

YouTube (n.d.) Building a RESTful API in C Sharp [Video]. Available at: <https://www.youtube.com/watch?v=Tj3qsKSNvMk>.

www.youtube.com. (n.d.). dialog = the easiest way to make a popup modal. [online] Available at: https://www.youtube.com/watch?v=TAB_v6yBXIE

Spotify (n.d.) Playlists in the Spotify Web API. Available at: <https://developer.spotify.com/documentation/web-api/concepts/playlists>.

Spotify (n.d.) Creating a playlist via the Web API. Available at: <https://developer.spotify.com/documentation/web-api/reference/create-playlist>.

Spotify (n.d.) Adding tracks to a playlist. Available at: <https://developer.spotify.com/documentation/web-api/reference/add-tracks-to-playlist>.

Microsoft (n.d.) Regular expressions in .NET. Available at: <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions>.

Microsoft (n.d.) Dependency injection in ASP.NET Core. Available at: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-9.0>.

Microsoft (n.d.) CORS in ASP.NET Core. Available at: <https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-9.0>.

W3Schools (n.d.) How to create modals in CSS. Available at: https://www.w3schools.com/howto/howto_css_modals.asp.

W3Schools (n.d.) Parsing JSON in JavaScript. Available at: https://www.w3schools.com/js/js_json_parse.asp

www.youtube.com. (n.d.). *Breadth-first search in 4 minutes*. [online] Available at:
<https://www.youtube.com/watch?v=HZ5YTanv5QE>

Gaur Associates (2016). *C# breadth first and depth first search in 5 min*. [online] YouTube.
Available at: https://www.youtube.com/watch?v=XU_ugVTSjSE

Authgear (n.d.) Using the OAuth 2.0 state parameter. Available at:
https://docs.authgear.com/how-to-guide/authenticate/how-to-use-the-oauth-2.0-state-parameter?utm_source=chatgpt.com.

Brandcolorcode.com. (2025). Coached by ContentKeeper. [online] Available at:
<https://www.brandcolorcode.com/spotify>

developer.mozilla.org. (n.d.). *Element.innerHTML - Web APIs | MDN*. [online] Available at:
<https://developer.mozilla.org/enUS/docs/Web/API/Element/innerHTML>.

dotnet-bot (2018). *DateTime.Parse Method (System)*. [online] Microsoft.com. Available at:
<https://learn.microsoft.com/en-us/dotnet/api/system.datetime.parse?view=net-9.0>

tdykstra (2023). *.NET SDK error list - .NET CLI*. [online] Microsoft.com. Available at:
<https://learn.microsoft.com/en-us/dotnet/core/tools/sdk-errors>